Escuela Técnica Superior de
Ingeniería Informática

Departamento de Lógica, Computación
e Inteligencia Artificial

Proyecto fin de máster

Modelos predictivos para ocupación de estacionamiento

Autor:
Ing. Germán Lorenz Vieta

Tutor:
Ignacio Pérez Hurtado de Mendoza

Sevilla, Viernes 7 de Junio de 2024

Agradecimientos

Quiero expresar mi más profundo agradecimiento a todas las personas y organizaciones que, de forma directa o indirecta, han hecho posible la realización de esta investigación. A mis mentores y profesores, por su invaluable guía y apoyo desde mis primeros años en ingeniería hasta la actualidad. A mi familia y amigos, por su constante ánimo y comprensión para afrontar estos desafíos, y a mis compañeros de grado y máster, con quienes he compartido innumerables horas de estudio durante todos estos años. También agradezco a la institución por proporcionar los recursos necesarios que, de otra forma, no habrían sido posibles para realizar este Trabajo de Fin de Máster.

Índice general

Lista de figuras **IX**

1. Introducción **1**

Introducción **1**

1. Motivación 1
2. Objetivo 2
3. Estructura de la Memoria 3

2. Marco Teórico **5**

1. Aprendizaje Automático 5
   - 1.1. Expectativa 6
   - 1.2. Función de pérdida o error 6
   - 1.3. Sesgo 7
   - 1.4. Varianza 7
   - 1.5. Sobreajuste (Overfitting) 8
   - 1.6. Subajuste (Underfitting) 9
   - 1.7. Descomposición del Sesgo, la Varianza y el Ruido 9
   - 1.8. Regularización 12
2. Aprendizaje Profundo 13

2.1. Redes Neuronales 14
2.2. Redes Neuronales Convolucionales 17
2.3. Redes Neuronales Recurrentes 22

3. Estado de arte **29**

4. Desarrollo del trabajo **33**

1. Introducción 33
2. Conjunto de Datos 33
2.1. Descripción del Conjunto de Datos 33
2.2. Estructura de datos 34
2.3. Análisis de distribución de los datos 35
2.4. Análisis de distribución de los datos de camara PUC . . 39
2.5. Análisis de las imágenes 41
2.6. Análisis temporal de la ocupación de estacionamiento . . 46
3. Investigación y selección de modelos Convolucionales 57
3.1. mLeNet (2014) 57
3.2. mAlexNet (2014) 58
3.3. VGG (2014) 59
3.4. ResNet50 (2015) 61
3.5. InceptionV3 (2015) 63
3.6. MobileNetV2 (2018) 66
3.7. MobileNetV3 (2019) 68
3.8. MobileNetV3 Small personalizada (2022) 69
4. Investigación y selección de modelos de Series Temporales 72
4.1. Modelo LSTM 72

4.2. Modelo GRU 74
4.3. Modelo CNN-1D 74
5. Estructura de la Información 76
5.1. Estructura de la Información y Segmentación de Imágenes 77
5.2. Construcción del DataFrame 78
6. Construcción de modelos e integración de características 80
6.1. Construcción y entrenamiento de CNNs 80
6.2. Integración de Características 81
6.3. Construcción y entrenamiento de RNNs 82

5. Resultados experimentales **85**

1. Evaluación de Modelos CNNs 85
1.1. Resultados de entrenamiento de CNNs 86
2. Evaluación de Modelos de Series Temporales 91
2.1. Resultados de los Modelos de series temporales 94

6. Conclusiones y líneas de trabajo futuro **125**

1. Introducción 125
2. Resumen de conclusiones 126
2.1. Sintonía entre mAlexNet y RNNs 126
2.2. Alternativas Livianas 126
2.3. Posibles Mejoras 127
3. Trabajo Futuro 128
4. Reflexiones Finales 129

7. Anexos **131**

1. Tablas de evaluaciones de Modelos RNNs 131

Bibliografía **137**

Índice de figuras

2.1. Relación entre error y complejidad del modelo para un conjunto de datos de entrenamiento fijo 8

2.2. Representación del error en función de la cantidad de datos de entrenamiento. 12

2.3. Ejemplo de Modelo multicapa con capa oculta 14

2.4. Gráfica de funciones de Activación 16

2.5. Ejemplo de aplicación de filtro a la entrada en una CNN. 19

2.6. Ejemplo de aplicación de Max Pooling con distintos Stride (pasos) 21

2.7. Diagrama de Arquitectura LSTM 23

2.8. Diagrama de Arquitectura GRU 25

2.9. Ejemplo de Arquitectura CNN-1D con múltiples capas y salida Densa para ser decidible 27

4.1. Diagrama de flujo que representa estrategia típica de evaluación de modelos CNN y RNN iterativa 34

4.2. Ocupación creciente en la cámara PUC. En la columna a la izquierda 20-09-2012 (soleado), en la central 28-10-2012 (nublado) y en la columna a la derecha 16-09-2012 (lluvioso) 35

4.3. Imágenes segmentadas del día 27-10-2012 (soleado), diferenciadas entre espacios ocupados (izquierda) y vacíos (derecha). 36

4.4. Análisis de Distribución de Clases del conjunto PKLot 38

4.5. Análisis de Distribución de Clases en PUC del Conjunto PKLot 40

4.6. Análisis de imágenes de cámara PUC. Alto, ancho, área, brillo, contraste y entropía. 43

4.7. Análisis de imágenes de cámara PUC. Haralick 1 a 10 45

4.8. Análisis de imágenes de cámara PUC. Haralick 11 a 13 46

4.9. Ocupación y disponibilidad del estacionamiento durante el 11-09-2012 a 20-09-2012 49

4.10. Ocupación y disponibilidad del estacionamiento durante el 21-09-2012 a 17-10-2012 50

4.11. Ocupación y disponibilidad del estacionamiento durante el 25-10-2012 a 08-11-2012 51

4.12. Ocupación y disponibilidad del estacionamiento durante el 09-11-2012 a 20-11-2012 52

4.13. Representación visual de la ocupación media de los estacionamientos por hora a lo largo de cada día de la semana 53

4.14. Mapa de Calor de correlaciones entre espacios de estacionamientos 54

4.15. Imagen obtenida de PKLot del 11-09-2012 a las 18:18:05 con indicadores de distintos espacios con sus cinco espacios mas correlacionados 56

4.16. Porcentaje de ocupación total por espacios de estacionamiento . 57

5.1. Metricas de entrenamiento de mLeNet 88

5.2. Metricas de entrenamiento de mAlexNet 88

5.3. Metricas de entrenamiento de VGG19 89

5.4. Metricas de entrenamiento de ResNet 89

5.5. Metricas de entrenamiento de InceptionV3 90

5.6. Metricas de entrenamiento de MobileNetV2 90

5.7. Metricas de entrenamiento de MobileNetV3 92

5.8. Metricas de entrenamiento de MobileNetV3Small 92

5.9. Métricas RMSE y MAE de RNNs en Validación con mLeNet . . 95

5.10. Métricas R2 y Precisión 5 min de RNNs en Validación con mLeNet 95

5.11. Métricas Precisión 15 min y Precisión 30 min de RNNs en Validación con mLeNet 96

5.12. Métricas Precisión 45 min y Precisión 60 min de RNNs en Validación con mLeNet 96

5.13. Métricas RMSE y MAE de RNNs en Prueba con mLeNet 98

5.14. Métricas R2 y Precisión 5 min de RNNs en Prueba con mLeNet 98

5.15. Métricas Precisión 15 min y Precisión 30 min de RNNs en Prueba con mLeNet 99

5.16. Métricas Precisión 45 min y Precisión 60 min de RNNs en Prueba con mLeNet 99

5.17. Métricas Precisión Promedio (Validación izquierda, Prueba Derecha) con mLeNet 100

5.18. Métricas RMSE y MAE de RNNs en Validación con mAlexNet . 100

5.19. Métricas R2 y Precisión 5 min de RNNs en Validación con mAlexNet 101

5.20. Métricas Precisión 15 min y Precisión 30 min de RNNs en Validación con mAlexNet 101

5.21. Métricas Precisión 45 min y Precisión 60 min de RNNs en Validación con mAlexNet 103

5.22. Métricas RMSE y MAE de RNNs en Prueba con mAlexNet . . 103

5.23. Métricas R2 y Precisión 5 min de RNNs en Prueba con mAlexNet 104

5.24. Métricas Precisión 15 min y Precisión 30 min de RNNs en Prueba con mAlexNet 104

5.25. Métricas Precisión 45 min y Precisión 60 min de RNNs en Prueba con mAlexNet 105

5.26. Métricas Precisión Promedio (Validación izquierda, Prueba Derecha) con mAlexNet 105

5.27. Métricas RMSE y MAE de RNNs en Validación con InceptionV3 107

5.28. Métricas R2 y Precisión 5 min de RNNs en Validación con InceptionV3 107

5.29. Métricas Precisión 15 min y Precisión 30 min de RNNs en Validación con InceptionV3 108

5.30. Métricas Precisión 45 min y Precisión 60 min de RNNs en Validación con InceptionV3 108

5.31. Métricas RMSE y MAE de RNNs en Prueba con InceptionV3 . 109

5.32. Métricas R2 y Precisión 5 min de RNNs en Prueba con InceptionV3 109

5.33. Métricas Precisión 15 min y Precisión 30 min de RNNs en Prueba con InceptionV3 110

5.34. Métricas Precisión 45 min y Precisión 60 min de RNNs en Prueba con InceptionV3 110

5.35. Métricas Precisión Promedio (Validación izquierda, Prueba Derecha) con InceptionV3 112

5.36. Métricas RMSE y MAE de RNNs en Validación con MobileNetV2 112

5.37. Métricas R2 y Precisión 5 min de RNNs en Validación con MobileNetV2 113

5.38. Métricas Precisión 15 min y Precisión 30 min de RNNs en Validación con MobileNetV2 113

5.39. Métricas Precisión 45 min y Precisión 60 min de RNNs en Validación con MobileNetV2 115

5.40. Métricas RMSE y MAE de RNNs en Prueba con MobileNetV2 . 115

5.41. Métricas R2 y Precisión 5 min de RNNs en Prueba con MobileNetV2 116

5.42. Métricas Precisión 15 min y Precisión 30 min de RNNs en Prueba con MobileNetV2 116

5.43. Métricas Precisión 45 min y Precisión 60 min de RNNs en Prueba con MobileNetV2 117

5.44. Métricas Precisión Promedio (Validación izquierda, Prueba Derecha) con MobileNetV2 117

5.45. Métricas RMSE y MAE de RNNs en Validación con MobileNetV3 Small Personalizada 119

5.46. Métricas R2 y Precisión 5 min de RNNs en Validación con MobileNetV3 Small Personalizada 119

5.47. Métricas Precisión 15 min y Precisión 30 min de RNNs en Validación con MobileNetV3 Small Personalizada 120

5.48. Métricas Precisión 45 min y Precisión 60 min de RNNs en Validación con MobileNetV3 Small Personalizada 120

5.49. Métricas RMSE y MAE de RNNs en Prueba con MobileNetV3 Small Personalizada 121

5.50. Métricas R2 y Precisión 5 min de RNNs en Prueba con MobileNetV2 121

5.51. Métricas Precisión 15 min y Precisión 30 min de RNNs en Prueba con MobileNetV3 Small Personalizada 122

5.52. Métricas Precisión 45 min y Precisión 60 min de RNNs en Prueba con MobileNetV3 Small Personalizada 122

5.53. Métricas Precisión Promedio (Validación izquierda, Prueba Derecha) con MobileNetV3 Small Personalizada 123

6.1. Predicción de Ocupación de espacios ocupados con datos de Prueba 127

6.2. Predicción de Ocupación de espacios ocupados con datos de Validación 128

Capítulo 1

Introducción

1. Motivación

En el contexto de las grandes ciudades, se manifiestan una serie de desafíos ambientales que se derivan de su proceso de urbanización acelerada. Estos desafíos incluyen problemáticas relacionadas con la inseguridad de la tierra, la contaminación del aire, la calidad del agua deteriorada, el ruido y la gestión de residuos inadecuada [41].

En este escenario, uno de los factores cruciales que contribuyen a estos problemas son los vehículos, ya que el crecimiento constante de la urbanización conlleva un incremento en el número de automóviles en circulación. Este aumento, a su vez, genera problemas como la contaminación atmosférica, el ruido excesivo, la insuficiencia de espacios de estacionamiento, los congestionamientos vehiculares [41], así como un aumento en el consumo de combustible y, en consecuencia, mayores emisiones de dióxido de carbono CO_2 [38].

Un estudio ha sugerido que la búsqueda de espacios de estacionamiento aumenta significativamente el congestionamiento del tráfico en las áreas urbanas, aumentándolo en un porcentaje que oscila entre el 25 % y el 40 % [12] Asimismo, se ha indicado que entre un 8 % y un 74 % del tráfico en las zonas urbanas de Estados Unidos se debe a conductores que buscan estacionamiento, lo cual, además, implica un tiempo promedio de búsqueda que varía entre 3,5 a 14 minutos [34]. Este proceso no solo tiene un impacto negativo en la fluidez del tráfico, sino que también afecta negativamente la eficiencia general del sistema.

Un estudio realizado por el Boston Consulting Group (BCG) ha estimado

que en el sudeste asiático, la búsqueda diaria de estacionamiento lleva entre 19 y 45 minutos en promedio, lo que resulta en una congestión del tráfico de entre 30 y 72 minutos por día, con una proyección alarmante de aumento de 4,5 veces para el año 2022 [6].

Además, se ha observado que la navegación en búsqueda de estos espacios de estacionamiento desperdicia 31,68 millones de litros de combustible los cuales producen 129.000 toneladas de emisiones de dióxido de carbono CO_2 cada año [3].

Por consiguiente, resulta de suma importancia desarrollar estrategias que atenúen la presión del tráfico vehicular, reduciendo el consumo de energía y mitigando las emisiones, en aras de combatir el cambio climático. La implementación de una estrategia que minimice la ineficiencia y la ineficacia asociadas con la búsqueda de espacios de estacionamiento disponibles juega un papel crucial en la consecución de este objetivo.

2. Objetivo

El propósito de esta investigación es desarrollar y validar un sistema inteligente para la predicción eficaz de la disponibilidad de plazas de estacionamiento en contextos urbanos, específicamente en estacionamientos, enfocándose en la integración de técnicas avanzadas de aprendizaje profundo y análisis temporal de series de datos.

El sistema se orientará hacia un equilibrio entre la precisión técnica y la aplicabilidad práctica:

1. **Exploración y Evaluación de Técnicas de Procesamiento de Imágenes:** Se enfocará en la exploración de modelos de Redes Neuronales Convolucionales para extraer características pertinentes directamente de las imágenes segmentadas de vídeos de espacios de estacionamiento.

2. **Exploración y Evaluación de Técnicas de Procesamiento Temporal:** Se buscará explorar y evaluar técnicas de procesamiento de series temporales para identificar y analizar patrones de ocupación y desocupación de plazas de estacionamiento, incorporando la naturaleza discontinua de los datos.

3. **Desarrollo de un Modelo Predictivo:** El objetivo es desarrollar un modelo de predicción robusto y preciso que combine las características visua-

les extraídas y el análisis de series temporales para anticipar la disponibilidad de plazas de estacionamiento.

4. **Utilización de datos públicos:** Se utilizará PKLot, ampliamente utilizado en investigaciones previas y con extensa documentación, como base para estandarizar, validar y comparar el enfoque propuesto.

3. Estructura de la Memoria

1. **Introducción.** Este capítulo inicial establece el escenario para la investigación, proporcionando la motivación y los objetivos inherentes de este trabajo.

2. **Marco Teorico.** Aquí se exploran los fundamentos del Aprendizaje Automático, dando una descripción por sus conceptos y enfoques, con un énfasis particular en aquellos aspectos que son especialmente necesarios para esta investigación. Además profundiza en Aprendizaje Profundo, enfocándose en las redes convolucionales y recurrentes que son especialmente relevantes para el desarrollo de este trabajo.

3. **Estado de arte.** Este capitulo comenta los avances en el área de investigación y las distintas propuestas para dar solución al problema investigado.

4. **Desarrollo del Trabajo.** En este capítulo se detalla el proceso de desarrollo de la solución propuesta, incluyendo una descripción y análisis de los datos utilizados, las arquitecturas implementadas, las adaptaciones realizadas, y el flujo de trabajo general desde el diseño inicial hasta la implementación.

5. **Experimentación y Resultados.** Este capítulo se dedica a presentar los resultados obtenidos, y un análisis exhaustivo de estos, proporcionando detalles y una evaluación detallada de la efectividad y las limitaciones de los enfoques adoptados.

6. **Conclusiones y Trabajos Futuros.** Este capítulo final reúne los hallazgos, reflexiones y lecciones aprendidas a lo largo de la memoria, y proyecta posibles direcciones para investigaciones futuras.

7. **Anexos.** Este segmento incluye tablas detalladas de las evaluaciones para proporcionar un registro completo del trabajo realizado.

Referencias Bibliográficas. Este apartado recoge todas las fuentes y referencias bibliográficas citadas a lo largo de la memoria.

Capítulo 2

Marco Teórico

Dado que una revisión exhaustiva del aprendizaje automático y profundo involucraría abordar conceptos más allá del alcance de esta memoria, se presentarán de manera concisa aquellos términos que son relevantes para comprender los modelos implicados. Debido a que las redes neuronales pueden adoptar múltiples topologías y aplicarse a una amplia variedad de problemas, se optará por presentar el modelo más simple para esclarecer los conceptos a desarrollar.

1. Aprendizaje Automático

El aprendizaje automático es una extensión fundamental de la estadística que pone énfasis en el uso de la computación para estimar funciones complejas y establecer intervalos de confianza alrededor de estas. Se logra mediante enfoques de inferencia bayesiana o estimación frecuentista, aprovechando la experiencia adquirida a través de datos para mejorar el rendimiento [13].

Entre los desafíos abordados por el aprendizaje automático se encuentran la clasificación, regresión, transcripción, traducción y detección de anomalías, por mencionar algunos.

En un contexto formal, consideramos un conjunto de datos de entrenamiento $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$, donde x_i es una entrada y y_i es su correspondiente salida (etiqueta). El objetivo es encontrar una función f que conecte las entradas x a las salidas y de manera efectiva. En otras palabras, buscamos la mejor función f que haga que las predicciones $f(x)$ se acerquen a las salidas reales y en el conjunto de datos de entrenamiento.

Para lograrlo, introducimos la noción de parámetros del modelo θ, que transforman $f(x)$ en $f(x;\theta)$. La función $f(x;\theta)$ es una representación paramétrica que toma las entradas x y produce las salidas y, donde θ son los parámetros que ajustamos para mejorar la precisión de las predicciones.

Por otro lado, los datos a menudo están sujetos a ruido. Esto se modela mediante una observación ruidosa y generada a partir de la función f más un término de error ε que es una variable aleatoria que captura el ruido $y = f(x)+\varepsilon$

1.1. Expectativa

La evaluación de una función f en el aprendizaje automático implica comparar sus predicciones, $f(x;\theta)$, con las salidas reales, y. Para esta evaluación, se utiliza el concepto de valor esperado o expectativa, que cuantifica la discrepancia entre las predicciones del modelo y los valores reales[13].

Dado un dato específico (x, y) de nuestro conjunto de entrenamiento $\mathcal{D}$, la eficacia de la función f para predecir y se expresa mediante $f(x;\theta)$. Si tomamos el valor esperado respecto a la distribución de los datos, podemos definir el valor esperado de la predicción como:

$$\mathbb{E}[f(x;\theta)] = \int f(x;\theta) \cdot p(x)\, dx$$

Donde $p(x)$ es la distribución de probabilidad de los datos. Este valor esperado proporciona una medida de cómo esperamos, en promedio, que f realice sus predicciones.

El proceso de entrenamiento involucra ajustar los parámetros θ de la función f de modo que las predicciones $f(x;\theta)$ se aproximen a las salidas reales en el conjunto de entrenamiento.

1.2. Función de pérdida o error

La función de pérdida L cuantifica el error entre las predicciones $f(x;\theta)$ y las salidas reales y en el aprendizaje automático. Esta función juega un papel crucial en la optimización de los modelos buscando hacer que las predicciones se alineen estrechamente con los valores reales observados [13].

A través del uso de algoritmos de optimización, se buscan los valores de θ

que minimizan la función de pérdida $L(f(x;\theta), y)$ a través del conjunto de datos de entrenamiento.

En contextos de regresión, una función de pérdida comúnmente empleada es el Error Cuadrático Medio (MSE, por sus siglas en inglés):

$$L(f(x;\theta), y) = (f(x;\theta) - y)^2$$

Por otro lado, en problemas de clasificación, la Entropía Cruzada es una función de pérdida frecuentemente utilizada:

$$L(f(x;\theta), y) = -y \log(f(x;\theta)) - (1-y) \log(1 - f(x;\theta))$$

En el sentido de la reducción del error en aprendizaje automático surgen dos problemas críticos que pueden afectar el rendimiento: el sobreajuste (overfitting) y el subajuste (underfitting). Estos problemas están vinculados con la capacidad del modelo para generalizar a partir de los datos de entrenamiento y tienen una relación estrecha con el sesgo, la varianza, y el error del modelo [13].

1.3. Sesgo

El sesgo de un modelo f con respecto a un conjunto de datos $\mathcal{D}$ se define como la diferencia entre las predicciones promedio del modelo y los valores verdaderos. Específicamente, mide la discrepancia entre las predicciones del modelo y los valores reales, siendo promediados sobre diferentes conjuntos de datos de entrenamiento [13]. Se define matemáticamente como:

$$\text{Sesgo}[f(x;\theta)] = \mathbb{E}[f(x;\theta)] - y$$

donde $\mathbb{E}[f(x;\theta)]$ es el valor esperado de las predicciones del modelo y y son los valores reales.

1.4. Varianza

La varianza de un modelo f con respecto a un conjunto de datos $\mathcal{D}$ mide la variabilidad de las predicciones del modelo cuando se entrena con diferentes conjuntos de datos de entrenamiento. La varianza se define como:

$$\text{Var}[f(x;\theta)] = \mathbb{E}[(f(x;\theta) - \mathbb{E}[f(x;\theta)])^2]$$

La varianza refleja la medida en que las predicciones $f(x;\theta)$ fluctúan alrededor de su valor esperado cuando el modelo es entrenado con diferentes conjuntos de datos [13].

Lograr un equilibrio entre el sesgo y la varianza es crucial para obtener un modelo que se generalice de manera adecuada a datos no observados durante el entrenamiento, como se ilustra en la Figura 2.1.

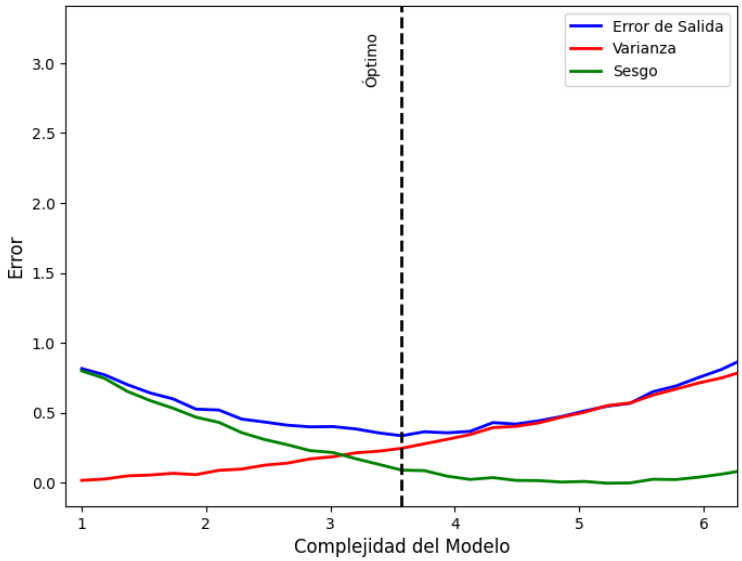

Figura 2.1: Relación entre error y complejidad del modelo para un conjunto de datos de entrenamiento fijo

1.5. Sobreajuste (Overfitting)

El sobreajuste ocurre cuando un modelo se ajusta demasiado a los datos de entrenamiento, llegando incluso a aprender el ruido presente en esos datos. Esto suele suceder cuando el modelo tiene una complejidad excesiva en relación con la cantidad y calidad de datos disponibles. En otras palabras, el modelo "sobreaprende" de los datos de entrenamiento. Como resultado, aunque el modelo puede tener un rendimiento excepcionalmente bueno en el conjunto de entrenamiento, suele presentar un rendimiento pobre en datos no vistos o de prueba, ya que no generaliza bien. Este fenómeno se asocia con una baja generalización, lo que significa que el modelo tiene una baja capacidad para predecir nuevos datos y una alta varianza, que se refleja en la sensibilidad extrema a las variaciones en los datos de entrenamiento [13].

1.6. Subajuste (Underfitting)

El subajuste, se refiere a la situación en la que un modelo es demasiado simple y no logra capturar patrones subyacentes en los datos. En este escenario, el modelo no se ajusta adecuadamente ni siquiera a los datos de entrenamiento, resultando en un rendimiento pobre tanto en el conjunto de entrenamiento como en el de prueba. El modelo, debido a su simplicidad inherente, no es lo suficientemente flexible como para representar adecuadamente la complejidad del problema subyacente. Este fenómeno se asocia con un alto sesgo y una baja varianza. El alto sesgo se refleja en la incapacidad del modelo para capturar los patrones en los datos, incluso en el conjunto de entrenamiento. La varianza es baja porque el modelo, debido a su naturaleza simplificada, es en gran medida insensible a las variaciones en los datos de entrenamiento[13].

1.7. Descomposición del Sesgo, la Varianza y el Ruido

Consideremos el conjunto de datos $\mathcal{D} = (X, Y)$ de N pares de variables independientes y dependientes donde suponemos que los datos se generan a partir de un modelo ruidoso. En este sentido, en lugar de que f sean pares (x, y), los pares ahora se componen de un valor y afectado por un ruido ε que sigue una distribución normal con media cero y desviación estándar σ_ϵ:

$$y = f(\boldsymbol{x}) + \varepsilon$$

Supongamos que tenemos un procedimiento estadístico para formar un predictor $f(x; \theta)$ que proporciona la predicción de nuestro modelo para un nuevo punto de datos x. Este estimador se elige minimizando una función de pérdida que en problemas de regresión tomamos como el error cuadrático medio (MSE):

$$L(f(x; \theta), y) = \sum_i (y_i - f(x_i; \theta))^2$$

Estas estimaciones dependen del conjunto de datos y del universo de posibles resultados que se obtengan como expectativa de la minimización del error.

$$\mathbb{E}_{\mathcal{D}}\left[\sum_i (y_i - f(x_i;\theta))^2\right] = \mathbb{E}_{\mathcal{D}}\left[\sum_i \underbrace{(y_i - f(x_i)}_{a} + \underbrace{f(x_i) - f(x_i;\theta))}_{b}^2\right]$$

$$= \sum_i \mathbb{E}_{\mathcal{D}}\left[\underbrace{(y_i - f(x_i)}_{a} + \underbrace{f(x_i) - f(x_i;\theta))}_{b}^2\right]$$

$$= \sum_i \mathbb{E}_{\mathcal{D}}\left[\begin{array}{l}\underbrace{(y_i - f(x_i))^2}_{a^2} + \underbrace{(f(x_i) - f(x_i;\theta))^2}_{b^2} \\ + \underbrace{2(y_i - f(x_i))(f(x_i) - f(x_i;\theta))}_{2ab}\end{array}\right]$$

Respecto al término a^2, podemos reescribirlo como:

$$a^2 = \sum_i \mathbb{E}_{\mathcal{D}}\left(f(x_i) + \epsilon - f(x_i)\right)^2 = \sum_i \mathbb{E}_{\epsilon}\left[\epsilon^2\right] = \sum_i \sigma_{\epsilon}^2 = \sigma_{\epsilon}^2$$

donde σ_{ϵ}^2 se conoce como "noise" (ruido) [31].

Respecto al término $2ab$, este se cancela y se vuelve cero en promedio debido a que el modelo de aprendizaje profundo se entrena para minimizar el error entre sus predicciones $f(x_i;\theta)$ y los valores reales y_i.

Respecto al término b^2, repetiremos la misma deducción [31] con la expectativa de $\mathcal{D}$ de $f(x_i;\mathcal{D})$. La expectativa se toma con respecto al conjunto de datos de entrenamiento aleatorio $\mathcal{D}$, que produce el ajuste:

$$b^2 = \sum_i \mathbb{E}_{\mathcal{D}}\left[(f(x_i) - f(x_i;\theta))^2\right]$$

$$= \sum_i \mathbb{E}_{\mathcal{D}}\left[\underbrace{(f(x_i) - f(x_i;\mathcal{D}))}_{c} + \underbrace{(f(x_i;\mathcal{D}) - f(x_i;\theta))}_{d}\right]^2$$

$$= \sum_i \mathbb{E}_{\mathcal{D}}\left[\begin{array}{l}\underbrace{(f(x_i) - f(x_i;\mathcal{D}))^2}_{c^2} + \underbrace{(f(x_i;\mathcal{D}) - f(x_i;\theta))^2}_{d^2} \\ + \underbrace{2(f(x_i) - f(x_i;\mathcal{D}))(f(x_i;\mathcal{D}) - f(x_i;\theta))}_{2cd}\end{array}\right]$$

El término c^2 representa el "sesgo cuadrático", que indica cuánto se desvían en promedio las predicciones de nuestro modelo ($f(x_i;\mathcal{D})$) con respecto al valor óptimo ($f(x_i;\theta)$).

Por otro lado, d^2 representa la "varianza", que indica cuánto varían las predicciones de nuestro modelo cuando se entrena con diferentes conjuntos de datos de entrenamiento ($\mathcal{D}$) pero se evalúa en la misma entrada (x_i).

En este contexto, el término c^2 representa el "sesgo cuadrático", que indica cuánto se desvían en promedio las predicciones de nuestro modelo ($f(x_i; \mathcal{D})$) con respecto al valor óptimo ($f(x_i; \theta)$).

Por otro lado, d^2 representa la "varianza", que indica cuánto varían las predicciones de nuestro modelo cuando se entrena con diferentes conjuntos de datos de entrenamiento ($\mathcal{D}$) pero se evalúa en la misma entrada (x_i).

En este contexto, el término $2cd$ representa la interacción entre el sesgo y la varianza, que tiende a ser pequeña o incluso cero cuando un modelo tiene un bajo sesgo (c) y alta varianza (d), o viceversa [31]. Esto significa que en situaciones donde las predicciones son en promedio cercanas al valor óptimo pero varían significativamente con diferentes conjuntos de datos de entrenamiento, las diferencias entre las predicciones promedio y las predicciones individuales suelen ser pequeña anulándose también.

En resumen:

$$\mathbb{E}_{\mathcal{D}}\left[\sum_i (y_i - f(x_i;\theta))^2\right] = \sum_i \mathbb{E}_{\mathcal{D}}\left[\underbrace{(f(x_i) - f(x_i;\mathcal{D}))^2}_{\text{bias}^2} + \underbrace{(f(x_i;\mathcal{D}) - f(x_i;\theta))^2}_{\text{variance}}\right] + \underbrace{\sigma_\epsilon^2}_{\text{noise}}$$

Habiendo descompuesto el error en sus componentes de sesgo, varianza y ruido, es evidente que la tarea del modelado no es solamente encontrar un modelo que se ajuste bien a los datos de entrenamiento, sino uno que también pueda generalizar bien a datos no vistos. El desafío principal radica en encontrar el equilibrio entre el sesgo y la varianza, de manera que el modelo ni subajuste ni sobreajuste [13].

La Figura 2.2 ilustra cómo, a medida que aumentamos el tamaño del conjunto de entrenamiento, la varianza disminuye y la capacidad del modelo para generalizar mejora, reduciendo la diferencia entre el Error de entrenamiento (Ein) y el Error con los datos de prueba o generalización (Eout). Esta diferencia es crucial para entender el comportamiento del modelo en el proceso de minimización de error. Un modelo con comportamientos diferentes entre datos de entrenamiento y prueba es probable que esté sobreajustado o los datos de entrenamiento sean escasos, mientras que un modelo con una diferencia pequeña

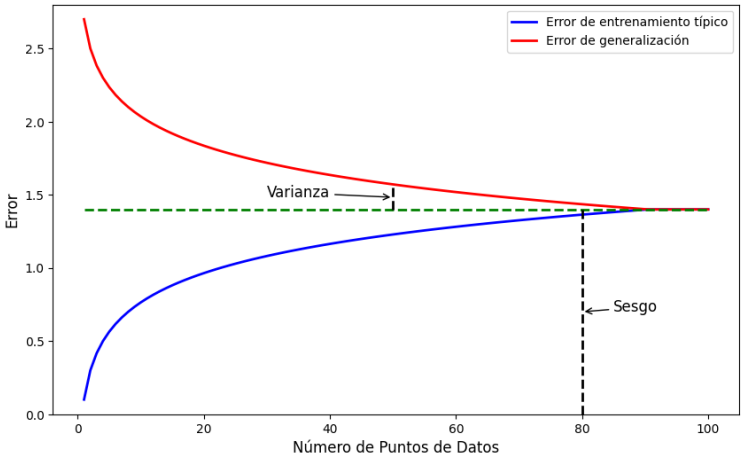

Figura 2.2: Representación del error en función de la cantidad de datos de entrenamiento.

generalmente indica que con datos suficientes encontró un equilibrio de varianza y sesgo. [23].

Para establecer un equilibrio entre sesgo y varianza se introduce el termino de regularización que penaliza el proceso de entrenamiento.

1.8. Regularización

La regularización es un conjunto de técnicas que expresar preferencias en la selección de una función sobre otra permitiendo controlar la capacidad del modelo para incluir o excluir miembros del espacio de hipótesis. En el campo del aprendizaje automático, es una preocupación central, comparada en importancia solo con la optimización.

Regularización L1 (Lasso): Agrega una penalización proporcional a la norma $L1$ de los parámetros θ:

$$L_{\text{reg}}(\theta) = L(\theta) + \lambda \|\theta\|_1$$

Esta técnica tiende a generar soluciones en las que muchos parámetros son exactamente cero, lo que resulta en modelos más escasos aunque uno de los inconvenientes que presenta Lasso es que no trabaja bien cuando existen variables altamente correlacionadas [25].

Regularización L2 (Ridge): Agrega una penalización proporcional a la

norma euclidiana al cuadrado de los parámetros θ:

$$L_{\text{reg}}(\theta) = L(\theta) + \lambda\|\theta\|_2^2$$

A diferencia de L1, esta técnica tiende a reducir todos los parámetros proporcionalmente, pero no los hace exactamente cero además de no producir un modelo parsimonioso ya que conserva todas las variables predictoras [25].

Dropout: Durante el entrenamiento, ciertas neuronas se *apagan* aleatoriamente. Si z_i es la activación de la neurona i, después de aplicar dropout, la activación se modifica como:

$$z_i' = \begin{cases} 0 & \text{con probabilidad } p \\ z_i & \text{con probabilidad } 1-p \end{cases}$$

Esta técnica previene que las neuronas dependan demasiado de otras neuronas específicas durante el entrenamiento siendo un avance significativo desde su invención [37].

2. Aprendizaje Profundo

El Aprendizaje Profundo se ha convertido en una herramienta esencial en la resolución de problemas complejos que involucran grandes cantidades de datos. A diferencia de los métodos tradicionales de aprendizaje automático que se basan en algoritmos diseñados manualmente, el aprendizaje profundo utiliza redes neuronales con múltiples capas (de ahí el término *profundo*) para aprender representaciones de datos de manera jerárquica [13].

El aprendizaje profundo se basa en una función $f : X \to Y$ parametrizada con θ en $\mathbb{R}^{n_\theta}$ (donde $n_\theta \in \mathbb{N}$):

$$y = f(x; \theta)$$

Una red neuronal profunda se caracteriza por una sucesión de múltiples capas de procesamiento. Cada capa consiste en una transformación no lineal y la secuencia de estas transformaciones conduce al aprendizaje de diferentes niveles de abstracción. Estas capas transforman la información entrante, capa por capa, para producir una salida con diferentes niveles de abstracción.

2.1. Redes Neuronales

Existen múltiples tipos de redes neuronales, pero describiremos de forma descriptiva una red neuronal multicapa conocida como perceptrón multicapa con una capa oculta completamente conectada.

En este caso, la primera capa recibe los valores de entrada (es decir, las características de entrada) x en forma de un vector columna de tamaño n_x (donde $n_x \in \mathbb{N}$). Los valores de la siguiente capa oculta son una transformación de estos valores por una función paramétrica no lineal, que es una multiplicación de matriz por W_1 de tamaño $n_h \times n_x$ (donde $n_h \in \mathbb{N}$), más un término de sesgo b_1 de tamaño n_h, seguido de una transformación no lineal donde θ es la función de activación

$$h = \theta(W_1 \cdot x + b_1)$$

Esta función de activación es la que determina la no linealidad, lo que proporciona la expresividad dela red neuronal. En la Figura 2.4 se muestran las principales funciones de activación.

La capa oculta h de tamaño n_h a su vez puede ser transformada a otros conjuntos de valores hasta la última transformación que proporciona los valores de salida y. En este caso:

$$y = (W_2 \cdot h + b_2)$$

donde W_2 tiene un tamaño de $n_y \times n_h$ y b_2 tiene un tamaño de n_y (donde $n_y \in \mathbb{N}$. Todas estas capas se entrenan para minimizar el error L al estar conectadas como se observa en la Figura 2.3.

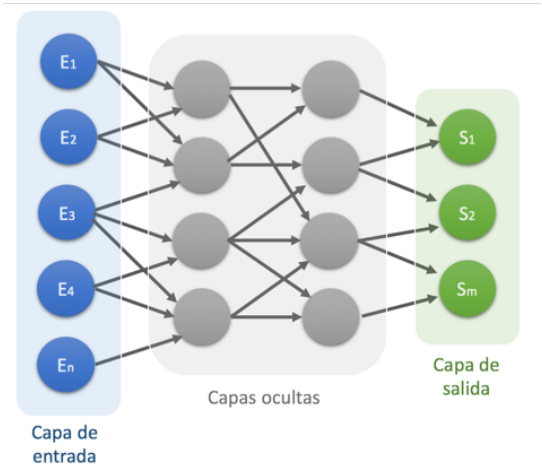

Figura 2.3: Ejemplo de Modelo multicapa con capa oculta

Funciones de Activación

Las funciones de activación son esenciales para las redes neuronales. Estas funciones determinan la salida de una neurona en función de su entrada ponderada, permitiendo la introducción de no linealidades en el modelo que permiten expresividad de la red o la capa donde se utiliza para representar situaciones complejas. En la Figura 2.4, se muestra algunas de las mas comunes:

- **Sign (Signo):** La función de activación signo asigna +1 si la entrada es positiva y -1 si es negativa. Esta función es sencilla y se utiliza en problemas de clasificación binaria como en Perceptron [29].
- **Sigmoid (Sigmoide):** La función de activación sigmoidea produce una curva en forma de S que mapea la entrada a un rango entre 0 y 1. Es ampliamente utilizada en redes multicapa como MLP [24] o LSTM [16].
- **Tanh (Tangente Hiperbólica):** La función de activación tangente hiperbólica mapea la entrada a un rango entre -1 y 1. Es similar a la sigmoidea pero tiene un rango simétrico siendo usada en redes como LSTM [16] y GRU [7].
- **Linear (Lineal):** La función de activación lineal es una transformación lineal simple sin no linealidad. Esta función mantiene la proporción de entrada y es útil en capas intermedias como en LeNet [21] y AlexNet [19]
- **ReLU (Rectified Linear Unit):** Esta función establece la salida en 0 si la entrada es negativa y mantiene la entrada si es positiva. Dada su rapidez de convergencia y eficacia computacional es usada en muchos modelos modernos como AlexNet [19], VGG [35] y ResNet [14].
- **ReLU6 (Rectified Linear Unit 6):** La función de activación ReLU6 es una variante de ReLU que limita la salida a 6 cuando la entrada es mayor que 6. Es útil en situaciones donde se desea limitar la activación máxima como por ejemplo en MobileNetV2 [30].
- **LeakyReLU6 (Leaky Rectified Linear Unit 6):** La función de activación LeakyReLU6 es una variante de ReLU6 que permite un pequeño valor negativo cuando la entrada es negativa, evitando así la "muerte" de neuronas y a su vez limitar la activación máxima.
- **Hard Sigmoid (Sigmoidea Dura):** La función de activación Hard Sigmoid es una aproximación más simple y rápida de la función sigmoidea. Limita la salida a 1 si la entrada es grande y a 0 si es pequeña y suele implementarse en modelos BNN [8].

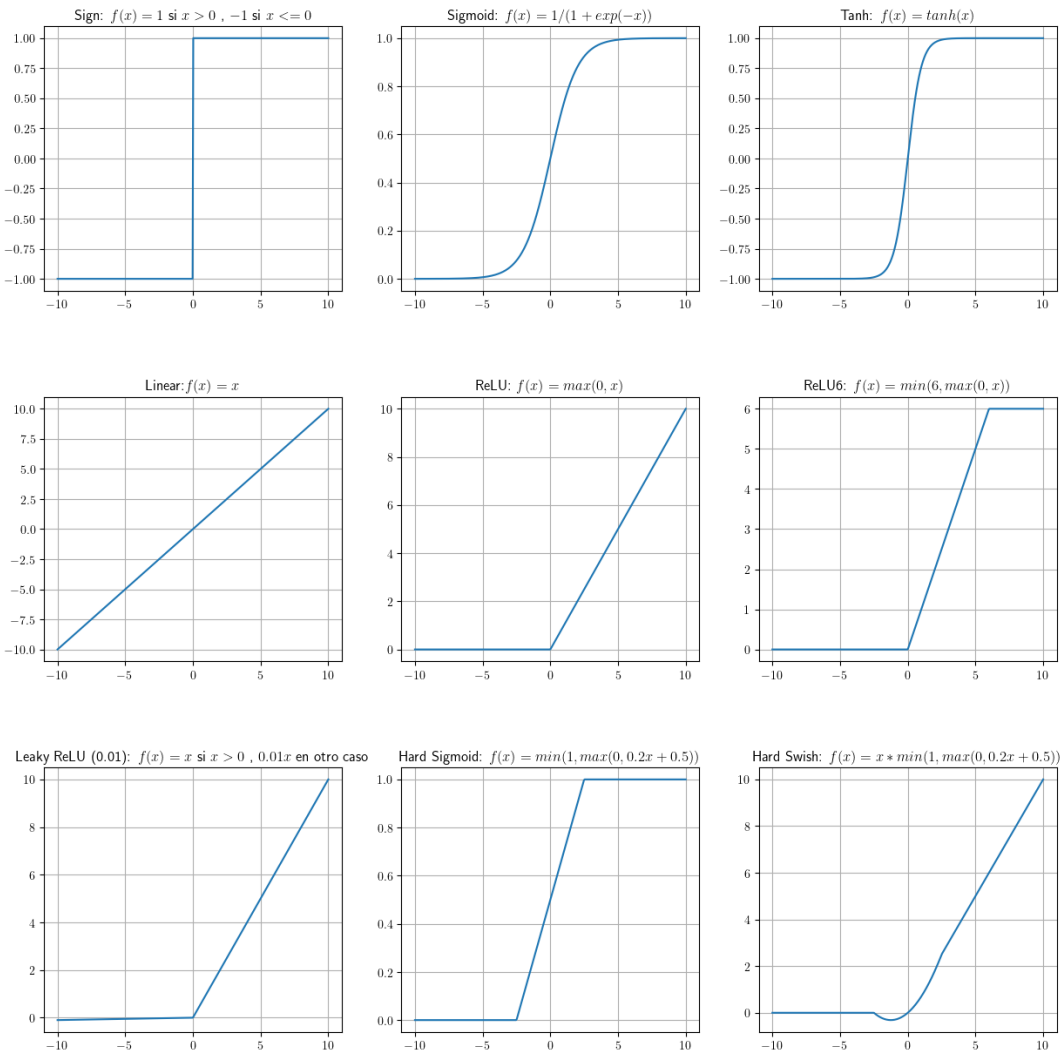

Figura 2.4: Gráfica de funciones de Activación

- **Hard Swish (Swish Dura):** La función de activación Hard Swish es una variante de la función Swish que utiliza una aproximación más simple. Combina la linealidad de ReLU con la suavidad de la sigmoidea siendo un factor de eficiencia en MobileNetV3 [17].

Optimización y Retropropagación

Una vez definida la arquitectura de la red y la función de pérdida, el siguiente paso es entrenar la red para ajustar los pesos. Para esto, se utiliza el algoritmo de retropropagación junto con un método de optimización, como el descenso de gradiente. Durante el entrenamiento, se calcula el gradiente de la función de pérdida con respecto a cada peso en la red y se actualizan los pesos en la dirección opuesta al gradiente:

$$\theta \leftarrow \theta - \alpha \nabla_\theta L$$

Donde θ son los parámetros de la red, α es la tasa de aprendizaje (learning rate) y L es la función de pérdida.

2.2. Redes Neuronales Convolucionales

Las Redes Neuronales Convolucionales se han destacado como una arquitectura dominante en el campo de la visión por computadora, particularmente en tareas que involucran análisis y procesamiento de imágenes. Esta capacidad de procesar imágenes y extraer características relevantes las hace especialmente útiles para tareas como la clasificación de imágenes, detección de objetos, entre otras [13].

Convolución

La convolución de f y g se escribe como $f * g$. Se define como la integral del producto de las dos funciones después de que una se invierte y se desplaza:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau$$

siendo equivalente el intercambio de roles de f y g en la definición debido a la conmutatividad. La fórmula de convolución puede describirse como el área bajo

la función $f(x)$ ponderada por la función $g(-x)$ desplazada por una cantidad t. Si f y g están definidas en el conjunto Z de enteros, la convolución discreta de f y g está dada por:

$$(f * g)(n) = \sum_{m=-\infty}^{\infty} f(m)g(n-m)$$

Por otro lado las redes convolucionales estan compuestas de tres tipos basicos de capas:

- Capas convolucionales, que son las que realizan la operación de convolución.
- Capas de agrupación, que son responsables de reducir, entre otros, la dimensionalidad de la red, el número de parámetros y la complejidad del modelo.
- Capas completamente conectadas, que contienen neuronas que están directamente conectadas a las dos capas adyacentes inmediatas.

Capas Convolucionales

Requiere algunos componentes: los datos de entrada, un filtro y un mapa de características. Los datos de entrada son proporcionados por una matriz dimensional. Si la entrada es una imagen a color, la matriz de entrada que representa los píxeles tendría 3 dimensiones (tres colores de la representación RGB). El detector de características, el kernel o filtro, se moverá a través de la imagen, realizando la convolución, que puede interpretarse como una comprobación de si la característica está presente. Luego, el filtro se aplica a un área de la imagen, y se calcula un producto escalar entre los píxeles de entrada y el filtro. Este producto escalar luego se alimenta en una matriz de salida. Después, el filtro se desplaza con un paso o salto configurable, repitiendo el proceso hasta que el kernel haya barrido toda la imagen como en la Figura 2.5. La salida final de la serie de productos escalares de la entrada y el filtro se conoce como un "mapa de características".

Usualmente en los modelos no aplicaremos solo un kernel, sino que tendremos muchos kernels (su conjunto es el llamado filtros). Por lo tanto, hay algunos parámetros que deben ajustarse durante el proceso: el número de filtros, el paso o salto y el relleno. Se utiliza un relleno de ceros cuando los filtros no se ajustan a la imagen de entrada. Esto establece todos los elementos que caen fuera de

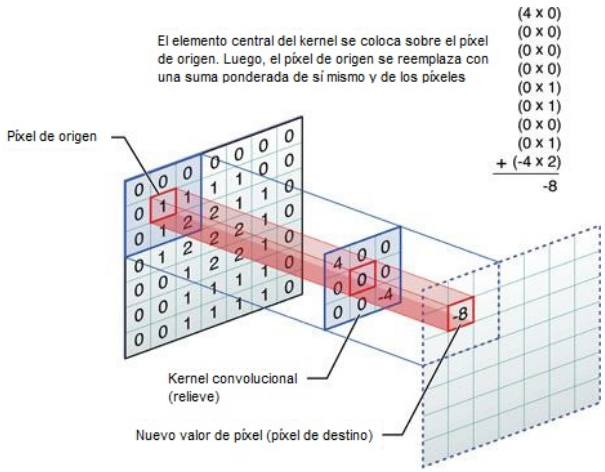

Figura 2.5: Ejemplo de aplicación de filtro a la entrada en una CNN.

la matriz de entrada en cero, produciendo una salida más grande o del mismo tamaño. Hay tres tipos de relleno:

- Relleno válido o sin relleno. En este caso, se descarta la última convolución si las dimensiones no se alinean.
- Relleno del mismo tamaño. Este relleno asegura que la capa de salida tiene el mismo tamaño que la capa de entrada.
- Relleno completo. Este relleno aumenta el tamaño de la salida agregando ceros al borde de la entrada.

Después de cada operación de convolución, una red convolucional aplica una transformación al mapa de características, introduciendo no linealidad al modelo a través de una función de activación como sucede en las redes neuronales tradicionales.

Capas de Agrupación

Las capas de agrupación, también conocidas como “pooling”, reducen la dimensión espacial (ancho x alto) de la entrada, lo que ayuda a disminuir la cantidad de parámetros y la complejidad computacional de la red En la Figura 2.6 se observa el proceso con distintos Stride y como afecta la dimensionalidad. Hay diferentes tipos de operaciones de agrupación:

- *Max pooling*: Selecciona el valor máximo de un conjunto de valores en una ventana definida.

$$f_{\max}(X) = \text{máx}(X)$$

- *Average pooling*: Calcula el valor promedio de un conjunto de valores en una ventana definida.

$$f_{\text{avg}}(X) = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Donde X es el conjunto de valores en la ventana y n es el número de valores en X.

Capas Totalmente Conectadas

Las capas totalmente conectadas se encuentran hacia el final de una red convolucional y tienen la función de realizar una clasificación de alto nivel basada en las características extraídas por las capas anteriores. En estas capas, cada neurona está conectada a todas las neuronas de la capa anterior y la siguiente, similar a una red neuronal tradicional.

La salida de las capas anteriores (usualmente la capa de agrupación final) se aplana en un vector y se alimenta a las capas totalmente conectadas. Estas capas pueden tener una o más capas ocultas, y cada neurona en estas capas calcula un peso sumatorio de sus entradas:

$$y_i = \sum_j w_{ij} x_j + b_i$$

Donde y_i es la salida de la neurona i, w_{ij} es el peso de la conexión entre la neurona i y la entrada j, x_j es la j-ésima entrada y b_i es el sesgo de la neurona i.

Las redes convolucionales, gracias a su capacidad para aprender jerarquías de características a partir de datos de entrada brutos, han demostrado ser eficientes en tareas de reconocimiento de imágenes y abstracción de características. Por ello, en esta memoria exploremos varias arquitecturas conocidas, algunas adaptadas específicamente para la tarea, con el objetivo de obtener el mejor modelo para la predicción de abandono de espacios de estacionamiento y a su vez características para la etapa de detección temporal.

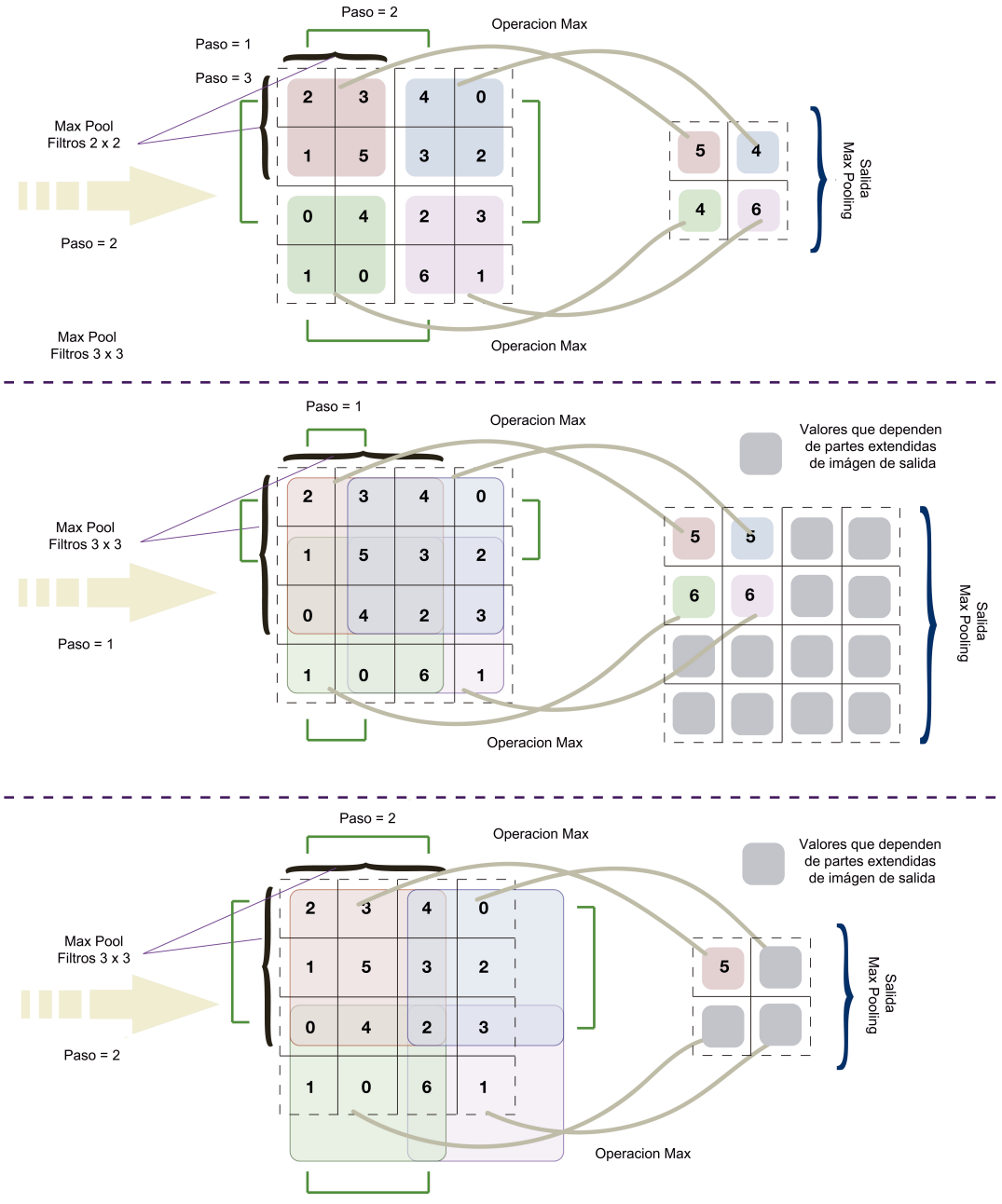

Figura 2.6: Ejemplo de aplicación de Max Pooling con distintos Stride (pasos)

2.3. Redes Neuronales Recurrentes

Las Redes Neuronales Recurrentes, son una clase especial de redes neuronales diseñadas específicamente para manejar datos secuenciales. A diferencia de las redes neuronales tradicionales, que asumen que todas las entradas (y salidas) son independientes entre sí, estas utilizan bucles para mantener la información a lo largo del tiempo. Esta capacidad de "memoria" las hace especialmente adecuadas para tareas como la predicción en series temporales, el procesamiento de lenguaje natural, la generación de texto y la traducción automática, porque tienen una naturaleza intrínsecamente temporal donde la información presente en un punto específico de la secuencia a menudo depende de los puntos anteriores. Esto se logra mediante la introducción de conexiones recurrentes en la red, lo que permite que la información circule a través de la red durante múltiples pasos temporales, proporcionando un mecanismo para aprender dependencias temporales a corto y, en teoría, largo plazo [13].

Por otro lado, una característica distintiva es su capacidad para procesar secuencias de longitud variable, lo que las distingue de las redes feedforward y convolucionales tradicionales que requieren entradas de tamaño fijo. Esta flexibilidad se logra a través de la naturaleza recurrente de la red, que le permite operar sobre un paso de tiempo y pasar su salida (y posiblemente su estado) como entrada para el paso de tiempo siguiente. Esta operación iterativa a lo largo de los pasos de tiempo permite que mantengan un "estado" a lo largo del tiempo.

Sin embargo, es importante notar que, aunque las redes recurrentes pueden teóricamente mantener información de pasos de tiempo arbitrariamente distantes, en la práctica enfrentan desafíos como el desvanecimiento y explosión del gradiente, que pueden limitar su capacidad para aprender dependencias a largo plazo. [15].

Desvanecimiento y explosión del gradiente

Durante el entrenamiento estos modelos pueden enfrentar problemas en los que los gradientes se vuelven demasiado pequeños (desvanecimiento) o demasiado grandes (explosión). Esto puede hacer que el entrenamiento sea difícil y a menudo impredecible.

- El desvanecimiento del gradiente se refiere al problema en el que el gradiente de la pérdida se vuelve muy pequeño, lo que lleva a actualizaciones insuficientes y, por lo tanto, a una convergencia extremadamente lenta.

- La explosión del gradiente ocurre cuando los gradientes se vuelven demasiado grandes, lo que puede causar que los pesos de la red se disparen, llevando a modelos inestables.

Esta problemática fue resuelta con distintos enfoques por distintos modelos. Entre los relevantes para la memoria, abordaremos el modelo de Memoria a Largo-Corto Plazo o Long Short-Term Memory, en adelante LSTM [44], el modelo de Unidad Recurrente con mecanismos de compuerta o Gated Recurrent Unit, en adelante GRU [22], y una variante de modelo convolucional unidimensional utilizado para series temporales o One-dim Convolutional Neural Network, en adelante CNN-1D [33].

Modelo LSTM (Long Short-Term Memory)

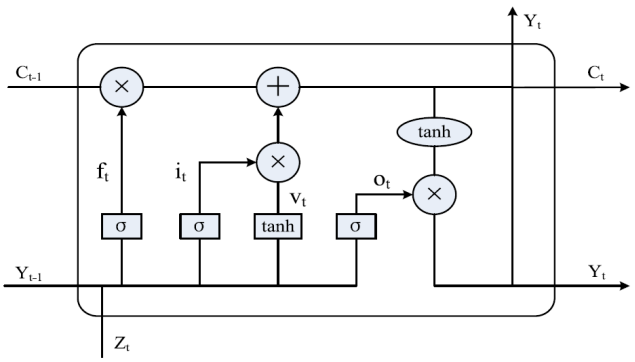

Figura 2.7: Diagrama de Arquitectura LSTM

LSTM es un tipo de red que actúa como una red recurrente mejorada. El problema de la explosión del gradiente y la desaparición del gradiente se resuelve eficazmente utilizando la estructura de puertas únicas [42], [32]. El modelo LSTM tiene una precisión de ajuste y predicción generalmente más alta en comparación con otras RNN, pero el proceso de entrenamiento lleva más tiempo debido a la cantidad excesiva de parámetros [44].

La información se almacena actualizando la información del estado neuronal, utilizando **puertas de entrada, puertas de salida y puertas de olvido** para controlar el peso de la información histórica. La introducción de tres umbrales en LSTM resuelve eficazmente el problema de la desaparición del gradiente mientras filtra la información relativamente no importante y reduce el tiempo de entrenamiento [13]. La estructura del modelo se muestra en la Figura 2.7.

Donde C_{t-1} es el estado de la neurona, Y_{t-1} es la salida del tiempo anterior, Z_t es la entrada del tiempo actual.

La **puerta de olvido** determina cuántas unidades C_{t-1} en el tiempo anterior pueden ser retenidas al tiempo actual C_t. La entrada de la puerta de olvido incluye la salida Y_{t-1} y la carga actual Z_t en el tiempo anterior. Finalmente, f_t se obtiene a través de la función de activación Sigmod más a la izquierda.

$$f_t = \sigma(W_f \cdot [Y_{t-1}, Z_t] + b_f)$$

Donde W_f es la matriz de peso de la puerta de olvido, b_f representa el vector de sesgo, y σ representa la función de activación Sigmod. f_t indica la probabilidad de que la última capa de neuronas sea olvidada, y el rango de valores es [0, 1], donde 0 indica descarte completo y 1 indica retención completa.

La puerta de entrada determina qué nuevas entradas en Z_t pueden ser almacenadas en las neuronas, que se dividen principalmente en i_t y v_t, y se calculan asi:

$$i_t = \sigma(W_i \cdot [Y_{t-1}, Zt] + b_i)$$

$$v_t = \tanh(W_c \cdot [Y_{t-1}, Z_t] + b_c)$$

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Donde W_i y W_c son las matrices de peso de las puertas de entrada. b_i y b_c son los vectores de sesgo. i_t y v_t representan la proporción de la información de carga que actualmente necesita ser retenida. Se suma con el estado neuronal retenido en el momento anterior para generar información de estado neuronal actualizada C_t.

$$C_t = C_{t-1} \cdot f_t + i_t \cdot v_t$$

La **puerta de salida** se utiliza para controlar la salida del estado neuronal y transferir el estado al siguiente neurón. La puerta de salida o_t se calcula asi:

$$o_t = \sigma(W_o \cdot [Y_{t-1}, Z_t] + b_o)$$

Donde W_o es la matriz de peso de la puerta de salida, b_o es el vector de sesgo, y Y_t es la salida de las neuronas en el momento actual. El cálculo de la última capa produce la fórmula de cálculo del valor predictivo de carga final:

$$Y_t = o_t \cdot \tanh(C_t)$$

Modelo GRU (Gated Recurrent Unit)

Como variante de LSTM, la GRU también puede resolver eficazmente el problema de explosión del gradiente y desaparición del gradiente. Comparada con la LSTM, la estructura de la GRU es más simple, fusionando la **puerta de olvido** y la **puerta de entrada** en una **puerta de actualización** [13]. Dado que la GRU reduce una puerta y una multiplicación de matriz más pequeña, la GRU puede ahorrar mucho tiempo cuando los datos de entrenamiento son grandes [22]. El diagrama estructural del modelo GRU se muestra en la Figura 2.9.

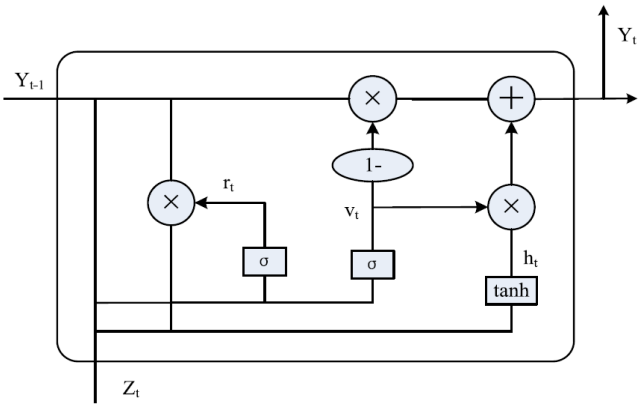

Figura 2.8: Diagrama de Arquitectura GRU

Donde Z_t es la entrada del momento actual, Y_{t-1} es la salida del momento anterior, y Y_t es la salida del momento actual.

La GRU tiene dos puertas. La primera puerta es la **puerta de actualización** v_t, que determina cuánta información histórica puede transmitirse al futuro, lo que significa que la puerta de actualización v_t determina si se debe actualizar el estado oculto a un nuevo estado. Funcionalmente, la puerta de actualización a de la GRU es similar a la **puerta de salida** de LSTM. La puerta de actualización v_t se calcula asi:

$$v_t = \sigma(W_v \cdot [Y_{t-1}, Z_t] + b_v)$$

Donde W_v es la matriz de peso de la puerta de actualización, y b_v es el vector de sesgo.

La segunda puerta es la **puerta de reinicio** r_t, que reinicia la función principal de la puerta r_t y determina cuánta información histórica no puede ser transmitida al siguiente estado, similar a la combinación de **puerta de olvido**

y puerta de entrada en LSTM. La puerta de reinicio r_t se calcula como se muestra a continuación:

$$r_t = \sigma(W_r \cdot [Y_{t-1}, Z_t] + b_r)$$

Donde W_r es la matriz de peso de la puerta de reinicio, y b_r es el vector de sesgo.

Después de calcular la **puerta de actualización** v_t y la **puerta de reinicio** r_t, la GRU calculará el estado oculto candidato h_t, como se muestra a continuación:

$$h_t = \tanh(W_h \cdot [r_t \cdot Y_{t-1}, Z_t] + b_h)$$

Donde W_h es el parámetro de peso correspondiente, y b_h es el parámetro de desviación correspondiente. Cuando el valor de la puerta de reinicio está cerca de 0, el valor del estado oculto candidato correspondiente a la puerta de reinicio también es 0, es decir, se descarta el estado oculto en el momento anterior. Cuando el valor de la puerta de reinicio r_t se acerca a 1, el estado oculto en el momento anterior es 1. Por lo tanto, el papel de la puerta de reinicio es descartar selectivamente la información histórica independiente de la predicción.

La salida Y_t de la GRU en el momento anterior t se calcula como se muestra a continuación:

$$Y_t = (1 - v_t) \cdot Y_{t-1} + v_t \cdot h_t$$

Modelo One-dim Convolutional Neural Network (CNN-1D)

Las redes neuronales convolucionales (CNN) llevadas a una dimensión pueden identificar patrones temporales locales o características temporales. En lugar de convolucionar sobre una imagen bidimensional, las CNN-1D convolucionan a lo largo de una dimensión temporal [33].

Dada las características antes comentadas de las CNNs, esta puede combinar técnicas anteriormente comentadas y puede agruparse en múltiples capas así como combinarse con capas densas. Por ejemplo en [33] se ilustra un modelado con cuatro capas de convolución: las tres primeras tienen 96 filtros y la última tiene 16. A medida que la red se adentra, los tamaños de los filtros decrecen en el orden 128, 64, 32 y 16. Posterior a cada capa de convolución, hay una capa de Max Pooling de tamaño $[2 \times 1]$. La entrada es una imagen 1D de dimensiones $[512 \times 1 \times 1]$. Esta estructura convolucional se complementa con una red totalmente conectada con tamaños de capas ocultas de 50, 25 y 2,

culminando en una capa softmax y una de clasificación como se puede observar en la Figura 2.9.

De esta forma se puede observar que si consideramos al tiempo una dimensión es posible adaptar las entradas temporales a este tipo de redes.

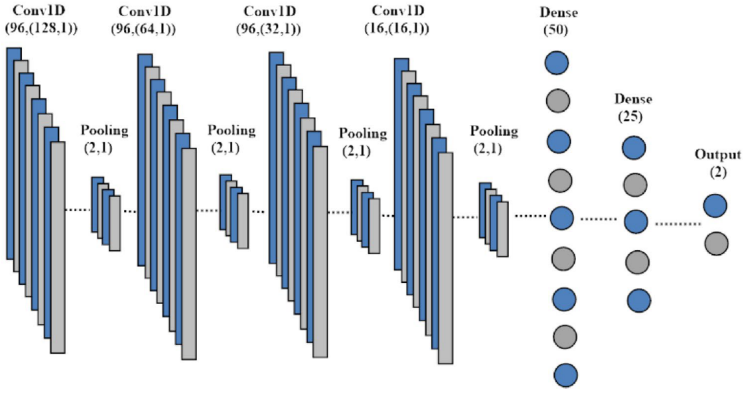

Figura 2.9: Ejemplo de Arquitectura CNN-1D con múltiples capas y salida Densa para ser decidible

Capítulo 3

Estado de arte

Las organizaciones responsables de administrar ciudades de forma progresiva comenzaron a combinar distintos dispositivos para convertirlas en ciudades inteligentes utilizando la combinación de cámaras para capturar el flujo de tráfico diariamente para su estudio. Este enfoque ha cobrado impulso a medida que las tecnologías de Internet de las Cosas (IoT) han madurado, otorgando a estas cámaras un carácter más inteligente y capaz. Paralelamente, los avances en el campo de la inteligencia artificial han permitido un procesamiento más eficiente y efectivo de estos datos, con aplicaciones que van desde la identificación de personas [4], conteo de personas y vehículos que cruzan intersecciones [36], así como diferentes soluciones en la búsqueda de estacionamiento [2] [26] [18] [28].

En el ámbito de la problemática del estacionamiento, se ha producido una evolución en las soluciones empleadas. Inicialmente, se exploraron soluciones basadas en sensores para las plazas vacantes, pero con el avance de las cámaras inteligentes y las técnicas de aprendizaje automático, se ha logrado una detección más precisa y económica de espacios vacantes, acompañada de su capacidad de reutilización en otros contextos [2].

El panorama metodológico refleja una diversidad de enfoques. Se han empleado técnicas como el marco jerárquico bayesiano (BHF) para la detección de plazas vacantes sin considerar las condiciones meteorológicas [18] (2013). Por otro lado, la máquina de vector de soporte (SVM), en combinación con clasificadores de histogramas de color, ha demostrado eficacia en el procesamiento de imágenes [28]. Sin embargo, el rendimiento de estas técnicas depende en gran medida de las características cromáticas y texturales de las imágenes afectando su rendimiento ante factores externos.

En contraposición, las redes neuronales convolucionales han demostrado su valía en la detección de plazas vacantes de forma mas reciente. Estas redes han permitido un procesamiento eficiente de la información capturada por las cámaras inteligentes, que incluyen dispositivos Raspberry Pi para distribuir la capacidad de procesamiento y descentralizar la información de video.

Ejemplos de estudio se incluyen la implementación de arquitecturas como mAlexNet o mLeNet [2] (2016). Además, se han explorado enfoques con CarNet, Xception y ResNet50 para la clasificación de plazas vacantes [27] (2019). Además, se ha aplicado el marco de trabajo Detectron2 para implementar modelos como Faster R-CNN.

Estos métodos requieren etiquetado manual de un escenario de estacionamiento para su entrenamiento lo cual puede no ser generalizable en otras escenas de estacionamiento. También hay que considerar que estos métodos de detección de objetos o clasificación deben superar muchas dificultades como el clima desfavorable, la oclusión entre objetos así como el ángulo de visión de la cámara que pueden provocar errores de juicio por parte del modelo tanto al momento de entrenarlo como al de utilizarlo.

En el ámbito de las notificaciones temporales de plazas vacantes, se han propuesto diversos enfoques que combinan modelos probabilísticos y aprendizaje profundo. Los modelos de Markov [20] (2010) proporcionan un marco sólido para modelar las transiciones entre estados en sistemas dinámicos, como la disponibilidad de plazas vacantes en estacionamientos. Los árboles de regresión [47] (2015) son herramientas que permiten modelar relaciones no lineales y heterogéneas en los datos, lo que puede resultar valioso para capturar la complejidad de los patrones de estacionamiento. Las máquinas de vectores de soporte [43] (2018) han demostrado su eficacia en el contexto de notificaciones de plazas vacantes, Además, estas técnicas pueden ser aplicadas de manera conjunta para aprovechar sus ventajas complementarias. Dentro de este conjunto diverso de enfoques, como el algoritmo de la mosca de la fruta , los modelos LSTM o redes neuronales de memoria a largo plazo que son una variante de las redes neuronales recurrentes pueden capturar relaciones temporales en los datos [10] (2018) siendo una alternativa a considerar.

Cada uno de estos enfoques ofrece sus propias ventajas y desafíos; sin embargo, los modelos LSTM han emergido como una opción particularmente prometedora para modelar los patrones de comportamiento en estos entornos [45] (2017), especialmente cuando los datos se segmentan en diferentes patrones según el comportamiento de los conductores. Un estudio llevado a cabo en una extensa área metropolitana de China [40] (2019) respalda la efectividad de los modelos LSTM en la predicción de la disponibilidad de plazas vacantes cuando

se tiene seguimiento de los vehiculos en las plazas. No obstante, existen oportunidades para explorar y mejorar la precisión y eficiencia de estos modelos cuando por privacidad no es posible dar seguimiento a particulares que ocupan las plazas o particularmente en el contexto de datos discontinuos como PKLot [9]. En esta memoria, se busca abordar algunas de las limitaciones identificadas en estudios previos respecto a los aspectos temporales, mediante la implementación y evaluación de estrategias modificadas que aspiran a superar y mejorar los enfoques existentes.

Capítulo 4

Desarrollo del trabajo

1. Introducción

Para comenzar, estableceremos una estrategia de investigación sistemática que nos permita abordar el problema de forma ordenada y eficiente. En el Diagrama 4.1, presentamos un diagrama de flujo de actividades, que constituye la primera aproximación de nuestra metodología de desarrollo de trabajo. Este se adaptará y especificará a lo largo del presente estudio para adecuarse a los objetivos y desafíos particulares de nuestra investigación.

En este capítulo, exploramos varias arquitecturas de redes neuronales convolucionales y recurrentes adaptadas específicamente para la tarea de detectar espacios de estacionamiento ocupados y libres. Estas arquitecturas se han elegido y/o adaptado para obtener el mejor modelo posible para nuestro objetivo.

2. Conjunto de Datos

2.1. Descripción del Conjunto de Datos

El conjunto de datos PKLot es un recurso valioso para la investigación en clasificación de espacios de estacionamiento. Contiene 12,417 imágenes de estacionamientos y 695,899 imágenes de espacios de estacionamiento etiquetadas manualmente. Estas imágenes se capturaron en los estacionamientos de la Universidad Federal de Paraná (UFPR) y la Pontificia Universidad Católica de

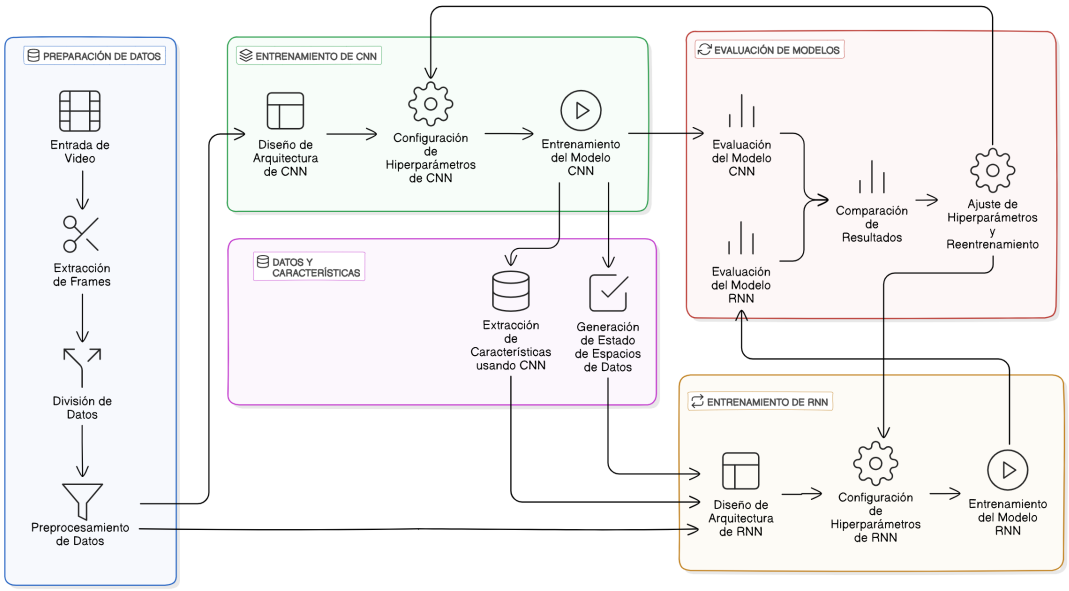

Figura 4.1: Diagrama de flujo que representa estrategia típica de evaluación de modelos CNN y RNN iterativa

Paraná (PUCPR) en Curitiba, Brasil.

Las imágenes se obtuvieron mediante una cámara de alta definición (Microsoft LifeCam, HD-5000) ubicada en la parte superior de un edificio. Se capturaron imágenes cada 5 minutos durante más de 30 días. El objetivo del set era obtener imágenes en diferentes condiciones climáticas, incluyendo días soleados, nublados y lluviosos [9] como se puede observar en la Figura 4.2.

2.2. Estructura de datos

Los datos se estructuran según la cámara que tomó las fotografías (PUC, UFPR04 y UFPR05). Dentro de cada categoría de cámara, se organizan según las condiciones climáticas (Sunny, Cloudy y Rainy). A continuación, en un nivel más detallado, cada categoría climática se divide por fechas diarias de captura, incluyendo tanto la fotografía como un archivo XML con anotaciones sobre los espacios de estacionamiento.

Adicionalmente, se ofrece una organización paralela de los datos. En esta, dentro del subgrupo de fechas diarias, se dividen en fotografías segmentadas que muestran cada espacio de estacionamiento en dos estados: *Empty* y *Occupied*.

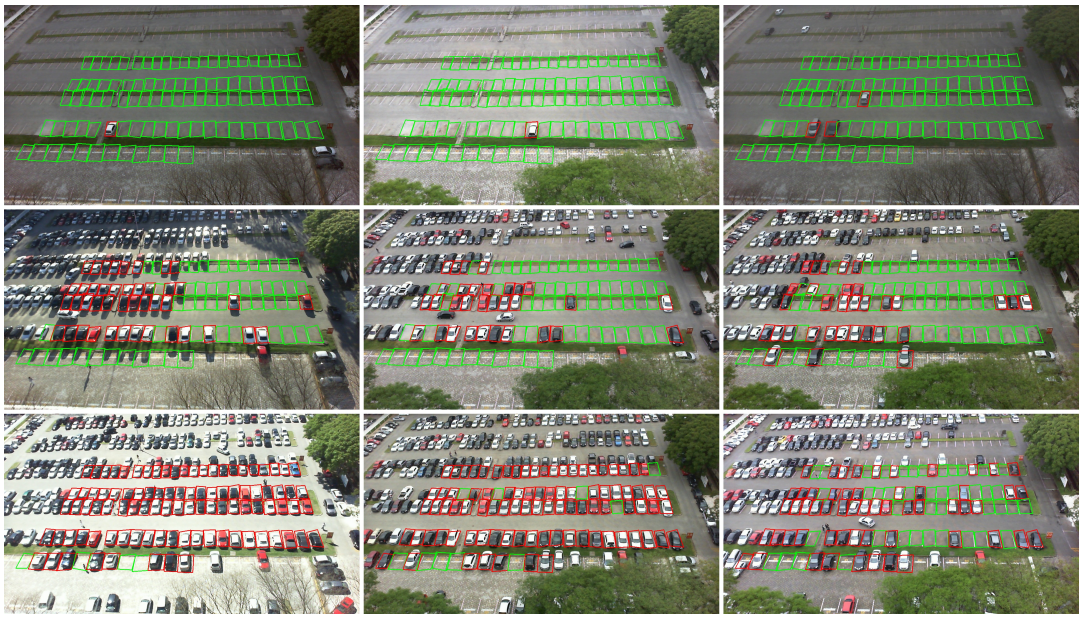

Figura 4.2: Ocupación creciente en la cámara PUC. En la columna a la izquierda 20-09-2012 (soleado), en la central 28-10-2012 (nublado) y en la columna a la derecha 16-09-2012 (lluvioso)

Finalmente, cada imagen segmentada lleva un nombre estructurado que combina la fecha completa (hasta segundos) con el espacio de estacionamiento correspondiente. En la Figura 4.5 se puede observar distintas imágenes segmentadas de forma aleatoria. Este formato facilita la generación de indicadores o conjuntos de datos sin necesidad de procesar directamente las imágenes.

2.3. Análisis de distribución de los datos

Los datos segmentados PKLot identifican un total de 358,071 imágenes clasificadas como *Empty* y 337,702 imágenes etiquetadas como *Occupied*. Estos números nos indican que la distribución entre ambas clases es bastante balanceada, lo que es esencial para garantizar un análisis robusto y un entrenamiento de modelos equitativo

En la Figura 4.4 se observa la Distribución imágenes por clases por cámara, por condición meteorológica, días laborales y días no laborales, por rango horario y por día de semana.

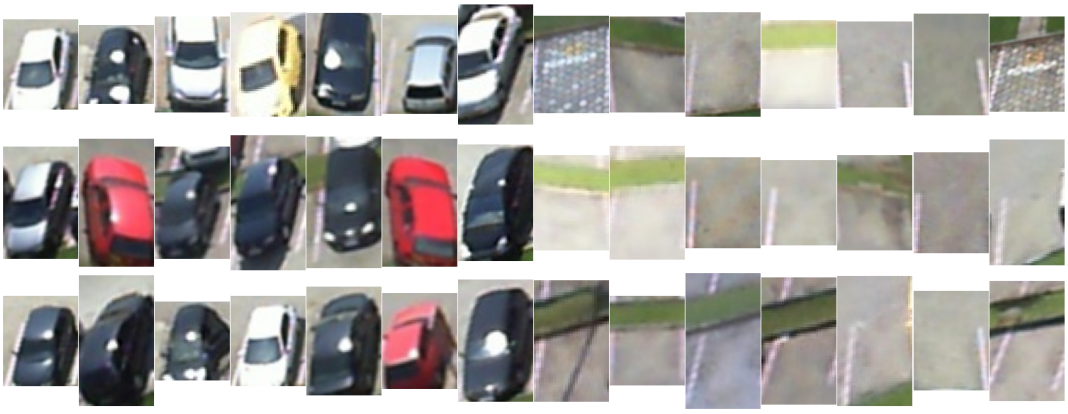

Figura 4.3: Imágenes segmentadas del día 27-10-2012 (soleado), diferenciadas entre espacios ocupados (izquierda) y vacíos (derecha).

Distribución de imagenes por clases y cámara

Analizando la distribución de imágenes por cámara, encontramos que:

- La cámara PUC contiene 229,994 imágenes *Empty* y 194,151 imágenes *Occupied*.
- La cámara UFPR04 alberga 59,718 imágenes *Empty* y 46,125 *Occupied*.
- En la cámara UFPR05, se registran 68,359 imágenes *Empty* y 97,426 *Occupied*.

Esto nos muestra que, si bien la cámara PUC mantiene una distribución bastante equilibrada entre las clases, en UFPR04 y UFPR05 se presentan diferencias notables en la proporción de clases.

Condición meteorológica

Al considerar las condiciones meteorológicas, observamos que:

- En días nublados, se registran 141,396 imágenes *Empty* y 87,735 *Occupied*.
- En jornadas lluviosas, contamos con 36,409 imágenes *Empty* y 63,534 *Occupied*.

- Durante días soleados, se identifican 180,266 imágenes *Empty* y 186,433 *Occupied*.

De este análisis, se desprende que en días nublados existe una mayor tendencia a encontrar estacionamientos vacíos, mientras que los días lluviosos suelen presentar una mayor ocupación. Los días soleados, en cambio, mantienen un equilibrio entre ambas clases.

Días laborales y días no laborales

En relación a la ocupación de acuerdo al tipo de día, se tiene que:

- Durante los días laborables, se contabilizan 169,338 imágenes *Empty* y 313,696 *Occupied*.
- En los días no laborables, se observan 188,733 imágenes *Empty* y 24,006 *Occupied*.

Estos datos sugieren que los estacionamientos tienden a estar más ocupados durante los días laborables, lo que es coherente con la rutina laboral y académica de la mayoría de las personas.

Distribución de Clases por Rango Horario

En cuanto al rango horario:

- Durante la mañana, se registran 321,791 imágenes *Empty* y 281,758 *Occupied*.
- En el mediodía, se contabilizan 28,028 imágenes *Empty* y 42,478 *Occupied*.
- Por la noche, se tienen 1,203 imágenes *Empty* y 875 *Occupied*.
- En otros horarios, sorprendentemente, no se tiene registro de imágenes.
- En la tarde, encontramos 7,049 imágenes *Empty* y 12,591 *Occupied*.

Estos datos nos señalan que la mayor actividad en los estacionamientos se da durante las mañanas, con una ocupación significativamente alta.

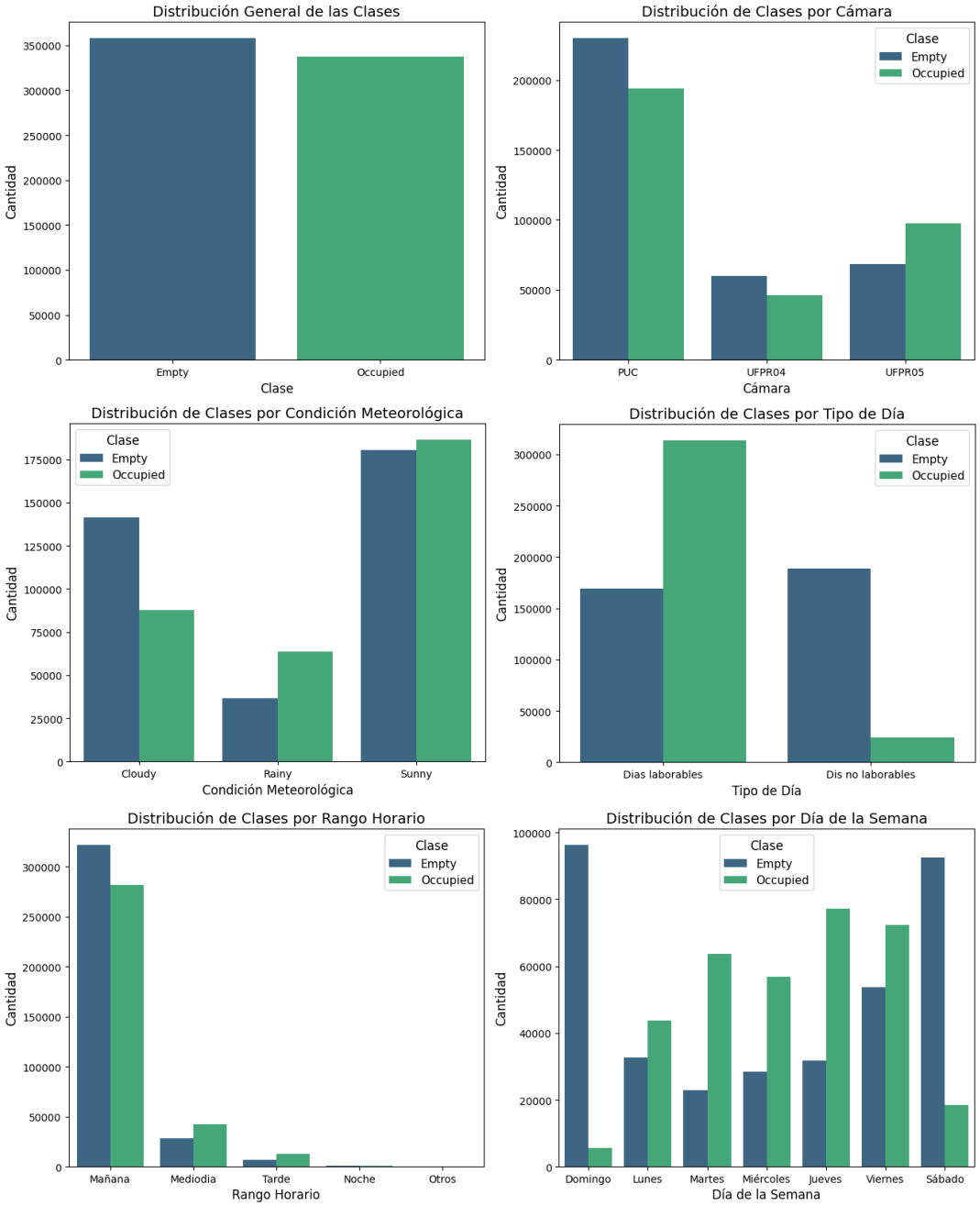

Figura 4.4: Análisis de Distribución de Clases del conjunto PKLot

Distribución de clases por día de la semana

Al analizar los días de la semana, se destaca que los días laborables presentan una mayor actividad en comparación con los fines de semana. Por ejemplo, el sábado se identifica una gran cantidad de espacios vacíos, lo que podría indicar una menor actividad en los estacionamientos ese día.

2.4. Análisis de distribución de los datos de camara PUC

Condición meteorológica en PUC

En la cámara PUC, la distribución de las imágenes según las condiciones meteorológicas es:

- Bajo condiciones nubladas: 90,431 imágenes *Empty* y 42,369 imágenes *Occupied*.
- En días lluviosos: 27,978 imágenes *Empty* y 55,122 *Occupied*.
- En días soleados: 117,332 imágenes *Empty* y 114,168 *Occupied*.

De aquí se desprende que, aunque en días soleados la distribución es relativamente equilibrada, en días nublados y lluviosos se observan diferencias notables, siendo más común encontrar estacionamientos vacíos en días nublados y más ocupados en días lluviosos.

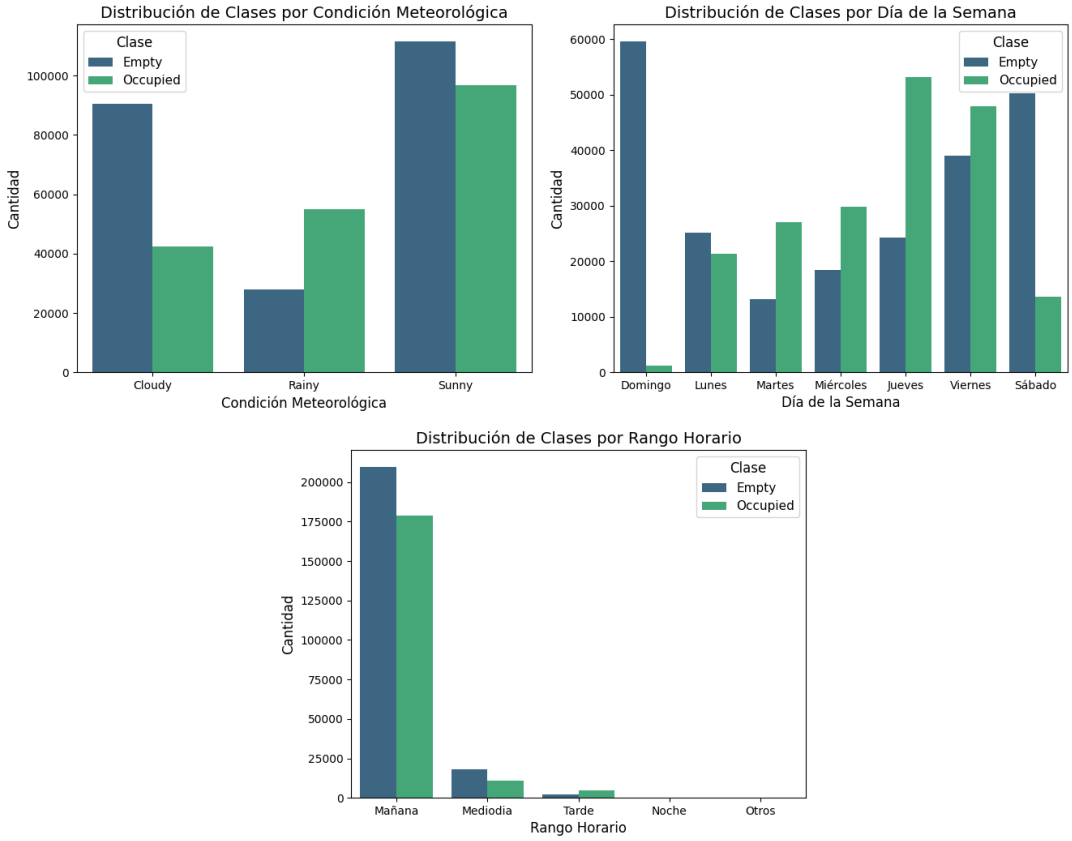

Figura 4.5: Análisis de Distribución de Clases en PUC del Conjunto PKLot

Distribución de clases por rango horario en PUC

En relación al rango horario en PUC, encontramos que:

- Durante la mañana, se registran 213,378 imágenes *Empty* y 194,676 *Occupied*.
- En el mediodía, se contabilizan 20,281 imágenes *Empty* y 12,263 *Occupied*.
- En la tarde, encontramos 2,082 imágenes *Empty* y 4,720 *Occupied*.

Estos datos resaltan una mayor actividad y muestreos fueron durante las mañanas en el estacionamiento de PUC.

Distribución de clases por día de la semana en PUC

Al observar la distribución por días de la semana en PUC, se destaca:

- Los viernes presentan 39,029 imágenes *Empty* y 48,071 *Occupied*.
- Los lunes muestran 25,110 imágenes *Empty* y 21,390 *Occupied*.
- Los sábados tienen una mayor cantidad de espacios vacíos con 50,301 imágenes *Empty* y 13,599 *Occupied*.
- Los domingos registran 59,659 imágenes *Empty* y tan solo 1,141 *Occupied*.
- Los jueves muestran una distribución de 24,284 imágenes *Empty* y 51,316 *Occupied*.
- Los martes presentan 16,810 imágenes *Empty* y 38,390 *Occupied*.
- Finalmente, los miércoles cuentan con 20,548 imágenes *Empty* y 37,752 *Occupied*.

De estos datos, se destaca que los domingos son los días con menor ocupación en el estacionamiento de PUC, mientras que los viernes y jueves tienden a mostrar una mayor actividad.

2.5. Análisis de las imágenes

En este análisis, se presentan y discuten las estadísticas derivadas del conjunto de datos PKLot referidos a la camara PUC y los 4474 registros temporales o segmentados en 447400 imágenes procesadas los cuales se reflejan en las Figuras 4.6, 4.7 y 4.8

- **Anchura (Width)**: Media de 48.68 píxeles, desviación estándar de 9.91 píxeles, con un rango de 33 a 77 píxeles y mediana de 47 píxeles. La anchura promedio de las imágenes es de aproximadamente 48.68 píxeles, con una variabilidad (desviación estándar) de 9.91 píxeles. La mayoría de las imágenes tienen una anchura entre 41 y 54 píxeles.
- **Altura (Height)**: Media de 52.95 píxeles, desviación estándar de 7.75 píxeles, rango de 38 a 82 píxeles y mediana de 54 píxeles. Las imágenes tienen una altura media de 52.95 píxeles y una variabilidad de 7.75 píxeles. La mayoría de las imágenes tienen alturas entre 46 y 60 píxeles.

- **Área de las Imágenes**: El área promedio de las imágenes es de 2631.28 píxeles cuadrados, con una desviación estándar de 851.86. Esto sugiere que, aunque las imágenes pueden variar en anchura y altura, tienden a tener un tamaño general similar.

- **Brillo (Brightness)**: El brillo promedio de las imágenes es de 139.56 con una variabilidad de 36.19. Las imágenes varían en brillo desde muy oscuras (25.20) hasta muy brillantes (252.31) lo cual es lógico dado el contexto diario horario de las mismas.

- **Contraste**: El contraste promedio es de 41.73, con una desviación estándar de 19.06. Esto indica que las imágenes tienen un nivel medio de contraste, aunque hay algunas que pueden ser muy planas (1.29) o muy contrastadas (93.46).

- **Entropía**: La entropía promedio es de 6.61, con una variabilidad de 0.77. La entropía es una medida de la complejidad de la información en la imagen. Una entropía más alta sugiere imágenes con más detalles o texturas como es este caso, mientras que una entropía baja puede indicar imágenes más uniformes o planas.

Características de Haralick

Las características de Haralick derivan de la matriz de co-ocurrencia y describen la textura de las imágenes. Basándonos en las estadísticas proporcionadas:

- **Haralick 1 (Contraste)**: El valor medio de 0.0068 sugiere que la mayoría de las imágenes tienen un contraste bajo en términos de textura, lo que indica una transición suave entre los píxeles vecinos.

- **Haralick 2 (Homogeneidad)**: El valor promedio de 101.44 sugiere una alta homogeneidad en las texturas de las imágenes.

- **Haralick 3 (Contraste de segundo ángulo)**: Con una media de 0.9718 y un rango entre 0.8361 a 0.999, las imágenes muestran un buen nivel de contraste en el segundo ángulo, lo que sugiere una variabilidad en las texturas.

- **Haralick 4 (Suma de cuadrados - Varianza)**: Con un valor promedio de 1911.90, esta característica mide la variabilidad de la distribución de niveles de gris en la imagen. Una alta varianza podría indicar una mayor variabilidad en las texturas de las imágenes, sugiriendo una rica presencia de detalles y patrones.

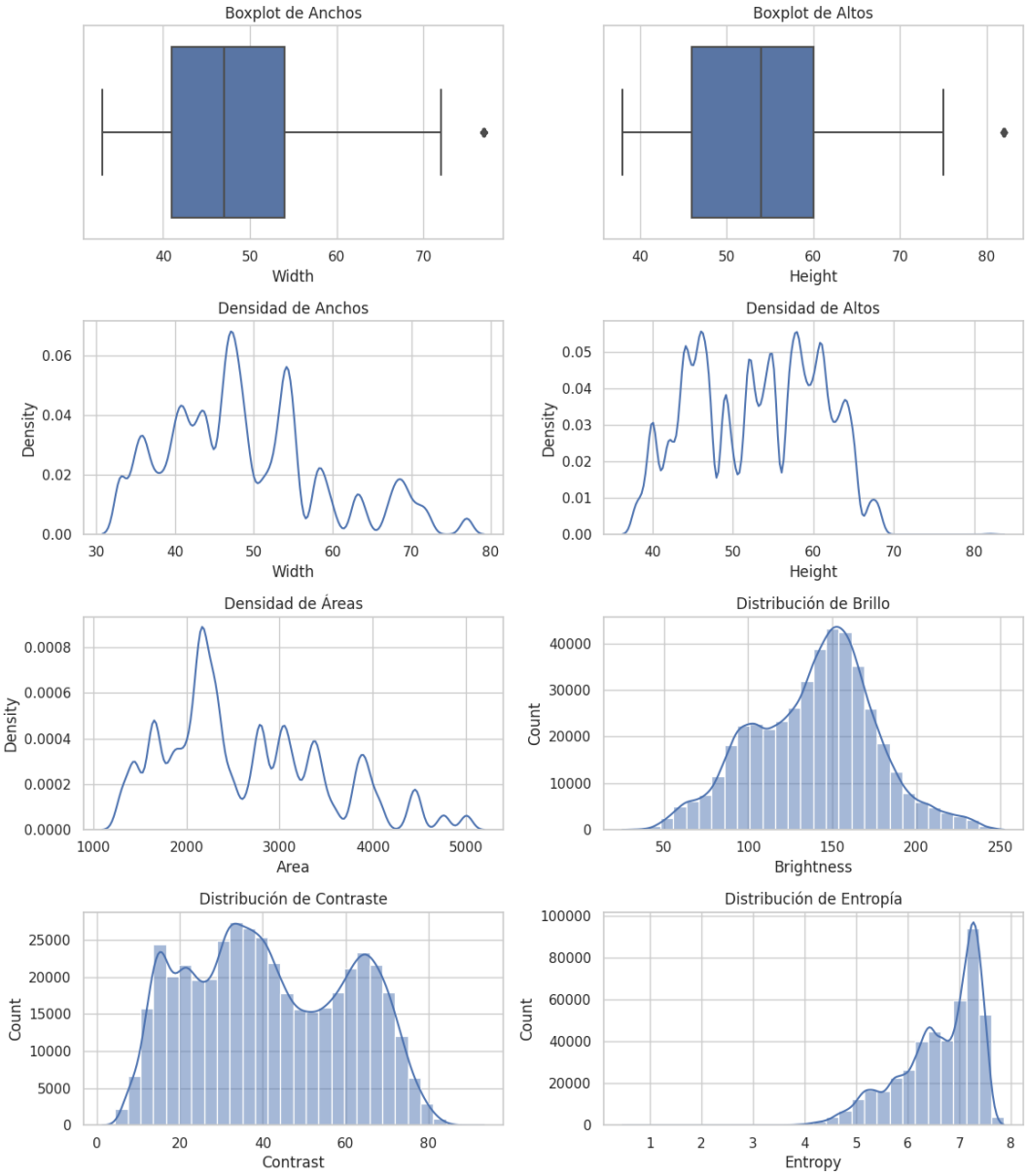

Figura 4.6: Análisis de imágenes de cámara PUC. Alto, ancho, área, brillo, contraste y entropía.

- **Haralick 5 (Suma promedio)**: Con un promedio de 0.5048, esta métrica refleja la relación entre los valores de los píxeles y sus vecinos. Un valor cercano a 0.5 sugiere que hay un equilibrio en la relación de los niveles de gris entre los píxeles vecinos.

- **Haralick 6 (Suma de varianza)**: El valor promedio de 281.85 indica que hay una variabilidad en la suma de los valores de los píxeles vecinos. Esto puede reflejar la presencia de diferentes texturas y patrones en las imágenes.

- **Haralick 7 (Suma de entropía)**: Con un valor promedio de 7546.18, indica la complejidad de las texturas en las imágenes. como en este caso donde hay muchas imágenes con suma de entropía baja y otra cierta cantidad por encima de 15250.

- **Haralick 8 (Entropía)**: Con un promedio de 7.41, esta métrica refleja la complejidad de la distribución de los niveles de gris similar al análisis de Entropía.

- **Haralick 9 (Diferencia de varianza)**: Con un promedio de 9.76, esta característica mide la variabilidad en la diferencia entre los valores de los píxeles y sus vecinos.

- **Haralick 10 (Diferencia de entropía)**: Con un valor promedio de 0.0010, indica la variabilidad en las diferencias entre los niveles de gris de los píxeles y sus vecinos. Un valor bajo sugiere que las transiciones de textura en las imágenes son bastante uniformes lo cual concuerda con que las imágenes tienen transiciones suaves.

- **Haralick 11 (Medida de información de correlación 1)**: El promedio de 3.14 sugiere una correlación moderada entre los niveles de gris de un píxel y sus vecinos. Esto puede indicar que las texturas en las imágenes tienen patrones repetitivos o correlacionados lo cual destacara luego al buscar patrones en las imágenes.

- **Haralick 12 (Medida de información de correlación 2)**: Con un promedio de -0.498, esta métrica indica la redundancia en la información de textura en las imágenes. Un valor cercano a cero sugiere que hay una buena cantidad de información única en las texturas de las imágenes.

- **Haralick 13 (Maximal Correlation Coefficient)**: Con un promedio de 0.9984, indica una alta correlación entre los niveles de gris de un píxel y sus vecinos. Esto sugiere que las texturas en las imágenes son coherentes y tienen patrones bien definidos.

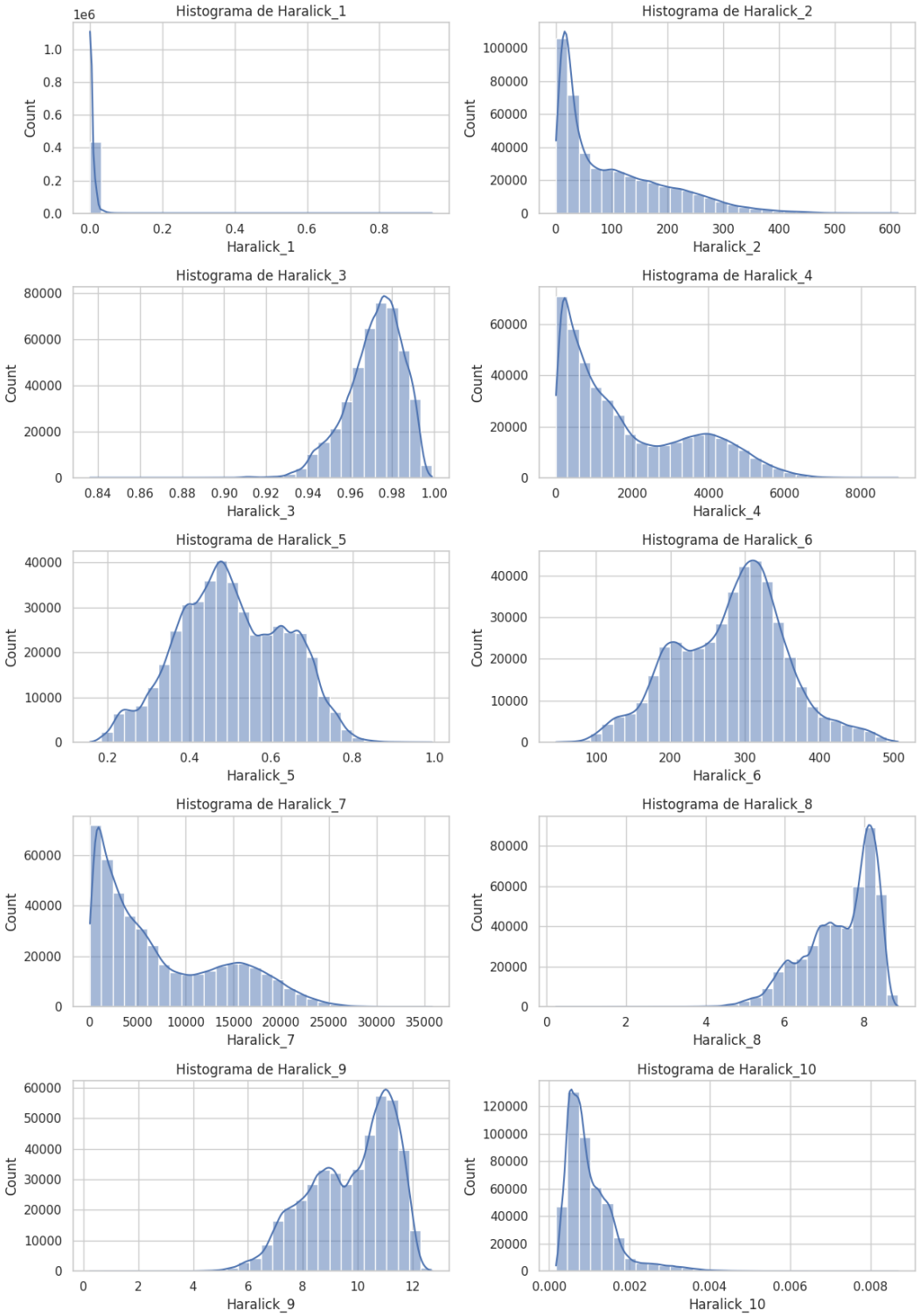

Figura 4.7: Análisis de imágenes de cámara PUC. Haralick 1 a 10

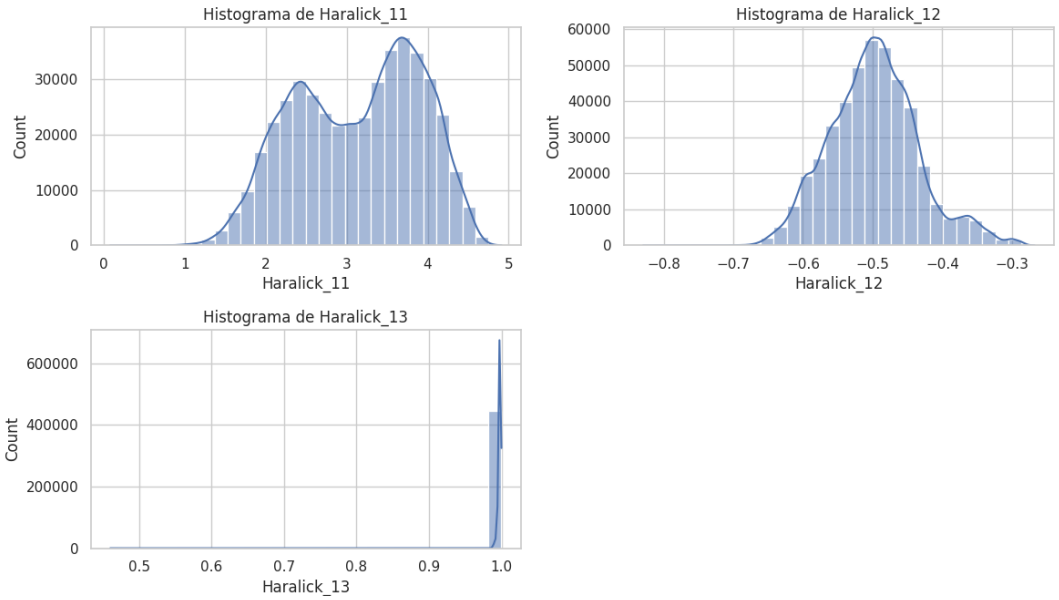

Figura 4.8: Análisis de imágenes de cámara PUC. Haralick 11 a 13

Conclusiones Generales

En este conjunto de datos, hay una considerable variación en muchas de las características, lo que sugiere una amplia variedad de texturas en las imágenes. Los valores de distintas métricas sugieren que hay muchas imágenes con texturas complejas. y niveles de correlación que permitirán extraer características interesantes para el proyecto.

2.6. Análisis temporal de la ocupación de estacionamiento

Dado que la mayor cantidad de datos proviene de la cámara PUC y a su vez estos datos son los más equilibrados bajo distintas condiciones climáticas, de ocupación y temporales se decidió acotar los datos muestréales a esta cámara. Por lo tanto, es esencial comprender la naturaleza temporal subyacente de la ocupación del estacionamiento. Esta sección busca identificar patrones y tendencias que se presenten en diferentes días de la semana como los que puede reflejar la Tabla 4.1.

Primero se agrupo por día de la semana los estados de los espacios de estacionamiento en su conjunto y se utilizo el método “describe” de pandas para

Estad.	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
Media	46.00 %	69.55 %	64.75 %	67.88 %	55.19 %	21.28 %	1.88 %
Mediana	55.00 %	73.00 %	70.00 %	71.00 %	67.00 %	14.00 %	1.00 %
Desv. Est.	38.20 %	24.28 %	28.77 %	23.69 %	34.85 %	25.87 %	2.40 %
P25	3.00 %	61.00 %	48.00 %	58.00 %	15.00 %	1.00 %	0 %
P75	79.00 %	90.00 %	90.00 %	86.00 %	85.00 %	30.00 %	3 %
Min	0 %	1 %	0 %	1 %	0 %	0 %	0 %
Max	100 %	100 %	100 %	100 %	100 %	90 %	10 %
Skewness	-0.07	-1.26	-0.93	-1.11	-0.56	1.42	1.38

Cuadro 4.1: Estadísticas de ocupación de estacionamiento segmentadas por día

comprender que reflejan los estados temporales. Este método incluye:

- Media (mean): Representa el promedio aritmético de los datos. Es una medida central que puede ser útil para entender el valor "típico"dentro de un conjunto de datos

- Mediana: Es el valor que separa la mitad superior de la mitad inferior de un conjunto de datos. A diferencia de la media, la mediana es robusta frente a valores extremos.

- Desviación estándar: Mide la cantidad de variación o dispersión en el conjunto de datos. Un valor bajo indica que los datos tienden a estar cerca de la media, mientras que un valor alto indica que los datos están distribuidos en un rango más amplio.

- Min y Max: Estos valores indican, el valor mínimo y máximo dentro del conjunto de datos.

- Percentil 25 (P25) y Percentil 75 (P75): Estos valores, también conocidos como cuartiles, dividen el conjunto de datos en cuatro partes iguales. El P25 es el valor por debajo del cual se encuentra el 25 % de los datos, mientras que el P75 es el valor por debajo del cual se encuentra el 75 % de los datos. Juntos, estos valores pueden ser útiles para entender la dispersión y la forma de la distribución.

- Skewness (Asimetría): Esta métrica mide la asimetría de la distribución de datos alrededor de su media. Una asimetría positiva indica que la cola en el lado derecho de la distribución es más larga que la del lado izquierdo, lo que significa que la mayoría de los valores están a la izquierda de la media (y viceversa para una asimetría negativa).

Estadísticas de ocupación por día de la semana

A partir de los datos recopilados, se generaron estadísticas segmentadas por día de la semana para comprender mejor la dinámica de ocupación de los estacionamientos. En las Figuras 4.9, 4.10, 4.11 y 4.12 se representan el estado global de ocupación del estacionamiento. Cada punto en cada gráfico representa una fotografía. Los marcadores de tiempo muestran que las fotografías fueron tomadas con una regularidad de 5 minutos pero también hay muestras con distinta periodicidad.

- **Lunes:** Aunque la media de ocupación es cercana al 50 %, la distribución muestra una ligera inclinación hacia la derecha (skewness negativo), indicando que hay más registros con alta ocupación. Esto podría deberse a que es el inicio de la semana laboral y académica.

- **Martes:** Se observa una alta media de ocupación, con una distribución inclinada hacia la derecha, lo que sugiere que los estacionamientos tienden a estar más ocupados este día.

- **Miércoles:** Similar al Martes, se observa una media de ocupación elevada, aunque con una distribución menos inclinada hacia la derecha que el Martes.

- **Jueves:** La tendencia de alta ocupación continúa, siendo este uno de los días con mayor ocupación media.

- **Viernes:** Aunque la media de ocupación disminuye en comparación con los días anteriores, la distribución sigue inclinada hacia la derecha, lo que indica que aún hay una considerable cantidad de registros con alta ocupación.

- **Sábado:** Se observa una notable disminución en la ocupación media, y la distribución está inclinada hacia la izquierda (skewness positivo). Esto sugiere que hay menos actividad en los estacionamientos, probablemente debido a la ausencia de actividades laborales y académicas.

- **Domingo:** Este es el día con la menor ocupación media. La distribución está fuertemente inclinada hacia la izquierda, lo que indica que la mayoría de los registros muestran estacionamientos casi vacíos.

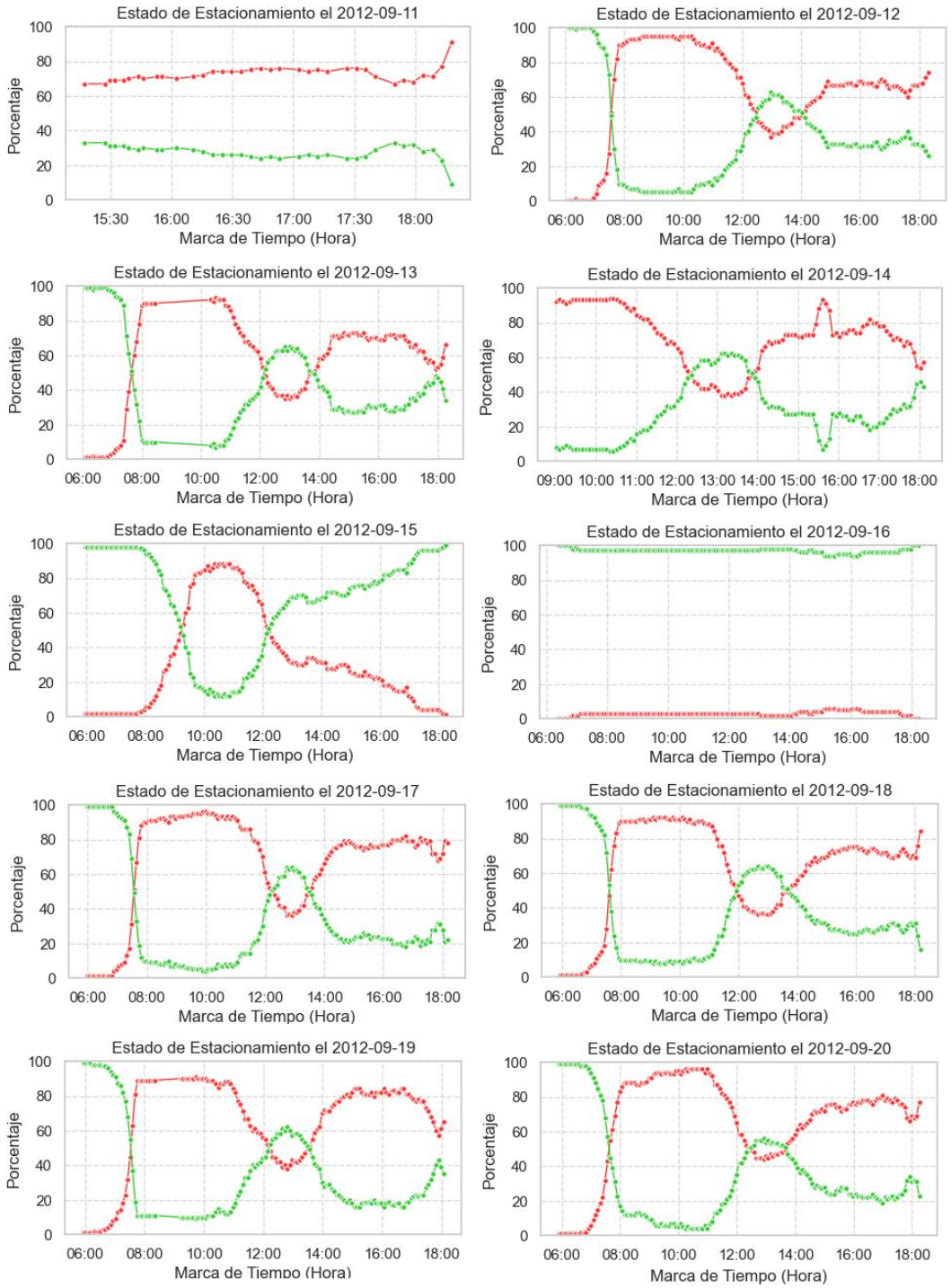

Figura 4.9: Ocupación y disponibilidad del estacionamiento durante el 11-09-2012 a 20-09-2012

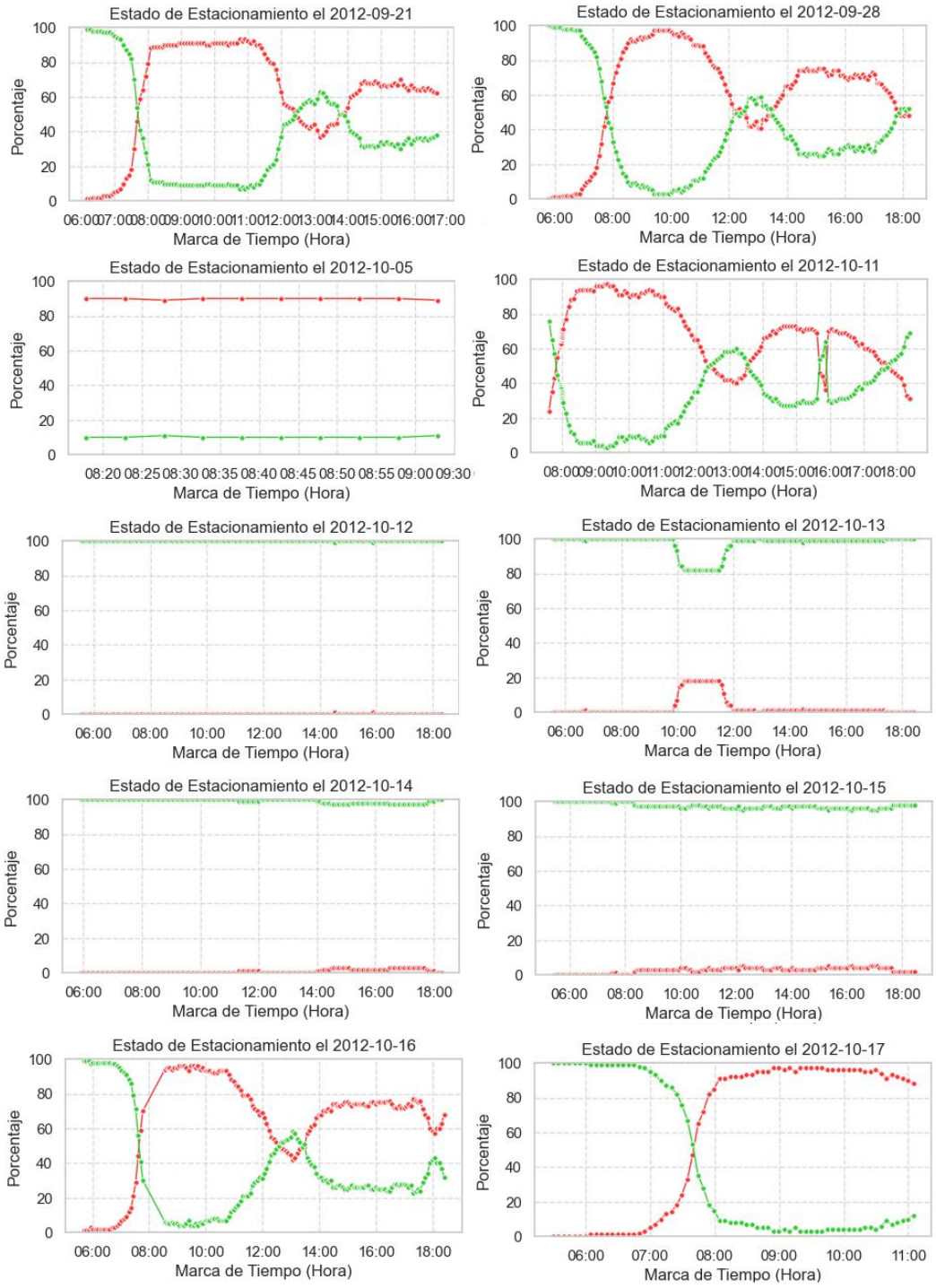

Figura 4.10: Ocupación y disponibilidad del estacionamiento durante el 21-09-2012 a 17-10-2012

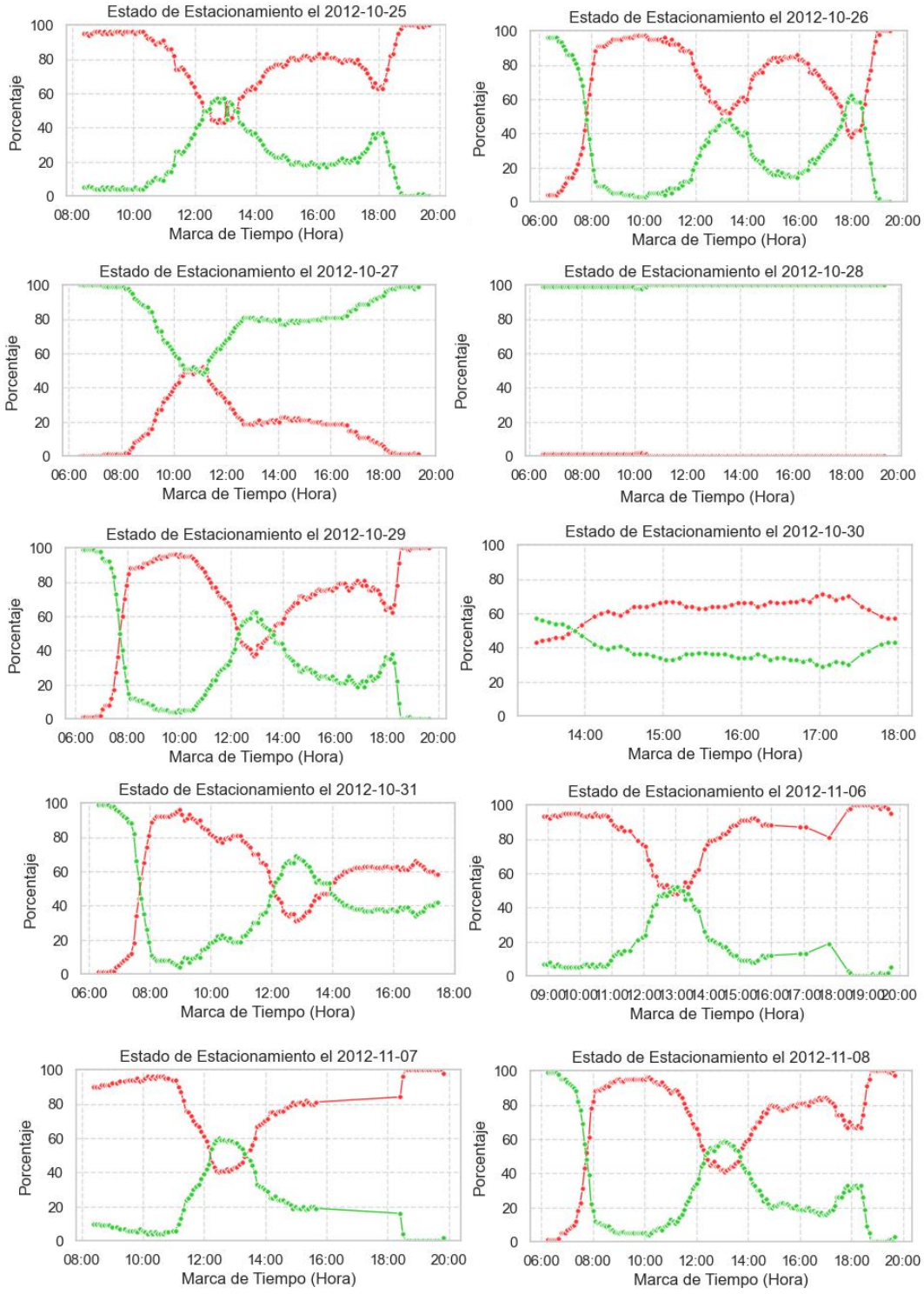

Figura 4.11: Ocupación y disponibilidad del estacionamiento durante el 25-10-2012 a 08-11-2012

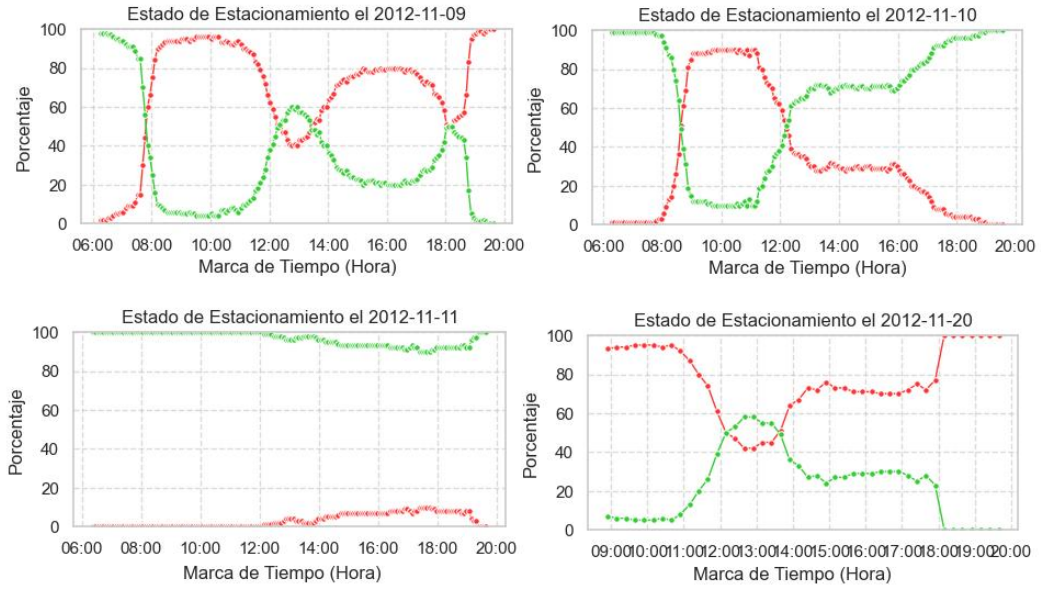

Figura 4.12: Ocupación y disponibilidad del estacionamiento durante el 09-11-2012 a 20-11-2012

Análisis temporal de la ocupación semanal por hora

A partir de los datos recopilados, se generaron estadísticas segmentadas por hora usando la media para evidenciar patrones claros y diferenciados en la dinámica de ocupación de los espacios de estacionamiento. En la Figura 4.13 es posible identificar las horas pico de ocupación y cómo estas varían según el día de la semana.

- **Días Laborables (Lunes a Viernes):** Durante las mañanas entre las 8:00 y las 10:00 hrs. se registra una notable elevación en la ocupación, alcanzando su pico máximo alrededor de las 9:00 hrs. A medida que avanza el día, se observa una disminución gradual en la ocupación, con un leve repunte en las primeras horas de la tarde, específicamente entre las 14:00 y las 16:00 hrs. Dado que el estacionamiento es universitario la ocupación vuelve a experimentar un auge significativo en las horas cercanas a las 19:00 hrs.

- **Fin de Semana (Sábado y Domingo):** El comportamiento durante los fines de semana es marcadamente distinto al de los días laborables. Los sábados presentan un incremento más moderado en la ocupación, con

un pico alrededor de las 9:00 hrs. Durante el domingo, la ocupación es notablemente baja.

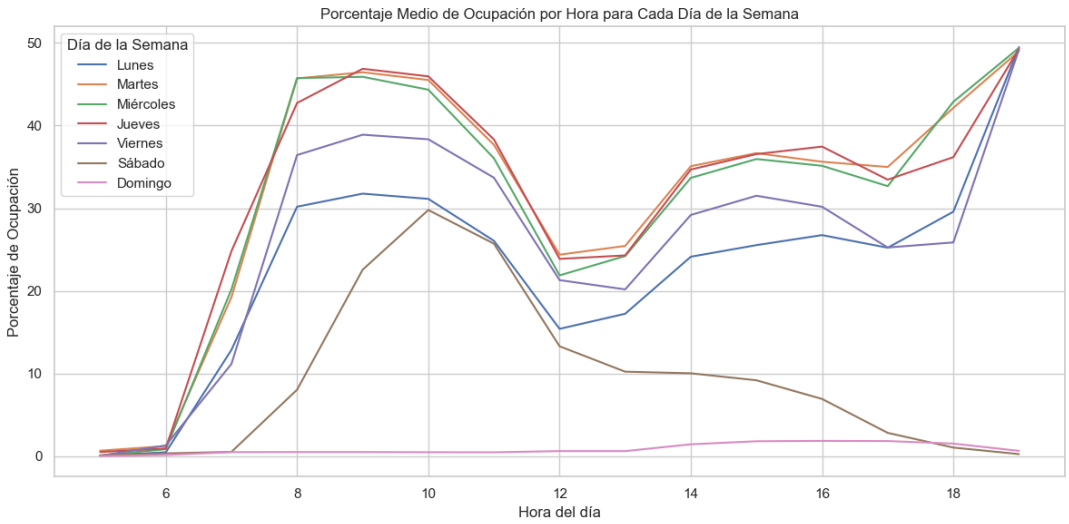

Figura 4.13: Representación visual de la ocupación media de los estacionamientos por hora a lo largo de cada día de la semana

Análisis de correlación de espacios de estacionamiento

Debido a que el proceso de selección de una plaza de estacionamiento es completamente aleatorio y que este factor es el más problemático para que un modelo temporal capture la naturaleza humana en la selección de los mismos, se realizó un análisis de correlación para comprender las relaciones entre diferentes espacios de estacionamiento, específicamente para identificar qué espacios muestran patrones similares en términos de ocupación y comprender si el procesamiento con un modelo temporal podrá detectar patrones con los estados de los mismos en los registros obtenidos de PUC en PKLot.

Esta información permitió generar un mapa de calor para visualizar las correlaciones entre los distintos espacios de estacionamiento y los datos muestreados que se encuentran en la Figura 4.14. En el mismo, se puede observar que, a pesar de que el comportamiento humano a priori parece ser aleatorio, existen indicios claros de que hay correlaciones entre los espacios de estacionamiento.

Los resultados de la correlación entre espacios de estacionamientos se reflejan ordenados de forma decreciente entre los 10 espacios de estacionamiento con

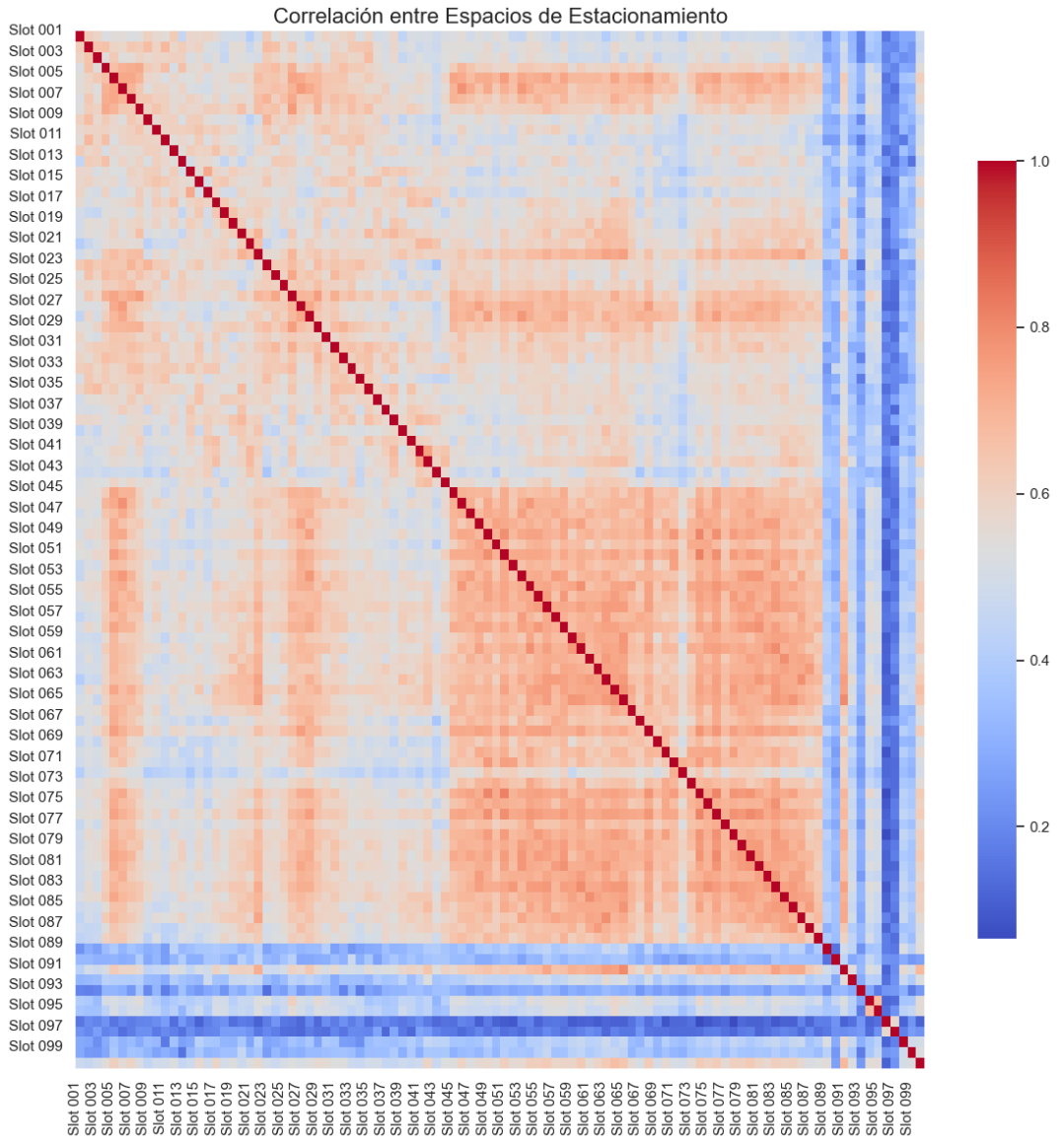

Figura 4.14: Mapa de Calor de correlaciones entre espacios de estacionamientos

mayor correlación con los demás espacios tienen una máxima correlación con los siguientes 5 espacios en la Tabla 4.2:

Espacio	1º		2º		3º		4º		5º		Media
074	050	0.8202	075	0.8126	048	0.8095	079	0.7798	067	0.7775	0.7999
076	073	0.8126	079	0.8014	050	0.7911	048	0.7855	077	0.7834	0.7948
051	073	0.8202	075	0.7911	048	0.7907	079	0.7622	077	0.7603	0.7849
049	073	0.8095	050	0.7907	075	0.7855	070	0.7655	077	0.7589	0.7820
065	090	0.7962	063	0.7839	084	0.7773	058	0.7731	082	0.7695	0.7800
080	076	0.8014	074	0.7798	053	0.7722	081	0.7700	058	0.7649	0.7777
083	060	0.7939	075	0.7830	065	0.7695	082	0.7683	056	0.7674	0.7764
081	058	0.7845	076	0.7789	053	0.7715	080	0.7700	063	0.7684	0.7747
060	083	0.7939	054	0.7854	059	0.7657	065	0.7640	082	0.7569	0.7732
078	076	0.7834	074	0.7759	083	0.7650	055	0.7608	051	0.7603	0.7691

Cuadro 4.2: Espacios de estacionamiento con correlación media mas alta respecto a otros 5 espacios mejor correlacionados

Por lo que refleja la Tabla 4.2 podemos comentar acerca de los mejores 6 casos que:

El espacio de estacionamiento 74 muestra una alta correlación con los espacios 50, 75, 48 , 79 y 67.

El espacio de estacionamiento 76 muestra una alta correlación con los espacios 73, 79, 50 , 48 y 77.

El espacio de estacionamiento 51 muestra una alta correlación con los espacios 73, 75, 48 , 79 y 77.

El espacio de estacionamiento 49 muestra una alta correlación con los espacios 73, 50, 75 , 70 y 77.

El espacio de estacionamiento 65 muestra una alta correlación con los espacios 90, 63, 84 , 58 y 52.

El espacio de estacionamiento 80 muestra una alta correlación con los espacios 76, 74, 53 , 81 y 58

En la Figura 4.15 señalamos estas 6 situaciones de ejemplo en las que se puede observar que las posiciones de estacionamiento correlacionadas tienen cierto grado de aleatoriedad posicional.

Por lo tanto, esto nos indica que existen patrones subyacentes en los datos que los modelos de series temporales podrán detectar dependiendo de como se diseñe la estructura de datos para ser entrenados.

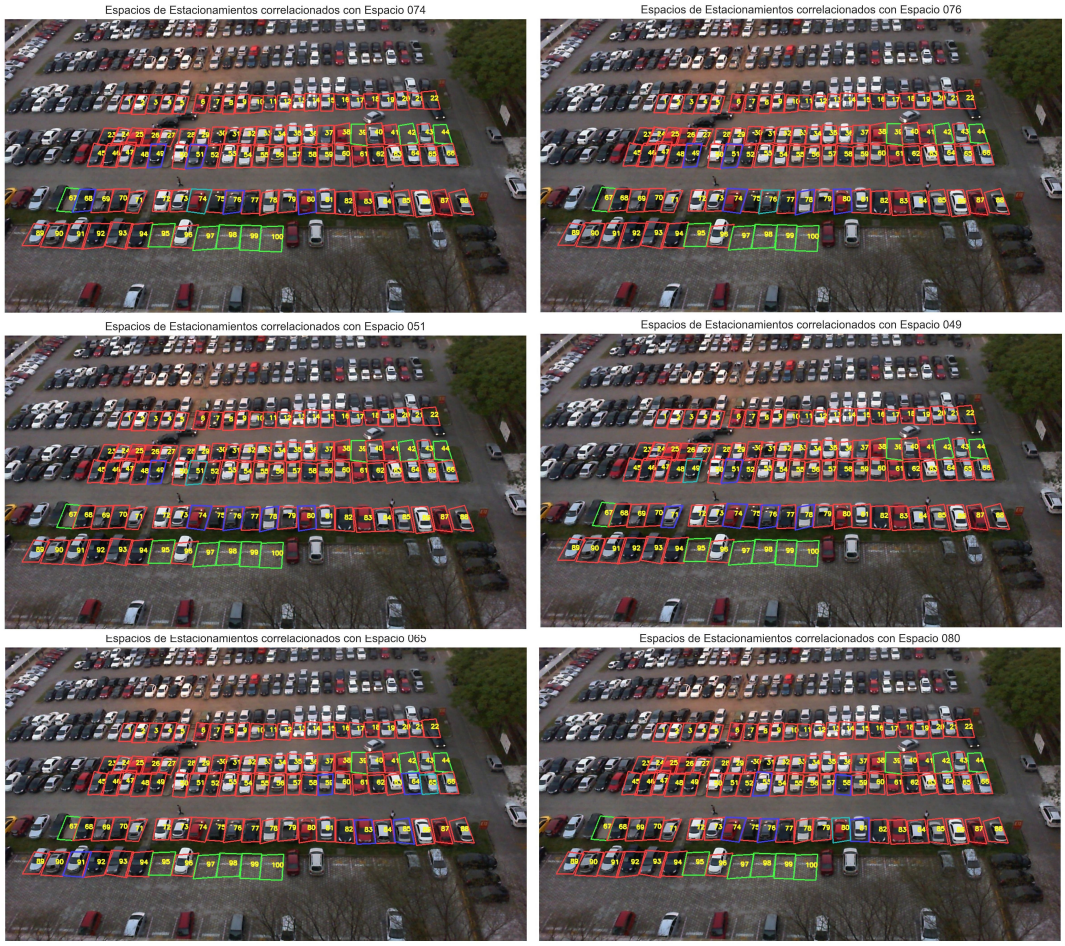

Figura 4.15: Imagen obtenida de PKLot del 11-09-2012 a las 18:18:05 con indicadores de distintos espacios con sus cinco espacios mas correlacionados

En cuanto al análisis pormenorizado de cada espacio de estacionamiento podemos decir que exceptuando los espacios 93 a 97 que están reservados para profesores de la Universidad todos los datos registrados muestran estar equilibrados entre ocupados y desocupados como se refleja en la Figura 4.16.

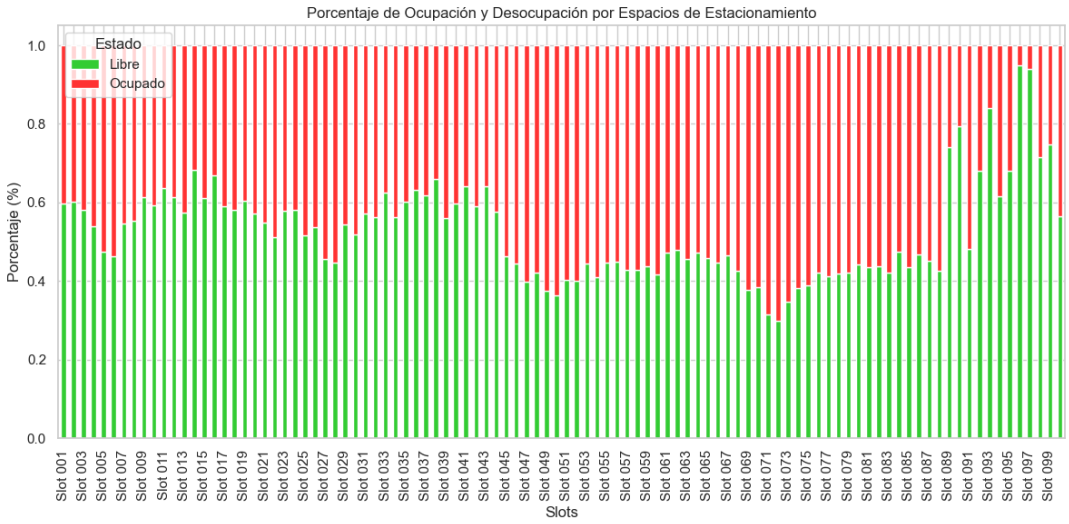

Figura 4.16: Porcentaje de ocupación total por espacios de estacionamiento

3. Investigación y selección de modelos Convolucionales

Dado el estado de arte y los distintos enfoques de la bibliografía investigada se eligieron modelos que poseen distinta naturaleza para adaptarlos a dos fines específicos.

Por un lado tener la capacidad de procesar imágenes de forma liviana y nos permitan capturar características generales mientras que en simultaneo se especialicen en la tarea de clasificación binaria por espacio de estacionamiento.

Por otro lado, se aprovecha el proceso de reducción final y clasificación binaria considerando una capa densa con 48 neuronas previa diseñada para servir como un extractor de características para la segunda parte del proyecto relacionada con la comunicación de información que enriquezca los modelos de series temporales seleccionados.

3.1. mLeNet (2014)

Considerando las arquitecturas bases de LeNet-5 (1998) propuesta por Yann LeCun y AlexNet propuesta por Alex Krizhevsky (2012), existe una adaptación de Amato et al. [1] que se inspira en ellas para crear modelos más pequeños

adaptados a la problemática de clasificación de estacionamientos. Esta adaptación se muestra en la Tabla 4.3.

Cuadro 4.3: Arquitectura de mLeNet para clasificación de imágenes

Tipo	**Operación**	**Kernel**	**Salida**	**Act.**	**Obs.**
Conv2D	Convolución	3x3	(224, 224, 32)	ReLU	
MaxPooling2D	Max Pooling	2x2	(112, 112, 32)	-	
Conv2D	Convolución	3x3	(112, 112, 64)	ReLU	
MaxPooling2D	Max Pooling	2x2	(56, 56, 64)	-	
Conv2D	Convolución	3x3	(56, 56, 128)	ReLU	
MaxPooling2D	Max Pooling	2x2	(28, 28, 128)	-	
Flatten	Aplanado	-	(100352)	-	
Dense	Densa	-	(512)	ReLU	
Dense	Densa	-	(48)	ReLU	Extr. Carac.
Dense	Densa (Salida)	-	(1)	Sigmoid	Clasif.

La arquitectura mLeNet comienza con una capa de convolución con 32 filtros y un tamaño de kernel de 3x3, seguida de una capa de MaxPooling. Este patrón se repite con capas de convolución y pooling adicionales, aumentando el número de filtros en cada capa de convolución. Después de las capas de convolución, la salida se aplana y se pasa a través de varias capas densas, finalizando con una capa de salida con una sola unidad y una activación sigmoide para la clasificación binaria.

El modelo LeNet original, propuesto por Yann LeCun, utilizaba solo dos capas convolucionales seguidas de capas densas para la tarea de reconocimiento de dígitos escritos a mano. La mLeNet de Amato et al. [1] es una versión adaptada y minimizada que toma inspiración de ambas arquitecturas para crear un modelo eficiente para la clasificación de espacios de estacionamiento.

3.2. mAlexNet (2014)

Otra adaptación de Amato et al. [1] se basa en la arquitectura de AlexNet propuesta por Alex Krizhevsky en 2012, creando un modelo más pequeño y adaptado para la clasificación de espacios de estacionamiento. Esta adaptación se muestra en la Tabla 4.4.

La arquitectura mAlexNet comienza con una capa de convolución con 16 filtros y un tamaño de kernel de 11x11, seguida de una capa de MaxPooling y una capa de normalización local de respuesta (LRN). Este patrón se repite con capas de convolución, pooling y normalización adicionales, aumentando el número de filtros en cada capa de convolución. Después de las capas de convolución, la

Cuadro 4.4: Arquitectura de mAlexNet para clasificación de imágenes

Tipo	**Operación**	**Kernel**	**Salida**	**Act.**	**Obs.**
Conv2D	Convolución	11x11	(224, 224, 16)	ReLU	
MaxPooling2D	Max Pooling	3x3	(112, 112, 16)	-	
LRN	Normalización	-	(112, 112, 16)	-	
Conv2D	Convolución	5x5	(112, 112, 20)	ReLU	
MaxPooling2D	Max Pooling	3x3	(56, 56, 20)	-	
LRN	Normalización	-	(56, 56, 20)	-	
Conv2D	Convolución	3x3	(56, 56, 30)	ReLU	
MaxPooling2D	Max Pooling	3x3	(28, 28, 30)	-	
Flatten	Aplanado	-	(23520)	-	
Dense	Densa	-	(512)	ReLU	
Dense	Densa	-	(48)	ReLU	Extr. Carac.
Dense	Densa (Salida)	-	(1)	Sigmoid	Clasif.

salida se aplana y se pasa a través de varias capas densas, finalizando con una capa de salida con una sola unidad y una activación sigmoide para la clasificación binaria.

El modelo AlexNet original, introducido por Alex Krizhevsky, fue un modelo mucho más grande y profundo diseñado para el desafío ImageNet, con más capas convolucionales y la incorporación de técnicas como ReLU y dropout. La mAlexNet de Amato et al. [1] es una versión adaptada y minimizada que toma inspiración de AlexNet para crear un modelo eficiente para la clasificación de espacios de estacionamiento.

3.3. VGG (2014)

La arquitectura VGG, propuesta por Simonyan y Zisserman en 2014, se caracteriza por su profundidad y el uso de capas convolucionales con tamaños de kernel de 3×3. La adaptación de Amato et al. [35] se inspira en la VGG16 original, creando una versión más compacta y adaptada para la clasificación de espacios de estacionamiento. Esta adaptación se muestra en la Tabla 4.5.

Arquitectura:

- **Capas Convolucionales**: Utiliza capas convolucionales con tamaños de kernel de 3×3, manteniendo la profundidad y la capacidad de aprendizaje de características detalladas.
- **Capas de MaxPooling**: Cada bloque de capas convolucionales es segui-

do por una capa de MaxPooling, lo que reduce las dimensiones espaciales y permite el aprendizaje de características más abstractas.

Cuadro 4.5: Arquitectura de VGG para clasificación de imágenes

Tipo	**Operación**	**Kernel**	**Salida**	**Act.**	**Obs.**
Conv2D	Convolución	3x3	(224, 224, 64)	ReLU	
Conv2D	Convolución	3x3	(224, 224, 64)	ReLU	
MaxPooling2D	Max Pooling	2x2	(112, 112, 64)	-	
Conv2D	Convolución	3x3	(112, 112, 128)	ReLU	
Conv2D	Convolución	3x3	(112, 112, 128)	ReLU	
MaxPooling2D	Max Pooling	2x2	(56, 56, 128)	-	
Conv2D	Convolución	3x3	(56, 56, 256)	ReLU	
Conv2D	Convolución	3x3	(56, 56, 256)	ReLU	
Conv2D	Convolución	3x3	(56, 56, 256)	ReLU	
MaxPooling2D	Max Pooling	2x2	(28, 28, 256)	-	
Conv2D	Convolución	3x3	(28, 28, 512)	ReLU	
Conv2D	Convolución	3x3	(28, 28, 512)	ReLU	
Conv2D	Convolución	3x3	(28, 28, 512)	ReLU	
MaxPooling2D	Max Pooling	2x2	(14, 14, 512)	-	
Conv2D	Convolución	3x3	(14, 14, 512)	ReLU	
Conv2D	Convolución	3x3	(14, 14, 512)	ReLU	
Conv2D	Convolución	3x3	(14, 14, 512)	ReLU	
MaxPooling2D	Max Pooling	2x2	(7, 7, 512)	-	
Flatten	Aplanado	-	(25088)	-	
Dense	Densa	-	(2048)	ReLU	
Dense	Densa	-	(1024)	ReLU	
Dense	Densa	-	(512)	ReLU	
Dense	Densa	-	(48)	ReLU	Extr. Carac.
Dense	Densa (Salida)	-	(1)	Sigmoid	Clasif.

La arquitectura VGG comienza con varios bloques de capas convolucionales, cada uno seguido de una capa de MaxPooling. Este patrón se repite con un número creciente de filtros en cada bloque, desde 64 hasta 512 filtros. Después de las capas de convolución, la salida se aplana y se pasa a través de varias capas densas, incluyendo una capa densa con 48 unidades para la extracción de características y una capa de salida con una sola unidad y una activación sigmoide para la clasificación binaria.

El modelo VGG original, propuesto por Simonyan y Zisserman, fue uno de los primeros en mostrar que el aumento de la profundidad de la red, manteniendo tamaños de kernel pequeños, podría mejorar significativamente el rendimiento en tareas de clasificación de imágenes. La versión adaptada por Amato et al. [35] reduce la complejidad y el número de parámetros del modelo original para

hacerlo más adecuado a la clasificación de espacios de estacionamiento.

3.4. ResNet50 (2015)

ResNet, o Redes Residuales", fue introducida por Kaiming He et al. [14] en 2015. Esta arquitectura aborda el problema del degradado del entrenamiento en redes profundas mediante **conexiones residuales** o **saltos**, que permiten a la señal sortear capas. ResNet50, específicamente, se refiere a una versión de ResNet con 50 capas. El diseño de estas conexiones residuales permite entrenar redes considerablemente más profundas sin sacrificar la precisión en el entrenamiento.

Para este proyecto, se adaptó ResNet50 utilizando el modelo preentrenado con ImageNet [19].

Arquitectura:

- **Conexiones Residuales**: Permiten a la señal sortear una o más capas, facilitando el entrenamiento de redes profundas.

- **Capas Convolucionales**: La arquitectura incluye múltiples capas convolucionales con diversos filtros y tamaños, interconectadas mediante conexiones residuales.

La arquitectura ResNet50 comienza con una capa de convolución y una capa de MaxPooling. Luego, se utilizan bloques residuales compuestos por varias capas convolucionales con diferentes tamaños de kernel y conexiones de salto, lo que facilita el entrenamiento de redes profundas. Los detalles de estos bloques se presentan en la Tabla 4.7. Después de las capas convolucionales y residuales, la salida se pasa a través de una capa de GlobalAvPool2D y varias capas densas, finalizando con una capa de salida con una sola unidad y una activación sigmoide para la clasificación binaria, como se detalla en la Tabla 4.6.

El modelo ResNet original, introducido por Kaiming He et al., resolvió el problema del degradado del entrenamiento en redes profundas, permitiendo entrenar modelos con un gran número de capas sin perder precisión. La versión adaptada para este proyecto utiliza ResNet50 preentrenada con ImageNet y añade capas densas específicas para la extracción de características y la clasificación de espacios de estacionamiento.

Cuadro 4.6: Arquitectura de ResNet50 para clasificación de imágenes

Tipo	**Operación**	**Kernel**	**Salida**	**Act.**	**Obs.**
Conv2D	Convolución	7x7	(112, 112, 64)	ReLU	
MaxPooling2D	Max Pooling	3x3	(56, 56, 64)	-	
		Bloques Residuales			
		Bloque Residual Tipo A			
		3 repeticiones			
		Bloque Residual Tipo B			
		4 repeticiones			
		Bloque Residual Tipo C			
		6 repeticiones			
		Bloque Residual Tipo D			
		3 repeticiones			
GlobalAvPool2D	Pooling Global	-	(2048)	-	
Dense	Densa	-	(2048)	ReLU	
Dense	Densa	-	(1024)	ReLU	
Dense	Densa	-	(512)	ReLU	
Dense	Densa	-	(48)	ReLU	Extr. Carac.
Dense	Densa (Salida)	-	(1)	Sigmoid	Clasif.

Cuadro 4.7: Detalles de los Bloques Residuales en ResNet50

Bloque	**Operación**	**Kernel**	**Salida**	**Act.**	**Obs.**
Tipo A	Conv2D	1x1	(56, 56, 64)	ReLU	
	Conv2D	3x3	(56, 56, 64)	ReLU	
	Conv2D	1x1	(56, 56, 256)	-	Conexión de Salto
Tipo B	Conv2D	1x1	(56, 56, 128)	ReLU	
	Conv2D	3x3	(28, 28, 128)	ReLU	
	Conv2D	1x1	(28, 28, 512)	-	Conexión de Salto
Tipo C	Conv2D	1x1	(28, 28, 256)	ReLU	
	Conv2D	3x3	(14, 14, 256)	ReLU	
	Conv2D	1x1	(14, 14, 1024)	-	Conexión de Salto
Tipo D	Conv2D	1x1	(14, 14, 512)	ReLU	
	Conv2D	3x3	(7, 7, 512)	ReLU	
	Conv2D	1x1	(7, 7, 2048)	-	Conexión de Salto

3.5. InceptionV3 (2015)

InceptionV3 es una arquitectura de red neuronal convolucional propuesta por Szegedy et al. [39] en 2015. Esta arquitectura es conocida por su diseño eficiente, que permite una alta capacidad representacional debido a sus módulos Inception, que permiten que la red realice múltiples operaciones en paralelo y luego las concatene. En este proyecto, se adaptó InceptionV3 utilizando el modelo preentrenado con ImageNet [19] para especializarse en la tarea de clasificación binaria de espacios de estacionamiento.

Arquitectura:

- **Módulos Factorizados**: Utiliza convoluciones factorizadas para reducir el número de parámetros y aumentar la eficiencia computacional.
- **Expansiones Asimétricas**: Aplica convoluciones asimétricas (por ejemplo, $1 \times n$ seguida de $n \times 1$) para capturar patrones espaciales.
- **Módulos Inception**: Estos módulos aplican varias operaciones en paralelo y luego concatenan sus salidas, lo que permite a la red aprender características a diferentes escalas y complejidades, como se detalla en la Tabla 4.8.
    - **Tipo A**: Diseñado para trabajar en imágenes de tamaño medio, utiliza combinaciones de convoluciones 1×1, 1×3 y 3×1, junto con pooling promedio para extraer características a diferentes escalas.
    - **Tipo B**: Actúa sobre imágenes de mayor resolución, combinando convoluciones 1×1, 1×7 y 7×1 para capturar patrones en diferentes orientaciones y escalas.
    - **Tipo C**: Se centra en imágenes de alta resolución, con convoluciones 1×1, 1×3 y 3×1, adaptadas para trabajar con detalles más finos.

Módulos de Reducción: Se utilizan para reducir la dimensión espacial de las imágenes y aumentar la profundidad, permitiendo a la red aprender características más abstractas y reducir el coste computacional de las capas subsiguientes, como se detalla en la Tabla 4.9.

La arquitectura InceptionV3 comienza con varias capas de convolución y MaxPooling, seguidas de una serie de módulos Inception de diferentes tipos (A, B y C), detallados en la Tabla 4.8, y módulos de reducción, detallados en la Tabla 4.9. Estos módulos permiten que la red aprenda características a diferentes escalas y complejidades, mejorando su capacidad de generalización.

Cuadro 4.8: Detalles de los Módulos Inception en InceptionV3

Módulo	Operación	Kernel	Salida	Act.	Obs.
Tipo A	Conv2D	1x1	(35, 35, 64)	ReLU	
	Conv2D	3x3	(35, 35, 96)	ReLU	
	AveragePooling2D	Pooling	(35, 35, 32)	-	
	Conv2D	1x1	(35, 35, 32)	ReLU	Concatenación
Tipo B	Conv2D	1x1	(17, 17, 128)	ReLU	
	Conv2D	7x1	(17, 17, 128)	ReLU	
	Conv2D	1x7	(17, 17, 192)	ReLU	
	MaxPooling2D	Pooling	(17, 17, 192)	-	Concatenación
Tipo C	Conv2D	1x1	(8, 8, 192)	ReLU	
	Conv2D	3x3	(8, 8, 320)	ReLU	
	AveragePooling2D	Pooling	(8, 8, 128)	-	Concatenación

Cuadro 4.9: Detalles de los Módulos de Reducción en InceptionV3

Módulo	Operación	Kernel	Salida	Act.	Obs.
Tipo A	Conv2D	3x3	(35, 35, 384)	ReLU	
	Conv2D	3x3	(35, 35, 192)	ReLU	
	Conv2D	3x3	(35, 35, 128)	ReLU	
	MaxPooling2D	Pooling	(35, 35, 128)	-	Concatenación
Tipo B	Conv2D	3x3	(17, 17, 192)	ReLU	
	Conv2D	1x7	(17, 17, 192)	ReLU	
	Conv2D	7x1	(17, 17, 192)	ReLU	
	MaxPooling2D	Pooling	(17, 17, 192)	-	Concatenación

Cuadro 4.10: Arquitectura de InceptionV3 para clasificación de imágenes

Tipo	**Operación**	**Kernel**	**Salida**	**Act.**	**Obs.**
Conv2D	Convolución	3x3	(149, 149, 32)	ReLU	
Conv2D	Convolución	3x3	(147, 147, 32)	ReLU	
Conv2D	Convolución	3x3	(147, 147, 64)	ReLU	
MaxPooling2D	Max Pooling	3x3	(73, 73, 64)	-	
Conv2D	Convolución	1x1	(73, 73, 80)	ReLU	
Conv2D	Convolución	3x3	(71, 71, 192)	ReLU	
MaxPooling2D	Max Pooling	3x3	(35, 35, 192)	-	
		Módulos Inception			
		Módulo Inception Tipo A			
		3 repeticiones			
		Módulo Inception Tipo B			
		4 repeticiones			
		Módulo Inception Tipo C			
		2 repeticiones			
		Módulos de Reducción			
		Módulo de Reducción Tipo A			
		1 repetición			
		Módulo de Reducción Tipo B			
		1 repetición			
Dense	Densa	-	(2048)	ReLU	
Dense	Densa	-	(1024)	ReLU	
Dense	Densa	-	(512)	ReLU	
Dense	Densa	-	(48)	ReLU	Extr. Carac.
Dense	Densa (Salida)	-	(1)	Sigmoid	Clasif.

Después de los módulos Inception y de reducción, la salida se aplana y se pasa a través de varias capas densas, finalizando con una capa de salida con una sola unidad y una activación sigmoide para la clasificación binaria, como se refleja en la Tabla 4.10.

3.6. MobileNetV2 (2018)

MobileNetV2 es una mejora sobre la arquitectura original de MobileNet, presentada por Sandler et al. [30] en 2018. Esta arquitectura es conocida por su ligereza y eficiencia, ideal para dispositivos móviles y sistemas embebidos, como cámaras que procesen la detección de espacios de estacionamiento. MobileNetV2 introduce el concepto de bloques inversos residuales, donde en lugar de reducir primero la dimensionalidad y luego expandirla, se expande primero y luego se reduce. Estos bloques emplean la técnica de convoluciones separables en profundidad, en la que una operación convolucional estándar se descompone en una convolución que opera en cada canal de entrada por separado (convolución en profundidad) seguida de una convolución 1×1 (convolución punto a punto). Esta técnica resulta en una reducción significativa del número de parámetros y de la carga computacional. Para el proyecto, se utilizó el modelo preentrenado con ImageNet [19], y su estructura se refleja en la Tabla 4.12.

Arquitectura:

- Se inicia con una capa convolucional estándar seguida de una serie de bloques residuales invertidos.
- Cada bloque utiliza:
  - Convoluciones separables en profundidad para capturar información espacial.
  - Convoluciones punto a punto para capturar relaciones entre canales.
- Estos bloques se conectan mediante conexiones residuales, lo que facilita el entrenamiento de la red y mejora la convergencia.

La arquitectura MobileNetV2 comienza con una capa de convolución estándar seguida de una serie de bloques residuales invertidos, detallados en la Tabla 4.11, que emplean convoluciones separables en profundidad y convoluciones punto a punto para capturar información espacial y relaciones entre canales, respectivamente. Estos bloques se conectan mediante conexiones residuales que facilitan el entrenamiento de la red y mejoran la convergencia. Después de las capas

Cuadro 4.11: Detalles de los Bloques Residuales en MobileNetV2

Bloque	**Operación**	**Kernel**	**Salida**	**Act.**	**Obs.**
Tipo A	DepthwiseConv2D	3x3	(112, 112, 32)	ReLU	
	Conv2D	1x1	(112, 112, 64)	ReLU	
	Conexión de Salto	-	(112, 112, 64)	-	
Tipo B	DepthwiseConv2D	3x3	(56, 56, 64)	ReLU	
	Conv2D	1x1	(56, 56, 128)	ReLU	
	Conexión de Salto	-	(56, 56, 128)	-	
Tipo C	DepthwiseConv2D	3x3	(28, 28, 128)	ReLU	
	Conv2D	1x1	(28, 28, 256)	ReLU	
	Conexión de Salto	-	(28, 28, 256)	-	

Cuadro 4.12: Arquitectura de MobileNetV2 para clasificación de imágenes

Tipo	**Operación**	**Kernel**	**Salida**	**Act.**	**Obs.**
Conv2D	Convolución	3x3	(112, 112, 32)	ReLU	
		Bloques Residuales Invertidos			
		Bloque Residual Tipo A			
		3 repeticiones			
		Bloque Residual Tipo B			
		4 repeticiones			
		Bloque Residual Tipo C			
		6 repeticiones			
GlobalAvPool2D	Pooling Global	-	(1280)	-	
Dense	Densa	-	(512)	ReLU	
Dense	Densa	-	(48)	ReLU	Extr. Carac.
Dense	Densa (Salida)	-	(1)	Sigmoid	Clasif.

convolucionales y residuales, la salida se pasa a través de una capa de GlobalAveragePooling2D y varias capas densas, finalizando con una capa de salida con una sola unidad y una activación sigmoide para la clasificación binaria, como se refleja en la Tabla 4.12.

3.7. MobileNetV3 (2019)

MobileNetV3 es una evolución de la arquitectura MobileNetV2, introducida por Howard et al. [17]. Esta arquitectura fue mejorada mediante técnicas de búsqueda automática, introduciendo bloques específicos optimizados por NAS (Neural Architecture Search) y la activación hard swish, una variante más eficiente de la función Swish.

Características nuevas:

- **Bloques específicos optimizados por NAS**: Estos bloques fueron encontrados automáticamente para proporcionar la mejor eficiencia y rendimiento.
- **Activaciónes**: En distintas partes del modelo se utiliza Una variante de la función de activación Swish que es más eficiente computacionalmente.

La especificación Large posee distintos operadores Bottleneck y diferentes configuraciones para cada uno como se aprecia en la Tabla 4.13 y la Tabla 4.14

Cuadro 4.13: Detalles de los Bloques Bottleneck en MobileNetV3

Bloque	Operación	Kernel	Salida	Act.	Obs.
Tipo A	Conv2D	1x1	(112, 112, 16)	ReLU	
	DepthwiseConv2D	3x3	(112, 112, 16)	ReLU	
	Conv2D	1x1	(112, 112, 24)	Linear	Conexión de Salto
Tipo B	Conv2D	1x1	(56, 56, 72)	ReLU	
	DepthwiseConv2D	3x3	(56, 56, 72)	ReLU	
	Conv2D	1x1	(56, 56, 24)	Linear	Conexión de Salto
Tipo C	Conv2D	1x1	(28, 28, 144)	Hard Swish	
	DepthwiseConv2D	3x3	(28, 28, 144)	Hard Swish	
	Conv2D	1x1	(28, 28, 48)	Linear	Conexión de Salto
Tipo D	Conv2D	1x1	(14, 14, 288)	Hard Swish	
	DepthwiseConv2D	3x3	(14, 14, 288)	Hard Swish	
	Conv2D	1x1	(14, 14, 96)	Linear	Conexión de Salto

La arquitectura MobileNetV3 comienza con una capa de convolución estándar seguida de una serie de bloques específicos optimizados mediante NAS, que se

Cuadro 4.14: Arquitectura de MobileNetV3 para clasificación de imágenes

Tipo	**Operación**	**Kernel**	**Salida**	**Act.**	**Obs.**
Conv2D	Convolución	3x3	(112, 112, 16)	Hard Swish	
		Bloques Bottleneck			
		Bloque Bottleneck Tipo A			
		1 repetición			
		Bloque Bottleneck Tipo B			
		4 repeticiones			
		Bloque Bottleneck Tipo C			
		3 repeticiones			
		Bloque Bottleneck Tipo D			
		3 repeticiones			
MaxPooling2D	Max Pooling	7x7	(1, 1, 1024)	-	
GlobalAvPool2D	Pooling Global	-	(1024)	-	
Dense	Densa	-	(512)	ReLU	
Dense	Densa	-	(48)	ReLU	Extr. Carac.
Dense	Densa (Salida)	-	(1)	Sigmoid	Clasif.

adaptan dinámicamente en función del factor de expansión y el tamaño del kernel. Estos bloques utilizan convoluciones separables en profundidad y convoluciones punto a punto, además de activaciones eficientes como hard swish y hard sigmoid. Después de las capas convolucionales y de los bloques Bottleneck, la salida se pasa a través de una capa de MaxPooling, una capa de GlobalAvPool2D y varias capas densas, finalizando con una capa de salida con una sola unidad y una activación sigmoide para la clasificación binaria.

El modelo MobileNetV3, introducido por Howard et al. [17], optimiza el uso de convoluciones separables en profundidad y bloques residuales invertidos, proporcionando una arquitectura eficiente para dispositivos móviles y sistemas embebidos. La versión adaptada para este proyecto utiliza MobileNetV3 preentrenada con ImageNet y añade capas densas específicas para la extracción de características y la clasificación de espacios de estacionamiento.

3.8. MobileNetV3 Small personalizada (2022)

Aprovechando las mejoras de MobileNetV3 y en la búsqueda de un mejor y más eficiente modelo para la detección de espacios de estacionamiento, se han explorado innovaciones significativas por Yuldashev et al. [46] para superar las limitaciones del modelo al manejar variaciones de iluminación y oclusiones, obteniendo mejoras significativas por las que se decidió incluirlas en el proyecto.

Mejoras y personalizaciones de la arquitectura:

- **Función de Activación Leaky-ReLU6**: Una modificación en la arquitectura original para mejorar la adaptabilidad del modelo a variaciones extremas y sutiles en las características de las imágenes de los estacionamientos.

- **Convolution Block Attention Module (CBAM)**: Esta mejora reemplaza el módulo **Squeeze and Excitation**, permitiendo que el modelo se enfoque más efectivamente en las características relevantes y mejore la capacidad de distinguir entre espacios ocupados y desocupados.

- **Blueprint Separable Convolutions (BSConv)**: Una adaptación que mejora la eficiencia de las convoluciones, haciendo que el modelo sea más adecuado y eficiente para los desafíos específicos presentados en la detección de ocupación de estacionamientos.

- **Hard-Swish**: Implementado para mantener un equilibrio entre la eficiencia computacional y la precisión del modelo, permitiendo una operación más suave y efectiva.

Contribución práctica: Esta versión personalizada de MobileNetV3 Small ha sido diseñada con un enfoque claro en la aplicabilidad práctica y la operación en tiempo real en entornos de estacionamiento reales [46]. Su arquitectura optimizada permite una detección precisa y eficiente de la ocupación, promoviendo una gestión más efectiva de los espacios de estacionamiento.

La arquitectura MobileNetV3 Small personalizada comienza con una capa de convolución estándar seguida de una serie de bloques específicos optimizados mediante NAS. Estos bloques utilizan convoluciones separables en profundidad (BSConv) y convoluciones punto a punto, además de activaciones eficientes como Hard Swish y Leaky ReLU6. También se incluye el módulo CBAM para mejorar la atención en características relevantes. Después de las capas convolucionales y de los bloques Bottleneck, la salida se pasa a través de una capa de MaxPooling, una capa de GlobalAvPool2D y varias capas densas, finalizando con una capa de salida con una sola unidad y una activación sigmoide para la clasificación binaria.

Cuadro 4.15: Arquitectura de MobileNetV3 Small personalizada para clasificación de imágenes

Tipo	**Operación**	**Kernel**	**Salida**	**Act.**	**Obs.**
Conv2D	Convolución	3x3	(112, 112, 16)	Hard Swish	
		Bloques Bottleneck			
		Bloque Bottleneck Tipo A			
		1 repetición			
		Bloque Bottleneck Tipo B			
		4 repeticiones			
		Bloque Bottleneck Tipo C			
		3 repeticiones			
		Bloque Bottleneck Tipo D			
		3 repeticiones			
MaxPooling2D	Max Pooling	7x7	(1, 1, 1024)	-	
GlobalAvPool2D	Pooling Global	-	(1024)	-	
Dense	Densa	-	(512)	ReLU	
Dense	Densa	-	(48)	ReLU	Extr. Carac.
Dense	Densa (Salida)	-	(1)	Sigmoid	Clasif.

Cuadro 4.16: Detalles de los Bloques Bottleneck en MobileNetV3 Small

Bloque	**Operación**	**Kernel**	**Salida**	**Act.**	**Obs.**
Tipo A	Conv2D	1x1	(112, 112, 16)	ReLU	
	BSConv2D	3x3	(112, 112, 16)	ReLU	
	Conv2D	1x1	(112, 112, 24)	Linear	Conexión de Salto
Tipo B	Conv2D	1x1	(56, 56, 72)	ReLU	
	BSConv2D	3x3	(56, 56, 72)	ReLU	
	Conv2D	1x1	(56, 56, 24)	Linear	Conexión de Salto
Tipo C	Conv2D	1x1	(28, 28, 144)	Hard Swish	
	BSConv2D	3x3	(28, 28, 144)	Hard Swish	
	Conv2D	1x1	(28, 28, 48)	Linear	Conexión de Salto
Tipo D	Conv2D	1x1	(14, 14, 288)	Hard Swish	
	BSConv2D	3x3	(14, 14, 288)	Hard Swish	
	Conv2D	1x1	(14, 14, 96)	Linear	Conexión de Salto

4. Investigación y selección de modelos de Series Temporales

4.1. Modelo LSTM

En el contexto de nuestro proyecto, el modelo LSTM se diseñó meticulosamente para capturar la esencia temporal y las relaciones intrincadas presentes en los datos. La arquitectura consta de tres capas LSTM consecutivas, seguidas por capas de dropout después de cada capa LSTM, y finalmente, una capa densa. La Tabla 4.17 muestra las distintas configuraciones probadas.

- **Primera capa LSTM**: Esta capa tiene $50, 100$ o 150 unidades según la configuración de hiperparámetros, multiplicado por un factor de 3. Acepta la forma de los datos de entrada y devuelve secuencias completas a la siguiente capa.
- **Segunda capa LSTM**: Esta capa tiene $50, 100$ o 150 unidades según la configuración de hiperparámetros, multiplicado por un factor de 2, y devuelve secuencias completas a la siguiente capa.
- **Tercera capa LSTM**: Esta capa tiene $50, 100$ o 150 unidades según la configuración de hiperparámetros y solo devuelve el último estado en la secuencia de salida.
- **Capas de Dropout**: Se añaden para prevenir el sobreajuste, con una tasa de dropout de 0,3.
- **Capa de Salida**: Una capa densa con 500 unidades y activación sigmoide para la clasificación.

La arquitectura del modelo LSTM comienza con una primera capa LSTM que recibe los datos de entrada y devuelve secuencias completas a la siguiente capa. La segunda capa LSTM también devuelve secuencias completas, y la tercera capa LSTM solo devuelve el último estado en la secuencia de salida. Entre cada una de estas capas LSTM, se añaden capas de dropout para prevenir el sobreajuste. Finalmente, la salida se pasa a través de una capa densa con 500 unidades y una activación sigmoide para la clasificación final. Esta estructura permite al modelo capturar de manera efectiva las dependencias temporales y complejas en los datos de entrada.

Cuadro 4.17: Configuraciones del Modelo LSTM para clasificación de imágenes

Conf.	**Capa**	**Tipo**	**Unidades**	**Act.**	**Dropout**	**Obs.**
1	LSTM	LSTM	150	-	-	Secuencias completas
	Dropout	Dropout	-	-	0.3	Después de LSTM1
	LSTM	LSTM	100	-	-	Secuencias completas
	Dropout	Dropout	-	-	0.3	Después de LSTM2
	LSTM	LSTM	50	-	-	Último estado
	Dropout	Dropout	-	-	0.3	Después de LSTM3
	Dense	Densa	500	Sigmoid	-	Clasificación
2	LSTM	LSTM	300	-	-	Secuencias completas
	Dropout	Dropout	-	-	0.3	Después de LSTM1
	LSTM	LSTM	200	-	-	Secuencias completas
	Dropout	Dropout	-	-	0.3	Después de LSTM2
	LSTM	LSTM	100	-	-	Último estado
	Dropout	Dropout	-	-	0.3	Después de LSTM3
	Dense	Densa	500	Sigmoid	-	Clasificación
3	LSTM	LSTM	450	-	-	Secuencias completas
	Dropout	Dropout	-	-	0.3	Después de LSTM1
	LSTM	LSTM	300	-	-	Secuencias completas
	Dropout	Dropout	-	-	0.3	Después de LSTM2
	LSTM	LSTM	150	-	-	Último estado
	Dropout	Dropout	-	-	0.3	Después de LSTM3
	Dense	Densa	500	Sigmoid	-	Clasificación

4.2. Modelo GRU

Similar al modelo LSTM, el modelo GRU fue diseñado para capturar las relaciones temporales intrincadas presentes en los datos. Se compone de tres capas GRU consecutivas, seguidas por capas de dropout y una capa densa. La Tabla 4.18 muestra las distintas configuraciones probadas.

- **Primera capa GRU**: Esta capa tiene $50, 100$ o 150 unidades según la configuración de hiperparámetros, multiplicado por un factor de 3. Acepta la forma de los datos de entrada y devuelve secuencias completas a la siguiente capa.
- **Segunda capa GRU**: Esta capa tiene $50, 100$ o 150 unidades según la configuración de hiperparámetros, multiplicado por un factor de 2, y devuelve secuencias completas a la siguiente capa.
- **Tercera capa GRU**: Esta capa tiene $50, 100$ o 150 unidades según la configuración de hiperparámetros y solo devuelve el último estado en la secuencia de salida.
- **Capas de Dropout**: Se añaden para prevenir el sobreajuste, con una tasa de dropout de 0,3.
- **Capa de Salida**: Una capa densa con 500 unidades y activación sigmoide para la clasificación.

La arquitectura del modelo GRU comienza con una primera capa GRU que recibe los datos de entrada y devuelve secuencias completas a la siguiente capa. La segunda capa GRU también devuelve secuencias completas, y la tercera capa GRU solo devuelve el último estado en la secuencia de salida. Entre cada una de estas capas GRU, se añaden capas de dropout para prevenir el sobreajuste. Finalmente, la salida se pasa a través de una capa densa con 500 unidades y una activación sigmoide para la clasificación final. Esta estructura permite al modelo capturar de manera efectiva las dependencias temporales y complejas en los datos de entrada.

4.3. Modelo CNN-1D

El modelo CNN-1D fue diseñado para adaptar los mecanismos de las CNNs, tradicionalmente usadas en visión por computadora, al dominio específico de las series temporales en nuestro contexto. La Tabla 4.19 muestra las distintas configuraciones probadas.

Cuadro 4.18: Configuraciones del Modelo GRU para clasificación de imágenes

Conf.	**Capa**	**Tipo**	**Unidades**	**Act.**	**Dropout**	**Obs.**
1	GRU	GRU	150	-	-	Secuencias completas
	Dropout	Dropout	-	-	0.3	Después de GRU1
	GRU	GRU	100	-	-	Secuencias completas
	Dropout	Dropout	-	-	0.3	Después de GRU2
	GRU	GRU	50	-	-	Último estado
	Dropout	Dropout	-	-	0.3	Después de GRU3
	Dense	Densa	500	Sigmoid	-	Clasificación
2	GRU	GRU	300	-	-	Secuencias completas
	Dropout	Dropout	-	-	0.3	Después de GRU1
	GRU	GRU	200	-	-	Secuencias completas
	Dropout	Dropout	-	-	0.3	Después de GRU2
	GRU	GRU	100	-	-	Último estado
	Dropout	Dropout	-	-	0.3	Después de GRU3
	Dense	Densa	500	Sigmoid	-	Clasificación
3	GRU	GRU	450	-	-	Secuencias completas
	Dropout	Dropout	-	-	0.3	Después de GRU1
	GRU	GRU	300	-	-	Secuencias completas
	Dropout	Dropout	-	-	0.3	Después de GRU2
	GRU	GRU	150	-	-	Último estado
	Dropout	Dropout	-	-	0.3	Después de GRU3
	Dense	Densa	500	Sigmoid	-	Clasificación

- **Capas Convolucionales**: Las capas convolucionales están diseñadas para detectar y aprender patrones temporales locales y estructuras jerárquicas a partir de las secuencias de entrada. Se utilizan tamaños de kernel de 2 y la función de activación es Hard Swish. El número de unidades (que determina el número de filtros) en estas capas es determinado por los hiperparámetros y puede ser $50, 100$ o 150.
- **Capas de Max-Pooling**: A continuación de las capas convolucionales, se incluyeron capas de max-pooling para reducir la dimensionalidad de los mapas de características, conservando las características más críticas y reduciendo el riesgo de sobreajuste.
- **Capa Flatten**: Después de las capas convolucionales y de pooling, los mapas de características se aplanan en un vector único que puede ser alimentado a las capas densas subsiguientes.
- **Capa Densa**: Se añade una capa densa con $50, 100$ o 150 unidades según la configuración de hiperparámetros y con función de activación Hard Swish.
- **Capa de Dropout**: Se inserta después de la capa densa para prevenir el sobreajuste, con una tasa de dropout de 0,3.
- **Capa de Salida**: Una capa densa con 500 unidades y activación sigmoide para la clasificación.

La arquitectura del modelo CNN-1D comienza con dos capas convolucionales diseñadas para detectar y aprender patrones temporales locales en las secuencias de entrada. Estas capas son seguidas por una capa Flatten que aplana los mapas de características en un vector único. Luego, se añade una capa densa con función de activación Hard Swish y una capa de dropout para prevenir el sobreajuste. Finalmente, la salida se pasa a través de una capa densa con 500 unidades y una activación sigmoide para la clasificación final. Esta estructura permite al modelo capturar de manera efectiva las dependencias temporales y complejas en los datos de entrada.

5. Estructura de la Información

Para segmentar las imágenes, se realizó inicialmente una comprensión de los ficheros XML que contienen la información anotada y previamente identificada del paquete de datos PKLot [9]. A continuación, se segmentaron las imágenes,

Cuadro 4.19: Configuraciones del Modelo CNN-1D para clasificación de imágenes

Conf.	Capa	Tipo	Unidades	Act.	Dropout	Obs.
1	Conv1D	Convolución	50	Hard Swish	-	Kernel 2
	Conv1D	Convolución	50	Hard Swish	-	Kernel 2
	Flatten	Aplanado	-	-	-	-
	Dense	Densa	50	Hard Swish	-	-
	Dropout	Dropout	-	-	0.3	Después de Dense
	Dense	Densa	500	Sigmoid	-	Clasificación
2	Conv1D	Convolución	100	Hard Swish	-	Kernel 2
	Conv1D	Convolución	100	Hard Swish	-	Kernel 2
	Flatten	Aplanado	-	-	-	-
	Dense	Densa	100	Hard Swish	-	-
	Dropout	Dropout	-	-	0.3	Después de Dense
	Dense	Densa	500	Sigmoid	-	Clasificación
3	Conv1D	Convolución	150	Hard Swish	-	Kernel 2
	Conv1D	Convolución	150	Hard Swish	-	Kernel 2
	Flatten	Aplanado	-	-	-	-
	Dense	Densa	150	Hard Swish	-	-
	Dropout	Dropout	-	-	0.3	Después de Dense
	Dense	Densa	500	Sigmoid	-	Clasificación

organizándolas en una estructura que permite identificar la cámara, el clima, la fecha y el estado del espacio de estacionamiento. Las nuevas imágenes resultantes adoptaron una estructura de nombres que combina la fecha completa con el identificador del espacio de estacionamiento.

5.1. Estructura de la Información y Segmentación de Imágenes

El proceso de preparación de los datos para el entrenamiento del modelo se realizó en etapas, buscando preservar la coherencia tanto en la representación visual de las imágenes como en la temporalidad de los estados de ocupación.

- **Comprensión de los ficheros XML**: Previo a la segmentación de las imágenes, se procesaron los ficheros XML incluidos en el paquete de datos PKLot. Estos XML proporcionan anotaciones detalladas que especifican la ubicación y el estado de cada espacio de estacionamiento en las imágenes correspondientes.
- **Segmentación de Imágenes**: Utilizando la información extraída de los

XML, se procesaron las imágenes originales para segmentar individualmente cada espacio de estacionamiento. Esto resultó en imágenes más pequeñas y específicas que representan los estados individuales de los espacios.

- **Organización y Nombrado**: Las imágenes segmentadas se organizaron en una estructura jerárquica de directorios y se renombraron para reflejar la cámara de origen, las condiciones climáticas, la fecha y el estado específico del espacio de estacionamiento.
- **Consideraciones Adicionales**: Durante este proceso, además de las carpetas para los estados "ocupado" y "vacío", se creó una carpeta para errores con el objetivo de gestionar y aislar imágenes que no se procesaran correctamente. Estos casos especiales se verificaron manualmente, garantizando así la integridad de la información al convertir posteriormente las imágenes en una matriz temporal de estados.

5.2. Construcción del DataFrame

Una vez segmentadas y organizadas las imágenes, el siguiente paso fue transformar esta información visual en una estructura de datos tabular para su posterior análisis y alimentación a los modelos. Este proceso se describe a continuación:

- **Extracción de información de directorios**: Se diseñó una función que, dada la ruta de una imagen segmentada, extrae información crucial como el estado del espacio de estacionamiento, el identificador del espacio y la fecha y hora de la imagen.
- **Construcción del DataFrame**: Se aplicó la función anterior a todas las rutas de imágenes segmentadas, extrayendo la información pertinente y almacenándola en un DataFrame. En este DataFrame, cada fila representa una marca temporal única, y las columnas reflejan el estado y la ruta de la imagen de cada espacio de estacionamiento.
- **Verificación de valores faltantes**: Tras la construcción inicial, se verificaron los valores NaN (o faltantes) en el DataFrame para asegurar la integridad de los datos.
- **Detección de duplicados**: Se implementaron funciones específicas para detectar tanto imágenes duplicadas (basadas en su hash SHA256) como nombres de archivos duplicados, garantizando la unicidad y evitando redundancias.

- **Ordenamiento temporal**: El DataFrame se ordenó según las marcas temporales para garantizar una secuencia cronológica.

- **Adición de columna temporal**: Se introdujo una columna al DataFrame para reflejar la diferencia temporal entre registros consecutivos, facilitando la comprensión de la temporalidad y periodicidad de los datos así como la discontinuidad temporal para los modelos de series temporales investigados.

- **Adición de columna de path**: Se introdujo una columna para tener la referencia de los ficheros que corresponden a los estados temporales segmentados por cada espacio de estacionamiento para poder posteriormente generar las características con los modelos CNNs entrenados y hacer diferentes pruebas

- **Conversión de rutas de archivos**: Considerando que el proyecto se desarrolló en ambientes locales y en Google Colab, se desarrollo una función para poder transformar las rutas según el contexto.

- **Guardado del DataFrame para Colab**: El DataFrame, con las rutas ya convertidas, se guardó en diversos formatos (CSV, JSON y Pickle) para su utilización posterior en Colab.

La estructura del DataFrame resultante D se puede representar como una matriz de 4474 filas (registros temporales) a los que se les quito 3 filas para que en la fase de inferencia temporal el ultimo registro tenga información futura hasta 60 minutos desde la perspectiva de su presente y 101 columnas (una para la diferencia temporal y 100 para las rutas de las imágenes de los espacios de estacionamiento). La matriz D se define como:

$$D = \begin{bmatrix} t_1 & p_{1,1} & p_{1,2} & \cdots & p_{1,100} \\ t_2 & p_{2,1} & p_{2,2} & \cdots & p_{2,100} \\ \vdots & \vdots & \vdots & & \vdots \\ t_{4471} & p_{4471,1} & p_{4471,2} & \cdots & p_{4471,100} \end{bmatrix}$$

Donde:
- t_i representa la diferencia temporal respecto al registro anterior, siendo cero para el primer registro.
- $p_{i,j}$ representa el estado del espacio de estacionamiento j en el tiempo i, y contiene la ruta de la imagen correspondiente.

6. Construcción de modelos e integración de características

6.1. Construcción y entrenamiento de CNNs

- **Procesamiento de datos**: Se estableció una función para extraer rutas de imágenes y etiquetas desde el DataFrame. Posteriormente, se extrajeron las imágenes manteniendo la estructura previamente creada.
- **División de datos**: Una vez que los datos estuvieron disponibles en el entorno de trabajo, se cargó el DataFrame previamente procesado y se dividió en conjuntos de entrenamiento (80 %), validación (10 %) y prueba (10 %) usando un enfoque de división temporal. Esta división garantiza que cada conjunto tenga datos suficientes para el entrenamiento de las CNNs y a su vez la extracción de características simule el desconocimiento de las características de las imágenes que al momento de ser evaluadas no existirían como características para la parte temporal del proyecto. Por este motivo de 4474 filas el DataFrame se redujo a 4471 ya que el registro sigueintes no tendrian la inferencia para las predicciones temporales que requiere el proyecto.
- **Construcción de Modelos CNN**: Se definieron múltiples funciones para establecer las arquitecturas mLeNet, mAlexNet, ResNet, VGG19, InceptionV3, MobileNetV2, MobileNetV3 y MobileNetV3 Small según el diseño previamente comentado. Estas arquitecturas fueron diseñadas de forma tal que permitieran el entrenamiento para la extracción de características y adquiriesen la capacidad de detección de espacios libres y ocupados.
- **Entrenamiento y Evaluación de Modelos**: Para el entrenamiento, se utilizó el optimizador SGD con una tasa de aprendizaje de 0.001 que decae exponencialmente con momentum en 0.9 y Nesterov activo. Además, se agregaron métricas adicionales, como Precision, Recall, Specificity, y F1 para evaluar el rendimiento del modelo durante el entrenamiento.
- **Extracción de características**: Se define una función posterior, que toma una imagen y devuelve sus características después de procesarla a través de los modelos de forma iterativa. Las características se guardan en lotes para evitar sobrecargas de memoria y para permitir la reanudación en caso de interrupciones.
- **Consideraciones Adicionales**: El proceso de entrenamiento también incluyó la implementación de callbacks para guardar el modelo con la

mejor precisión de validación y para guardar el historial de entrenamiento para análisis posterior realizados con 5 épocas ya que los modelos con mas épocas mostraban sobreajuste.

6.2. Integración de Características

El diseño que se estableció en las CNNs permite la extracción de características en la capa previa a la final. Dado que se desea modelar la evolución de los espacios de estacionamiento a lo largo del tiempo, es esencial tener una representación que capture de manera efectiva el estado de cada espacio en un momento dado.

- **Combinación de características**: Las características extraídas de las CNNs se combinaron con las series temporales del conjunto de datos PKLot. Para cada slot y en cada instante temporal, se añadió un vector de características correspondiente a la imagen del slot en ese momento. Este proceso resultó en un DataFrame donde cada fila representa un instante temporal y cada columna un slot. Además de la etiqueta de ocupación, y el valor de la distancia temporal entre registros.
- **Relevancia de la combinación**: La integración de estas características visuales con las series temporales tiene el potencial de mejorar ya que puede ayudar a identificar patrones más sutiles relacionados con la ocupación de los espacios.

El DataFrame final D'' se representa entonces como una matriz de 4471 filas (registros temporales) y 4901 columnas (una para la diferencia temporal y 100 para los espacios de estacionamiento con 1 características de estado y 48 características extraídas del modelo CNN correspondiente). La matriz D'' se define como:

$$D'' = \begin{bmatrix} t_1 & e_{1,1} & c_{1,1,1} & \cdots & c_{1,1,48} & \cdots & e_{1,100} & c_{1,100,1} & \cdots & c_{1,100,48} \\ t_2 & e_{2,1} & c_{2,1,1} & \cdots & c_{2,1,48} & \cdots & e_{2,100} & c_{2,100,1} & \cdots & c_{2,100,48} \\ \vdots & \vdots & \vdots & & \vdots & & \vdots & \vdots & & \vdots \\ t_{4471} & e_{4471,1} & c_{4471,1,1} & \cdots & c_{4471,1,48} & \cdots & e_{4471,100} & c_{4471,100,1} & \cdots & c_{4471,100,48} \end{bmatrix}$$

En este DataFrame final, cada fila representa un registro temporal y las columnas contienen: - t_i: la diferencia temporal - $e_{i,j}$: el estado del espacio de

estacionamiento j en el tiempo i (0 para vacío, 1 para ocupado). - $c_{i,j,k}$: las características k extraídas de las imágenes mediante CNN para el espacio de estacionamiento j en el tiempo i.

6.3. Construcción y entrenamiento de RNNs

El entrenamiento de RNNs se llevó a cabo utilizando el conjunto de datos enriquecido con las características visuales de las CNNs en diferentes pasos probando cual de los modelos enriquesen mejor la capacidad de los modelos de series temporales en su predicción futura.

- **Preparación de los datos**: Antes del entrenamiento, los datos estaban divididos en conjuntos de entrenamiento, validación y prueba. Además, se aplicó una transformación logarítmica a la columna de diferencias temporales entre registros para normalizar su distribución.
- **Estructura de los modelos RNNs**: Los modelos utilizados son los comentados anteriormente
- **Exploración de Hiperparámetros**: Con estos modelos se llevó a cabo una búsqueda exhaustiva de hiperparámetros, explorando combinaciones específicas para identificar la configuración óptima. Los hiperparámetros considerados fueron:
  - Tipo de arquitectura RNN: LSTM, GRU, CNN-1D.
  - Número de unidades en las capas: 50, 100, 150 unidades o filtros según la interpretación de la arquitectura.
  - Tasa de dropout: para la regularización, se considero tasas de 0.3.
  - Funciones de activación: Hard Swish en CNN-1D.
  - Número de épocas: se experimentó con diferentes duraciones de entrenamiento, como 20, 40 y 60 épocas.

  Posteriormente en el Capitulo de Resultados experimentales se explicara los motivos de estas decisiones.
- **Técnica de Optimización**: La técnica utilizada para la selección de hiperparámetros fue la búsqueda exhaustiva o "grid search". Esta técnica consiste en evaluar sistemáticamente todas las combinaciones posibles de hiperparámetros para determinar cuál ofrece el mejor rendimiento en el conjunto de validación.

- **Función de pérdida y optimización**: Se utilizó el error cuadrático medio como función de pérdida, que mide la diferencia entre las predicciones del modelo y las verdaderas etiquetas de ocupación. El optimizador Adam fue empleado para ajustar los pesos de la red durante el entrenamiento reduciéndolo a 0,00005 ya que en valores superiores los modelos LSTM y GRU encontraban fuerte inestabilidad en sus predicciones.

- **Métricas y monitoreo**: Durante el entrenamiento, se monitorizaron varias métricas, incluyendo el RMSE, MAE, R2 y la Correlación de Pearson. Además, se implementó una métrica personalizada para calcular la precisión binaria en diferentes intervalos de tiempo. Los registros de entrenamiento se almacenaron utilizando TensorBoard, permitiendo una visualización detallada del proceso de entrenamiento y la evolución de al capacidad de detección de los modelos.

El DataFrame final D'' se le normalizo la columna temporal quedando:

$$D'' = \begin{bmatrix} \log(t_1) & e_{1,1} & c_{1,1,1} & \cdots & c_{1,1,48} & \cdots & e_{1,100} & c_{1,100,1} & \cdots & c_{1,100,48} \\ \log(t_2) & e_{2,1} & c_{2,1,1} & \cdots & c_{2,1,48} & \cdots & e_{2,100} & c_{2,100,1} & \cdots & c_{2,100,48} \\ \vdots & \vdots & \vdots & & \vdots & & \vdots & \vdots & & \vdots \\ \log(t_{4471}) & e_{4471,1} & c_{4471,1,1} & \cdots & c_{4471,1,48} & \cdots & e_{4471,100} & c_{4471,100,1} & \cdots & c_{4471,100,48} \end{bmatrix}$$

En este DataFrame final, cada fila representa un registro temporal y las columnas contienen: - $\log(t_i)$: la diferencia temporal normalizada con logaritmo. - $e_{i,j}$: el estado del espacio de estacionamiento j en el tiempo i (0 para vacío, 1 para ocupado). - $c_{i,j,k}$: las características k extraídas de las imágenes mediante CNN para el espacio de estacionamiento j en el tiempo i.

Por ultimo para la evaluación de las RNNs los datos de salida Y para las predicciones temporales se diseñaron contemplando los 4471 registros temporales en las filas con 100 columnas que tuvieran 5 registros cada una con las predicciones temporales. De esta forma los 5 valores representan el estadio ocupado o vació futuro de 5, 15, 30, 45 y 60 minutos resultando la siguiente estructura:

$$Y = \begin{bmatrix} y_{1,1,f} & y_{1,2,f} & \cdots & y_{1,100,f} \\ y_{2,1,f} & y_{2,2,f} & \cdots & y_{2,100,f} \\ \vdots & \vdots & & \vdots \\ y_{4471,1,f} & y_{4471,2,f} & \cdots & y_{4471,100,f} \end{bmatrix}$$

Donde: - $y_{t,j,f}$ representa el estado futuro (0 para vacío, 1 para ocupado) del espacio de estacionamiento j en el tiempo t. - f es un vector que contiene los estados en los intervalos de tiempo: 5 minutos, 15 minutos, 30 minutos, 45 minutos y 60 minutos.

Esta estructura permite que los modelos RNN predigan el estado futuro de cada espacio de estacionamiento en estos intervalos de tiempo específicos, basándose en la información histórica y las características extraídas.

Capítulo 5

Resultados experimentales

1. Evaluación de Modelos CNNs

En este análisis, se evalúan diferentes métricas para comprender y comparar el rendimiento de varios modelos. Las métricas consideradas incluyen:

- **Pérdida (Loss)**: Representa el error entre las predicciones del modelo y los valores reales. Utilizamos la pérdida de entropía cruzada binaria, que se calcula usando la fórmula:

$$L(y, \hat{y}) = -\frac{1}{N}\sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- **Exactitud (Accuracy)**: Es el ratio de predicciones correctas respecto al total de predicciones.

$$\text{Accuracy} = \frac{\text{Número de predicciones correctas}}{\text{Número total de predicciones}}$$

- **Precisión (Precision)**: Es el ratio de verdaderos positivos respecto al total de casos clasificados como positivos.

$$\text{Precision} = \frac{\text{Verdaderos Positivos}}{\text{Verdaderos Positivos} + \text{Falsos Positivos}}$$

- **Sensibilidad (Recall)**: Es el ratio de verdaderos positivos respecto al total de casos que son realmente positivos.

$$\text{Recall} = \frac{\text{Verdaderos Positivos}}{\text{Verdaderos Positivos} + \text{Falsos Negativos}}$$

- **Especificidad**: Es el ratio de verdaderos negativos respecto al total de casos que son realmente negativos.

$$\text{Especificidad} = \frac{\text{Verdaderos Negativos}}{\text{Verdaderos Negativos} + \text{Falsos Positivos}}$$

- **Puntuación F1 (F1 Score)**: Es una métrica que combina precisión y recall utilizando la media armónica.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

1.1. Resultados de entrenamiento de CNNs

Los resultados de las pruebas realizadas se pueden condensar en la siguiente Tabla 5.1 luego de iterar 5 epocas con parada por mejor detección en validación:

Modelo	**Pérdida**	**Accuracy**	**Prec.**	**Recall**	**Espec.**	**F1**
mLeNet	0.0150	0.9951	0.9936	0.9966	0.9936	0.9951
mAlexNet	0.0143	0.9956	0.9951	0.9966	0.9947	0.9959
VGG19	0.0133	0.9955	0.9941	0.9969	0.9941	0.9955
ResNet	0.0014	0.9997	0.9997	1.0000	0.9994	0.9999
InceptionV3	0.0013	0.9996	0.9996	1.0000	0.9992	0.9998
MobileNetV2	0.0016	0.9996	0.9996	0.9996	0.9996	0.9996
MobileNetV3	0.0062	0.9980	0.9975	0.9985	0.9975	0.9980
MobileNetV3Small	0.0027	0.9997	1.0000	0.9996	1.0000	0.9997

Cuadro 5.1: Métricas de validación para diferentes modelos

mLeNet

En la Figura 5.1 se observa el entrenamiento de mLeNet, el cual presenta una exactitud del 99.51 %. Con una pérdida de 0.0150 y un F1 Score de 0.9951, muestra un balance entre precisión y recall, siendo una opción confiable para tareas de clasificación.

mAlexNet

En la Figura 5.2 se observa el entrenamiento de mAlexNet, el cual supera ligeramente a mLeNet con una exactitud del 99.56 %. Su baja pérdida y alto

F1 Score de 0.9959 sugieren un rendimiento sólido y consistente en el conjunto de datos.

VGG19

En la Figura 5.3 se observa el entrenamiento de VGG19, conocido por su profundidad, tiene una exactitud del 99.55 %. A pesar de su complejidad, sigue siendo eficaz en la clasificación con un F1 Score de 0.9955.

ResNet

En la Figura 5.4 se observa el entrenamiento de ResNet, destaca con una asombrosa exactitud del 99.97 %. Su diseño de conexiones residuales le permite un aprendizaje profundo efectivo, reflejado en su F1 Score de 0.9999.

InceptionV3

En la Figura 5.5 se observa el entrenamiento de InceptionV3, con su estructura innovadora, logra una exactitud del 99.96 %. Su F1 Score de 0.9998 demuestra su capacidad para balancear precisión y recall.

MobileNetV2

En la Figura 5.6 se observa el entrenamiento de MobileNetV2, diseñado para dispositivos livianos, muestra una exactitud del 99.96 %. Su rendimiento, comparable a modelos más grandes, lo hace ideal para aplicaciones en tiempo real como la pretendida en nuestro proyecto.

MobileNetV3

En la Figura 5.7 se observa el entrenamiento de MobileNetV3 alcanza una exactitud del 99.80 %. Aunque es una mejora respecto a V2, su F1 Score de 0.9980 sugiere un ligero desequilibrio entre precisión y recall.

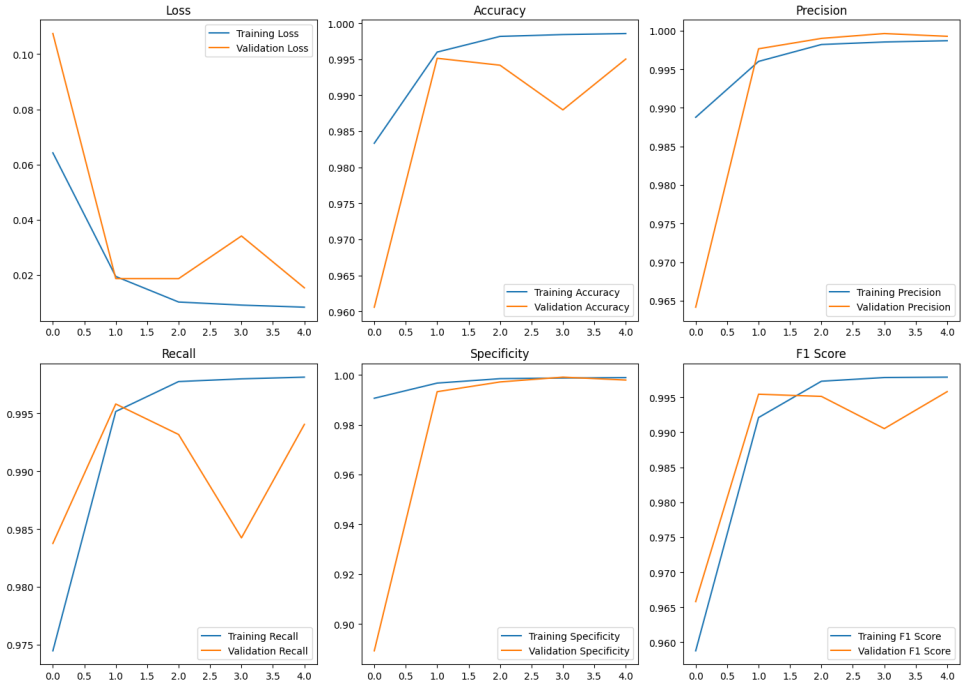

Figura 5.1: Metricas de entrenamiento de mLeNet

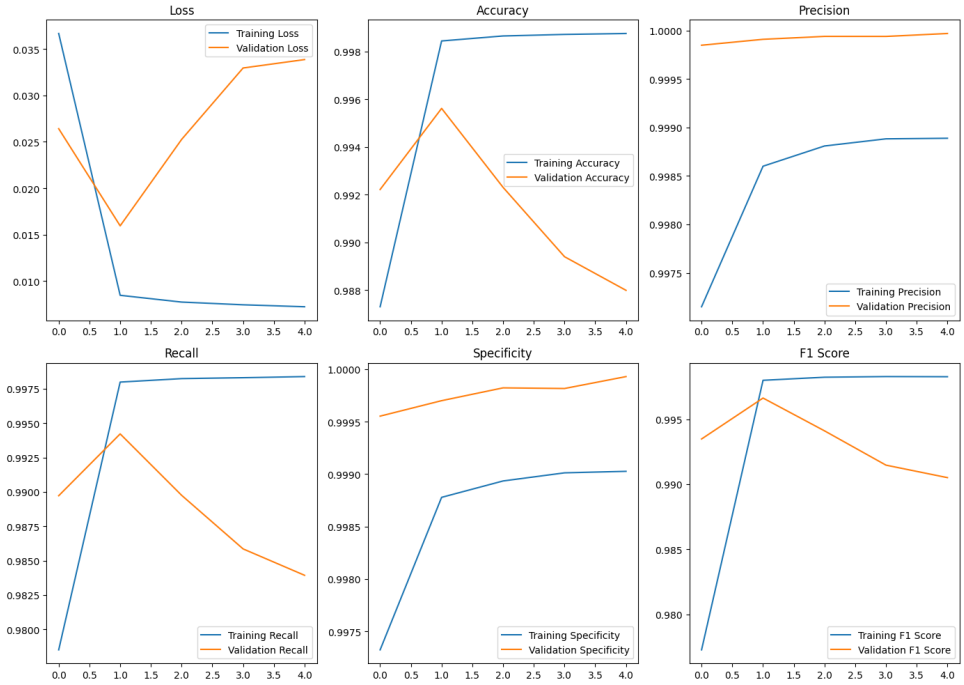

Figura 5.2: Metricas de entrenamiento de mAlexNet

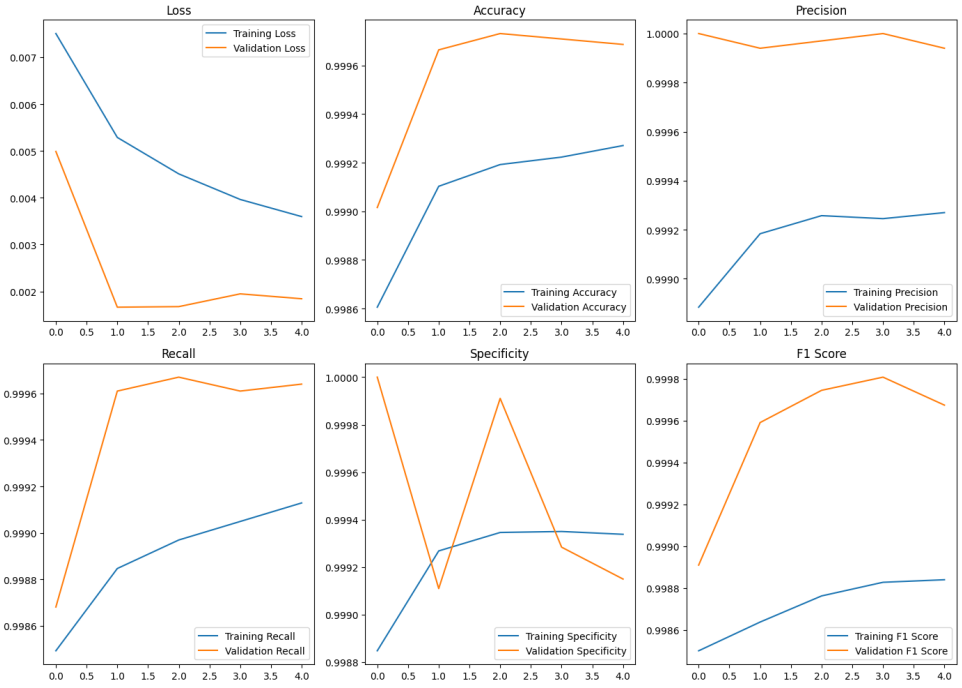

Figura 5.3: Metricas de entrenamiento de VGG19

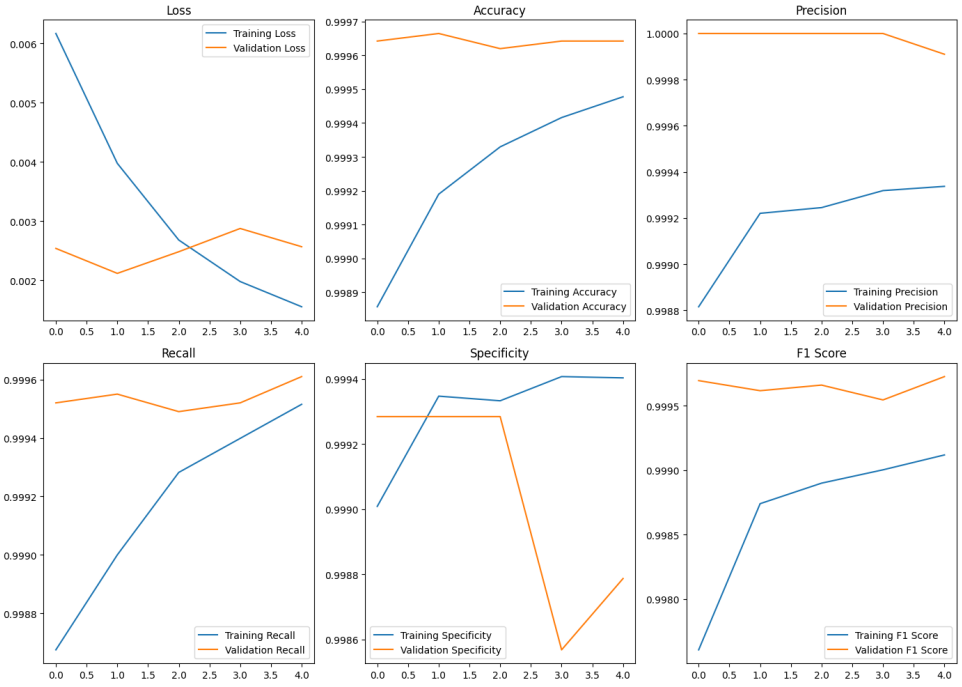

Figura 5.4: Metricas de entrenamiento de ResNet

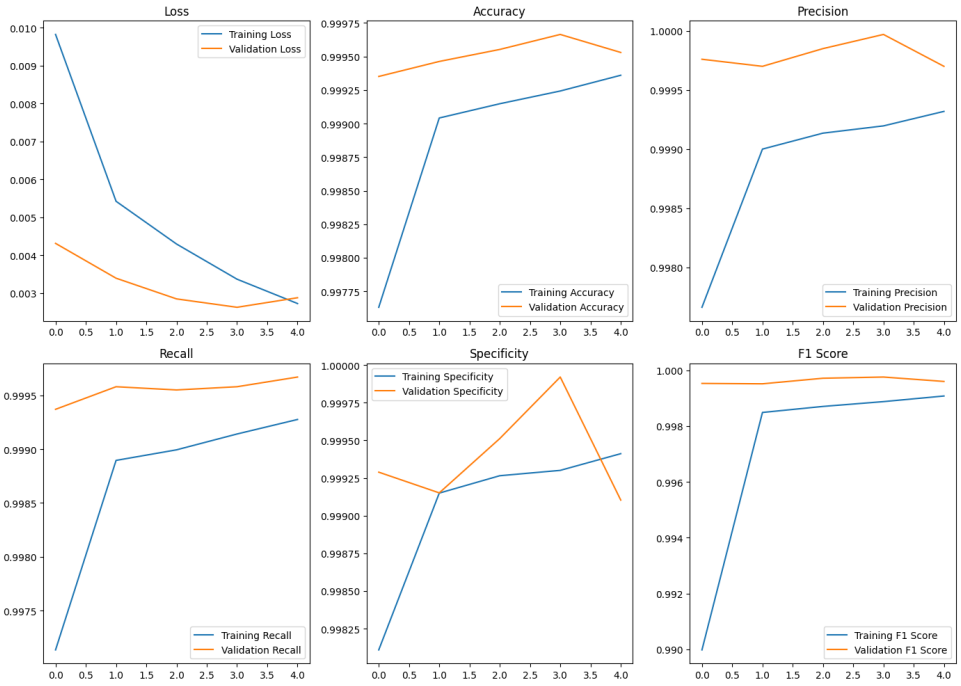

Figura 5.5: Metricas de entrenamiento de InceptionV3

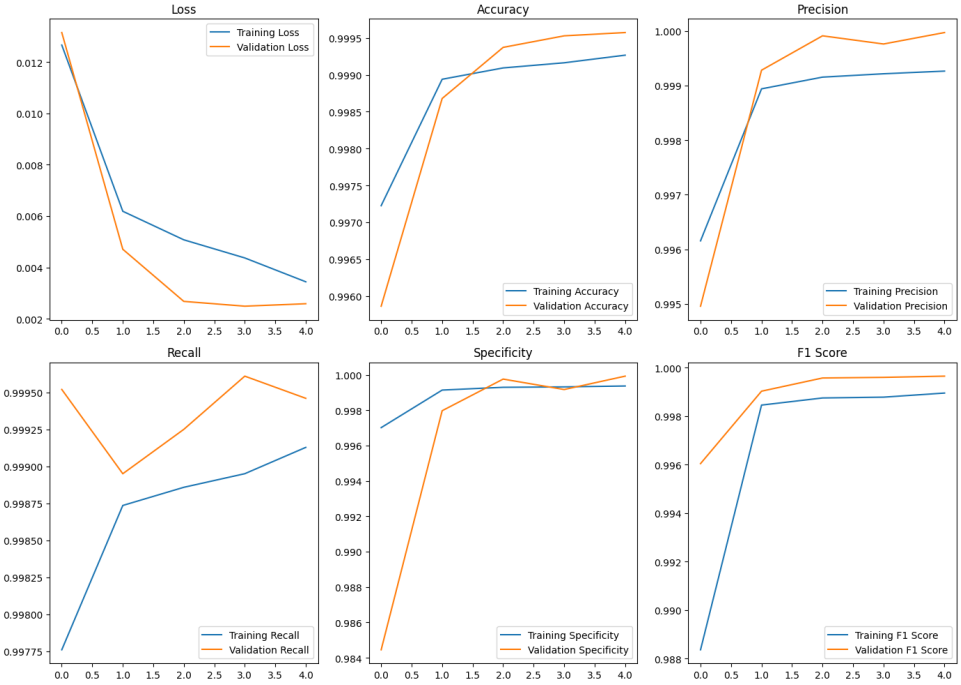

Figura 5.6: Metricas de entrenamiento de MobileNetV2

MobileNetV3Small

En la Figura 5.8 se observa el entrenamiento de MobileNetV3 Small, a pesar de ser una versión reducida, MobileNetV3Small logra una exactitud del 99.97 %, evidenciando que el rediseño compacto de este modelo es altamente efectivo, siendo ideal para nuestro proyecto.

Por otro lado, también consideramos el tiempo de entrenamiento, la cantidad de parámetros y el tamaño de los modelos en la Tabla 5.2 ya que estos podrían ser integrados de forma practica en cámaras inteligentes.

Modelo	Parámetros	Tiempo por época
mLeNet	51,498,657	3.82 min
mAlexNet	12,086,699	4.17 min
VGG19	74,054,305	29.01 min
ResNet	231,705,185	23.12 min
InceptionV3	129,275,649	14.23 min
MobileNetV2	2,904,417	8.52 min
MobileNetV3	4,664,209	8.25 min
MobileNetV3Small	2,562,669	12,47 min

Cuadro 5.2: Cantidad de parámetros de los modelos analizados y su velocidad de entrenamiento bajo Google Colab en entornos Python V100

Conclusión

Los modelos analizados muestran un rendimiento excepcional en el conjunto de datos. ResNet e InceptionV3 destacan por su alta exactitud, pero MobileNetV2 y V3Small demuestran que se puede lograr un rendimiento comparable con estructuras más ligeras. Al seleccionar un modelo para despliegue además de considerar el equilibrio entre precisión, tamaño del modelo y velocidad de inferencia debemos analizar como impacta las características extraídas en los modelos de series temporales.

2. Evaluación de Modelos de Series Temporales

En el capítulo previo, detallamos las arquitecturas de modelos de series temporales seleccionadas para nuestro estudio. La Tabla 5.3 resume los 27 modelos

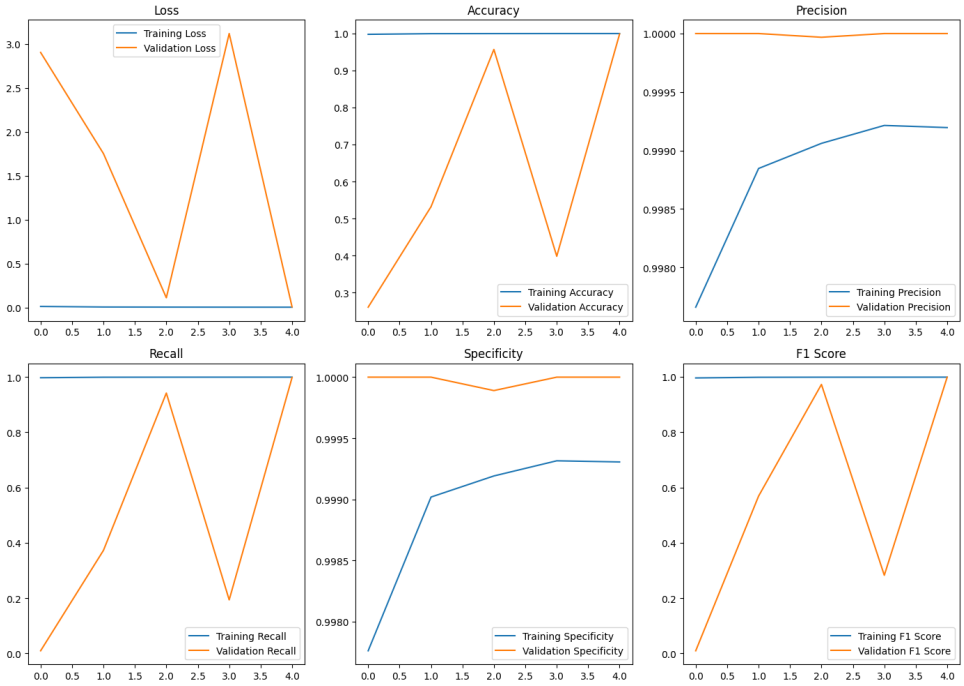

Figura 5.7: Metricas de entrenamiento de MobileNetV3

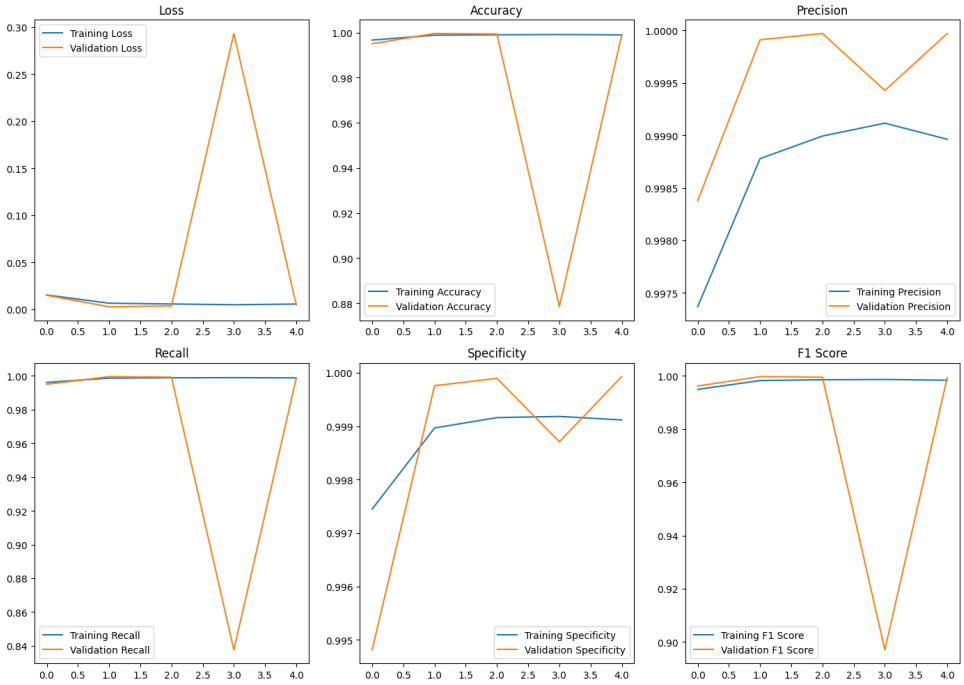

Figura 5.8: Metricas de entrenamiento de MobileNetV3Small

evaluados, cada uno con características distintivas derivadas de diferentes modelos de redes convolucionales (CNN). Los modelos se diseñaron con 50, 100 y 150 unidades, aplicando un factor de Dropout de 0.3 (valor que optimizamos a través de ensayos preliminares). Se optó por la función de activación Hard Swish para las CNN-1D. Las diferentes duraciones de entrenamiento, fijadas en 20, 40 y 60 épocas, se establecieron bajo el criterio de que los modelos tienden a aprender en las primeras 20 épocas, se ajustan apropiadamente en 40 y tienden al sobreajuste después de 60 épocas. Esto nos permitió estandarizar la fase de entrenamiento y asegurar una comparación justa entre las arquitecturas.

Letra	Modelo	Unidades	Dropout	Activación	Épocas
A	CNN-1D	100	0.3	Hard Swish	20
B	CNN-1D	100	0.3	Hard Swish	40
C	CNN-1D	100	0.3	Hard Swish	60
D	CNN-1D	150	0.3	Hard Swish	20
E	CNN-1D	150	0.3	Hard Swish	40
F	CNN-1D	150	0.3	Hard Swish	60
G	CNN-1D	50	0.3	Hard Swish	20
H	CNN-1D	50	0.3	Hard Swish	40
I	CNN-1D	50	0.3	Hard Swish	60
J	GRU	100	0.3	-	20
K	GRU	100	0.3	-	40
L	GRU	100	0.3	-	60
M	GRU	150	0.3	-	20
N	GRU	150	0.3	-	40
Ñ	GRU	150	0.3	-	60
O	GRU	50	0.3	-	20
P	GRU	50	0.3	-	40
Q	GRU	50	0.3	-	60
R	LSTM	100	0.3	-	20
S	LSTM	100	0.3	-	40
T	LSTM	100	0.3	-	60
U	LSTM	150	0.3	-	20
V	LSTM	150	0.3	-	40
W	LSTM	150	0.3	-	60
X	LSTM	50	0.3	-	20
Y	LSTM	50	0.3	-	40
Z	LSTM	50	0.3	-	60

Cuadro 5.3: Descripción de los modelos de series temporales utilizados en el estudio

2.1. Resultados de los Modelos de series temporales

Modelos LSTM, GRU y CNN-1D con caracteristicas mLeNet

Con las características extraídas de mLeNet los resultados de validación de los modelos RNNs indican que los modelos B, C y E sobresalen por su precisión promedio superior, indicando una robustez general en la predicción. El Modelo B, en particular, muestra un desempeño destacado con la mejor precisión promedio de 0.889 como se observa en la Figura 5.17, lo que lo convierte en el candidato más sólido para predicciones consistentes a través de todos los horizontes de tiempo.

El Modelo H, aunque no es el mejor, lidera específicamente en predicciones a corto plazo con una precisión a los 5 minutos de 0.922 como se observa en la Figura 5.9.

En la ventana de 15 minutos, los modelos B y C mantienen el mejor rendimiento, con el Modelo B como el más destacado como se observa en la Figura 5.11 y 5.12. Esta tendencia se mantiene en los modelos con una precisión aceptable en los rangos de tiempo restantes.

Considerando que los valores de RMSE y MAE de todos los modelos se encuentran en rangos estrechos de 0.342 a 0.356 y 0.183 a 0.245 respectivamente como se observa en la Figura 5.9, y que los coeficientes R2 están también muy cercanos entre sí, estos no ofrecen diferencias significativas que justifiquen ser el centro del análisis comparativo.

Para mas detalle se puede observar en el Capitulo Anexos la Tabla 7.1

En el análisis del conjunto de prueba, los modelos D, F, C, Ñ y N presentan un rendimiento destacado. El modelo D sobresale con la predicción más precisa en el intervalo de tiempo de 30 minutos y comparte las mejores predicciones a 45 minutos con otros modelos líderes como se observa en la Figura 5.15 y 5.16. Los modelos F y C también exhiben fuertes resultados, siendo C el que tiene el MAE más bajo, señal de una precisión consistente. Además, el modelo Ñ logra el mejor R2, indicando una varianza explicada superior en sus predicciones.

Los modelos mantienen un RMSE entre 0.318 y 0.343 y un MAE entre 0.163 y 0.194, valores bastante ajustados que indican una calidad predictiva similar como se observa en la Figura 5.13. En cuanto a la capacidad de predicción promedio en distintos rangos de tiempo, los modelos oscilan mayormente entre 0.905 y 0.929 como se observa en la Figura 5.14, con el modelo D liderando con un promedio de 0.929, lo que sugiere una habilidad robusta para predecir en

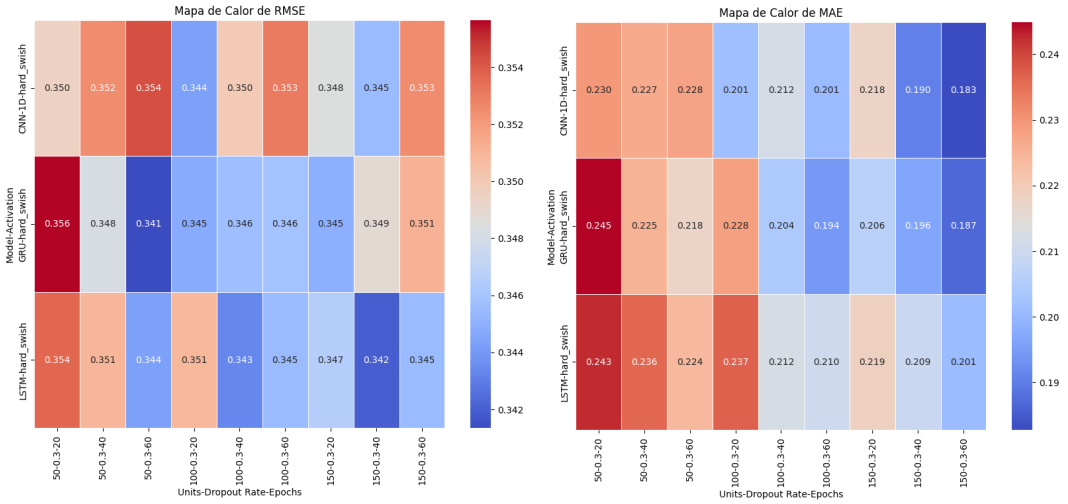

Figura 5.9: Métricas RMSE y MAE de RNNs en Validación con mLeNet

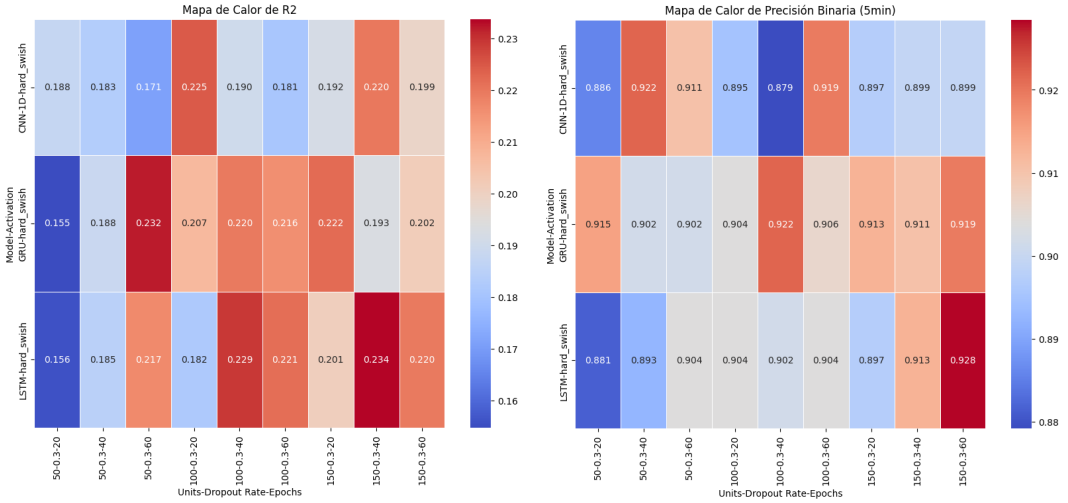

Figura 5.10: Métricas R2 y Precisión 5 min de RNNs en Validación con mLeNet

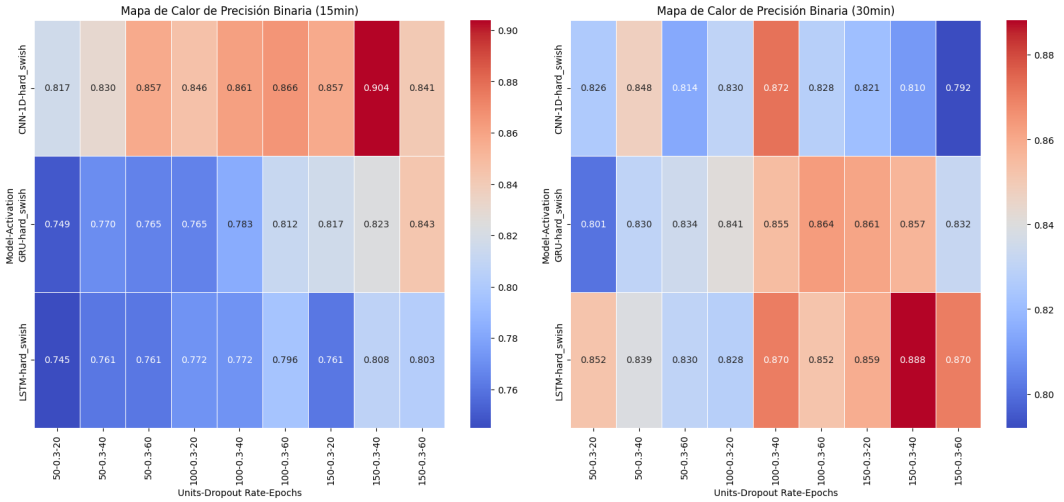

Figura 5.11: Métricas Precisión 15 min y Precisión 30 min de RNNs en Validación con mLeNet

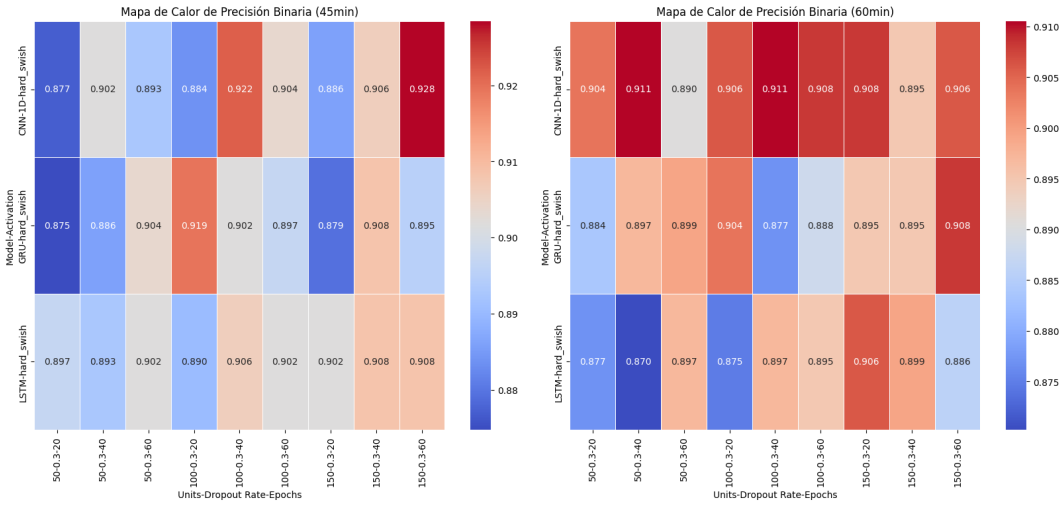

Figura 5.12: Métricas Precisión 45 min y Precisión 60 min de RNNs en Validación con mLeNet

distintos horizontes temporales.

En resumen, aunque hay variaciones menores en las métricas de RMSE, MAE y R2 entre los modelos más destacados, las diferencias en el rendimiento predictivo son relativamente pequeñas, lo que sugiere que varios modelos tienen la capacidad de realizar predicciones precisas y podrían ser adecuados para implementar según el contexto practico.

Para mas detalle se puede observar en el Capitulo Anexo la Tabla 7.2

Analizando los resultados de validación y prueba con características de mLeNet, el modelo C destaca por su consistencia en ambos conjuntos, mostrando altas tasas de detección y precisión promedio. Este modelo ofrece un equilibrio entre precisión a corto y largo plazo, lo cual es crucial para su aplicación en escenarios reales. En la validación, sus predicciones a 15 y 45 minutos son sobresalientes, mientras que en la prueba, mantiene una alta precisión en el intervalo de 30 minutos como se observa en la Figura 5.17.

Dada su robustez y capacidad de generalización, el modelo C se presenta como una opción sólida para implementaciones futuras.

Modelos LSTM, GRU y CNN-1D con caracteristicas mAlexNet

Para el conjunto de validación que utiliza características de mAlexNet, los modelos F y B destacan por su alta precisión promedio. En particular, el modelo F lidera con un RMSE de 0.321, un MAE de 0.158 y un R2 de 0.311 como se observa en la Figura 5.18 y 5.19, sugiriendo que posee una capacidad predictiva superior en términos de precisión del error y ajuste del modelo. En términos de precisión temporal, el modelo B muestra la mejor performance a los 15 minutos con una precisión de 0.902, mientras que el modelo F demuestra ser más equilibrado con una sólida precisión en las predicciones a 30 minutos y 45 minutos, con valores de 0.919 y 0.928 respectivamente como se observa en la Figura 5.20 y 5.21. Estos resultados resaltan la competencia de ambos modelos en diferentes horizontes temporales, con el modelo F como el más estable en pronósticos a mediano plazo y el modelo B como una opción robusta para predicciones inmediatas.

Para mas detalle se puede observar en el Capitulo Anexo la Tabla 7.3

En el conjunto de prueba con características de mAlexNet, el modelo $\tilde{\text{N}}$ sobresale con la mejor precisión promedio de 0.946 y exhibe excelentes resultados en todas las métricas, incluido el R2 más alto de 0.513, indicando una fuerte correlación entre las predicciones y los valores reales como se observa en la Fi-

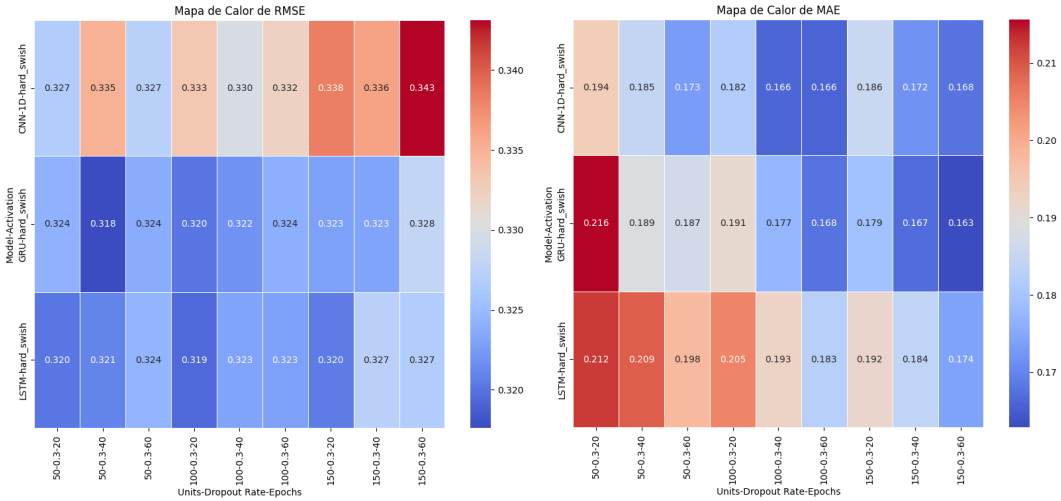

Figura 5.13: Métricas RMSE y MAE de RNNs en Prueba con mLeNet

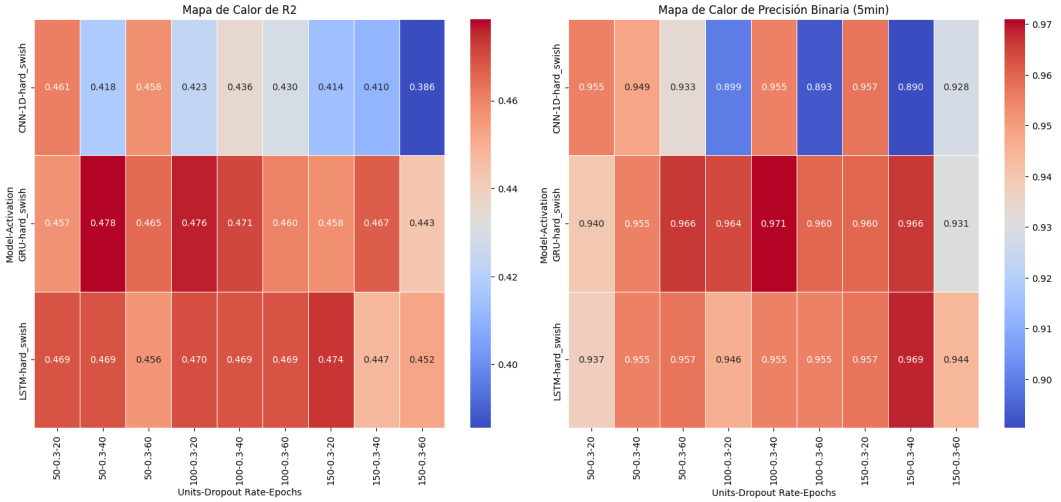

Figura 5.14: Métricas R2 y Precisión 5 min de RNNs en Prueba con mLeNet

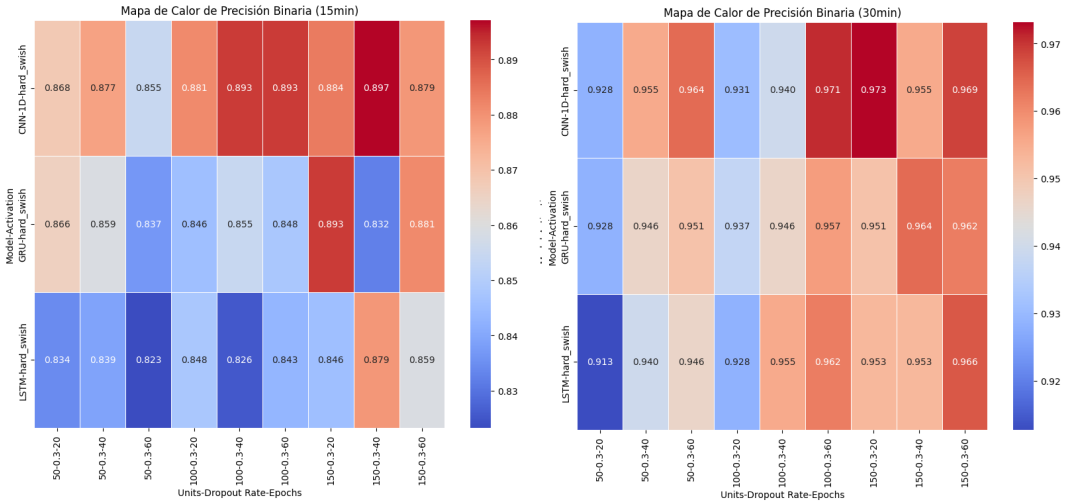

Figura 5.15: Métricas Precisión 15 min y Precisión 30 min de RNNs en Prueba con mLeNet

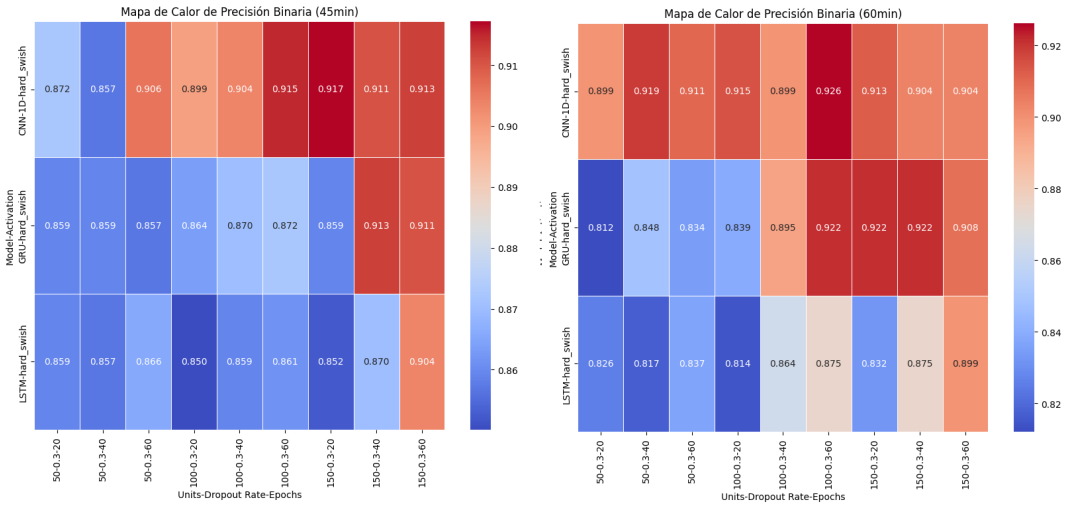

Figura 5.16: Métricas Precisión 45 min y Precisión 60 min de RNNs en Prueba con mLeNet

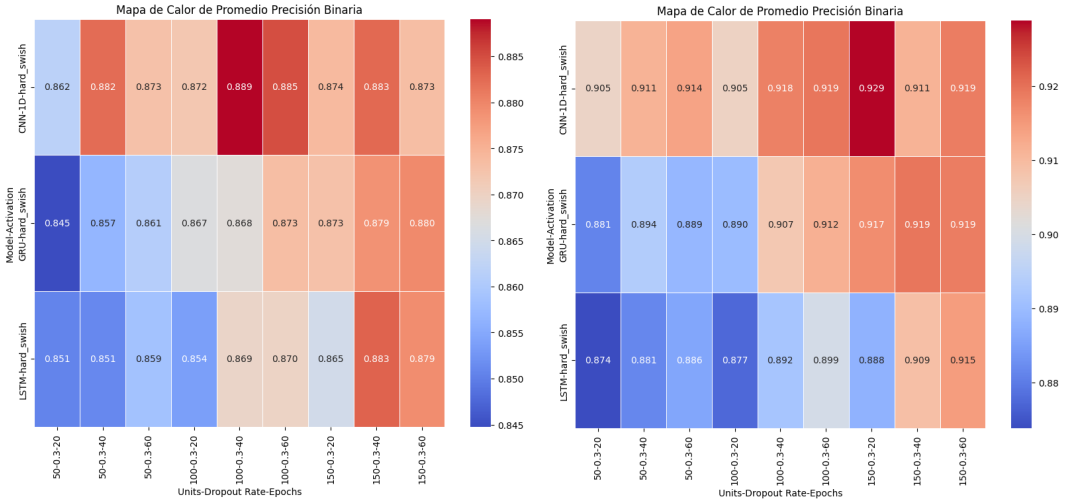

Figura 5.17: Métricas Precisión Promedio (Validación izquierda, Prueba Derecha) con mLeNet

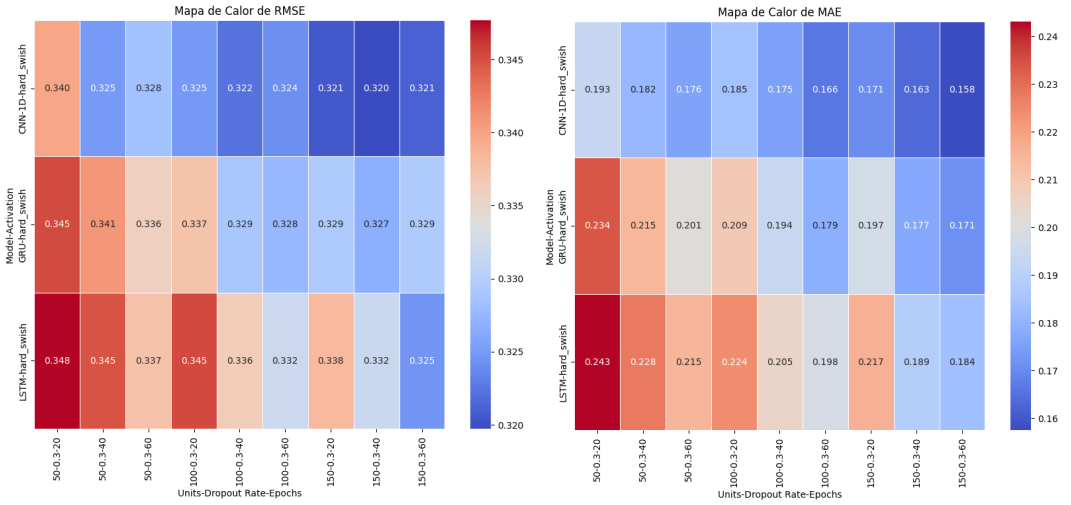

Figura 5.18: Métricas RMSE y MAE de RNNs en Validación con mAlexNet

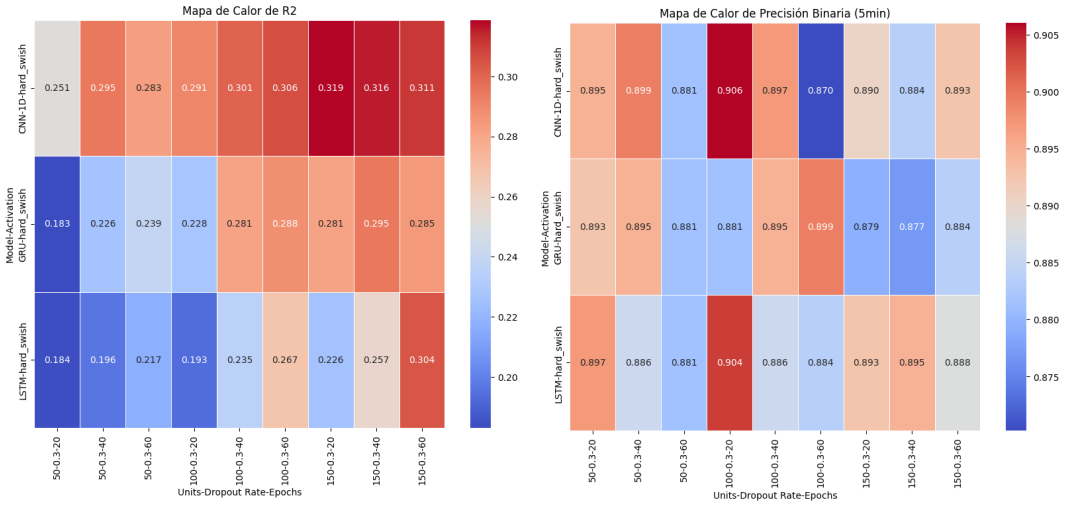

Figura 5.19: Métricas R2 y Precisión 5 min de RNNs en Validación con mAlexNet

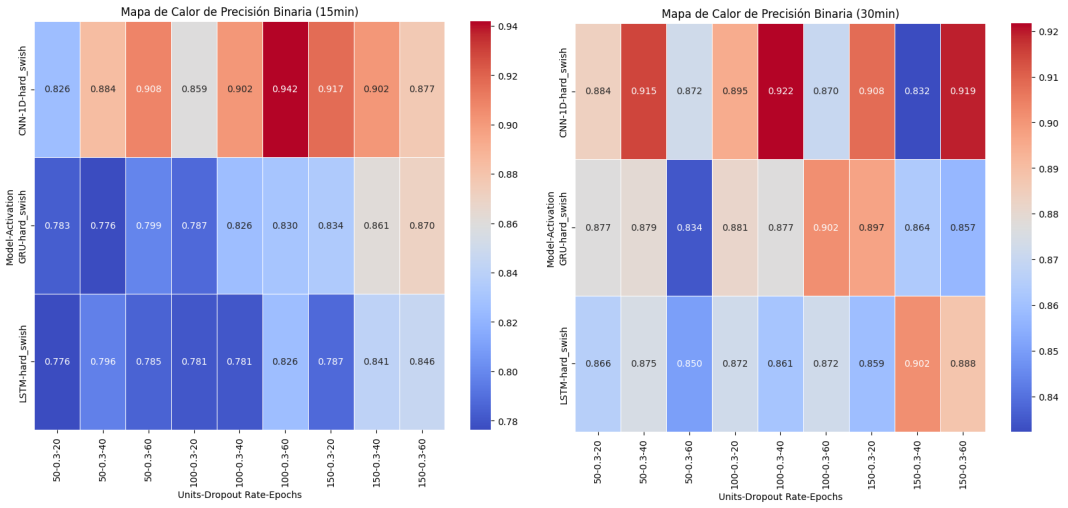

Figura 5.20: Métricas Precisión 15 min y Precisión 30 min de RNNs en Validación con mAlexNet

gura 5.22 y 5.23. El modelo B también destaca, con el RMSE más bajo de 0.303 y sólidas predicciones a través de los intervalos de tiempo , particularmente a los 15 minutos con una precisión de 0.951. El modelo W es notable por su alta precisión en 5 minutos con una puntuación de 0.969 y mantiene una fuerte coherencia en general.

Por otro lado, el modelo F, aunque tiene un MAE bajo de 0.142, muestra la mejor predicción a los 30 minutos con una precisión impresionante de 0.984 como se observa en la Figura 5.24 y 5.25, lo que sugiere una precisión excepcional a medio plazo. Además, el modelo E resalta por su bajo MAE de 0.138 y un alto R2 de 0.534, demostrando una habilidad destacada para minimizar los errores absolutos en sus predicciones.

La consistencia en el rendimiento a través de los distintos intervalos de tiempo es clave para la confiabilidad de los modelos, y modelos como $\tilde{\text{N}}$, B y W muestran esta habilidad de manera impresionante.

Para mas detalle se puede observar en el Capitulo Anexo la Tabla 7.4

Al analizar conjuntamente los resultados de los conjuntos de validación y prueba con características de mAlexNet, el modelo F emerge como uno de los más consistentes, con el MAE más bajo en el conjunto de prueba y una de las más altas precisión promedio de 0.932. En la validación, aunque no tiene el RMSE más bajo, su precisión promedio está muy cerca del mejor valor, y muestra robustez en los intervalos de tiempo considerados, lo cual es crucial para pronósticos precisos a corto y medio plazo como se observa en la Figura 5.26.

Modelos LSTM, GRU y CNN-1D con caracteristicas InceptionV3

Al analizar los resultados del conjunto de validación con características InceptionV3, el modelo E emerge como el líder en rendimiento, con un RMSE destacado de 0.337, un MAE líder de 0.176 y el más alto R2 de 0.256 como se observa en la Figura 5.27 y 5.28. Esta combinación de métricas demuestra su precisión y capacidad de ajuste superior en las predicciones. Además, el modelo E sobresale en consistencia a lo largo de todos los intervalos de tiempo, con una precisión destacada en los 15, 30, 45 y 60 minutos, teniendo el valor más alto en cada uno de estos y logrando la mayor precisión promedio con 0.894.

Cercano en rendimiento se encuentra el modelo D, que con un RMSE de 0.340 y un MAE de 0.180, muestra gran competencia y una impresionante precisión promedio de 0.894, comparable a la del modelo E. Aunque sus otras

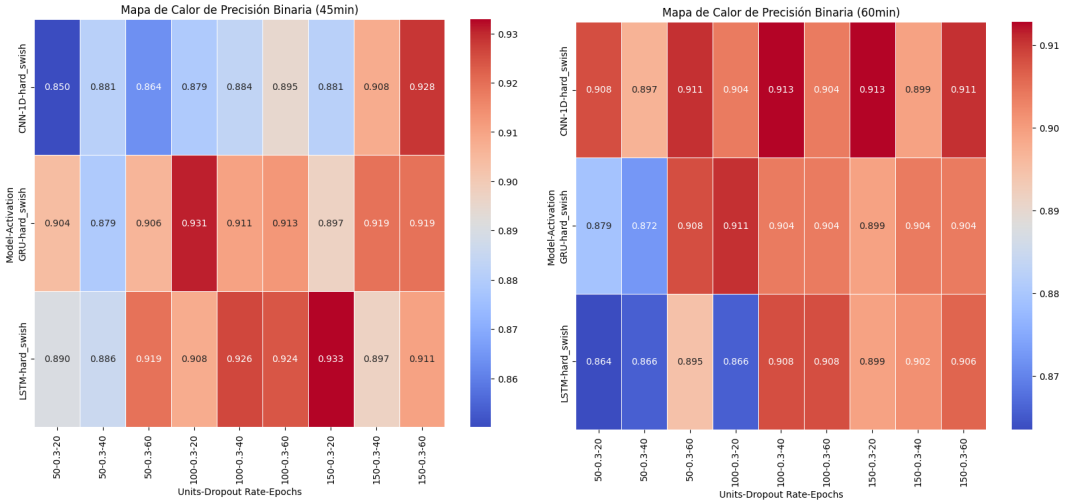

Figura 5.21: Métricas Precisión 45 min y Precisión 60 min de RNNs en Validación con mAlexNet

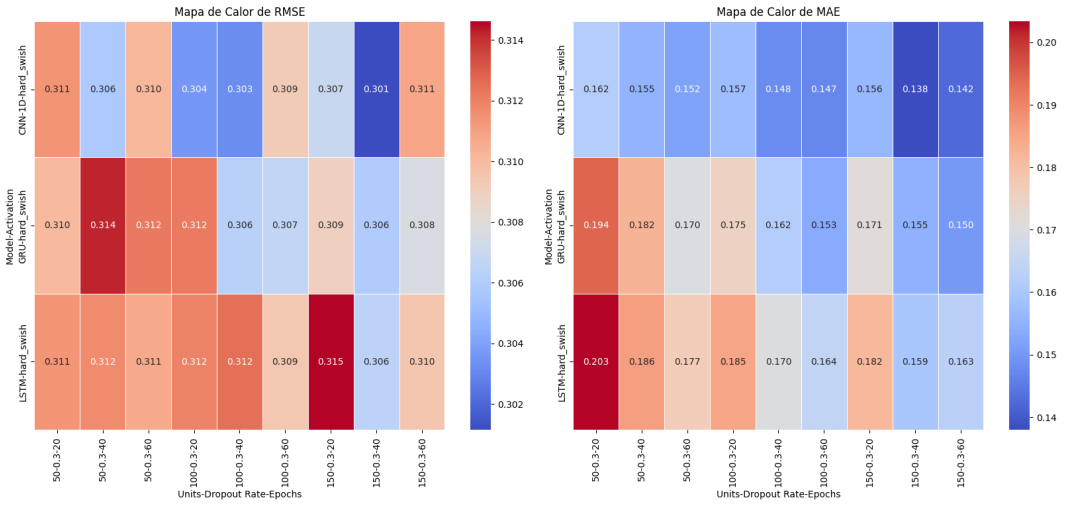

Figura 5.22: Métricas RMSE y MAE de RNNs en Prueba con mAlexNet

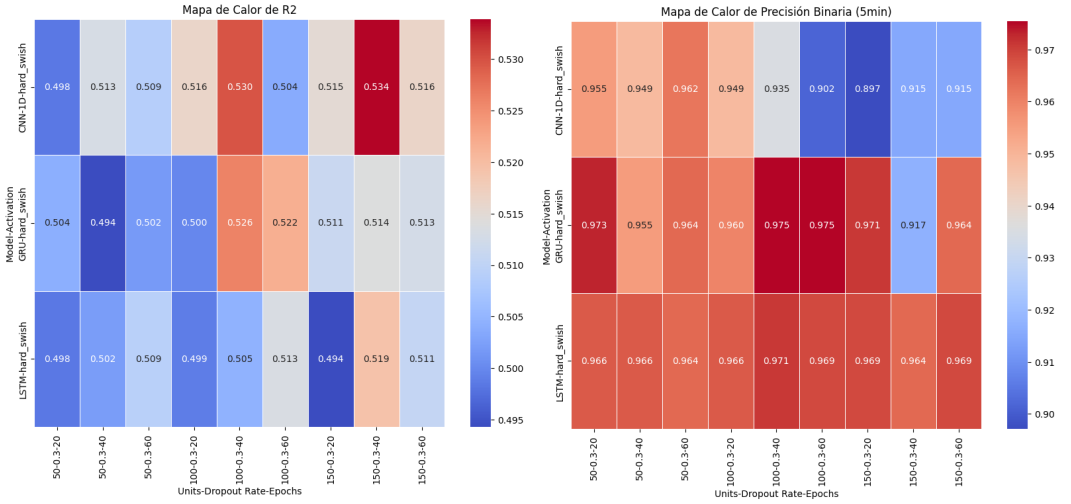

Figura 5.23: Métricas R2 y Precisión 5 min de RNNs en Prueba con mAlexNet

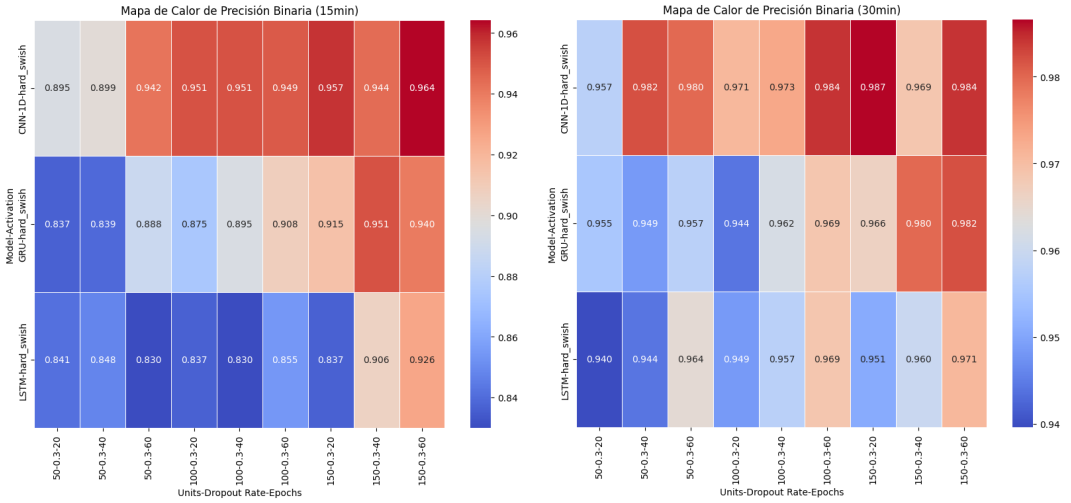

Figura 5.24: Métricas Precisión 15 min y Precisión 30 min de RNNs en Prueba con mAlexNet

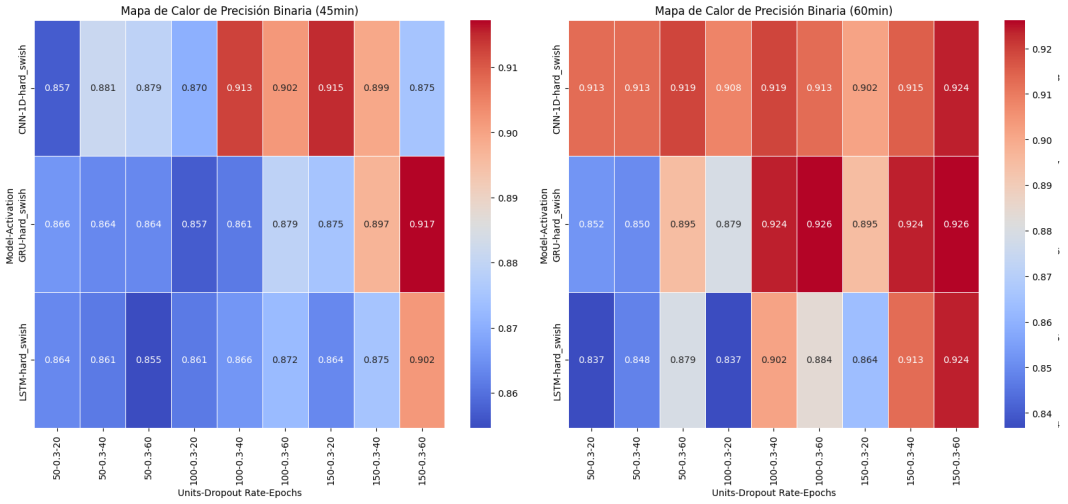

Figura 5.25: Métricas Precisión 45 min y Precisión 60 min de RNNs en Prueba con mAlexNet

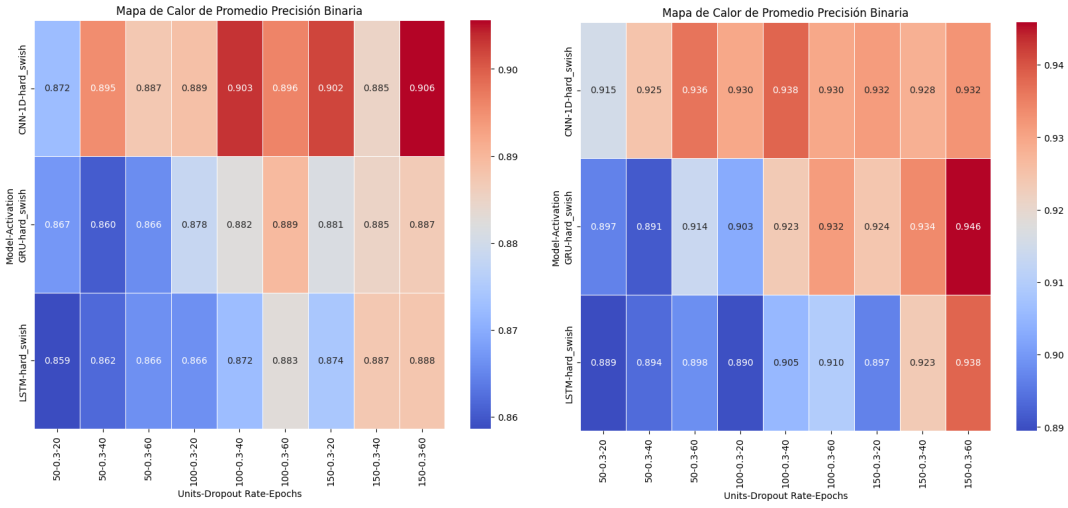

Figura 5.26: Métricas Precisión Promedio (Validación izquierda, Prueba Derecha) con mAlexNet

métricas son ligeramente inferiores, su precisión es particularmente notable en los intervalos de 15 minutos con 0.846 y de 60 minutos con 0.904, lo que indica su robustez y confiabilidad en diferentes plazos como se observa en la Figura 5.29 y 5.30.

Estos modelos, en particular el modelo E, demuestran una combinación excepcional de alta precisión y capacidad de generalización, lo cual los posiciona como candidatos ideales para aplicaciones prácticas si se corrobora esta capacidad con datos de prueba.

Para mas detalle se puede observar en el Capitulo Anexo la Tabla 7.5

Al examinar los resultados del conjunto de prueba con características InceptionV3, se observa un patrón similar al de los datos de validación, con algunos modelos específicos que sobresalen.

El modelo C emerge como el más destacado en este conjunto, con el valor más bajo de RMSE 0.315 y MAE 0.149, lo que implica que es el más preciso en términos de error y ajuste. Además, presenta una precisión promedio muy alta de 0.930 y es el mejor en la predicción a los 15 minutos con una precisión de 0.937, lo cual es relevante para pronósticos a corto plazo como se observa en la Figura 5.31 y 5.32.

El modelo Ñ iguala al modelo C en términos de RMSE 0.315 y comparte una precisión promedio idéntica de 0.930, además de tener un rendimiento muy similar en la mayoría de las métricas de tiempo. Esto lo establece como otro modelo consistente y fiable.

El modelo E, que se destacó en el conjunto de validación, mantiene un alto rendimiento en el conjunto de prueba también, especialmente con la precisión más alta a los 30 minutos con 0.978 y 45 minutos con 0.913 como se observa en la Figura 5.33 y 5.34. Con un promedio de precisión de 0.928, esto sugiere que el modelo E tiene una habilidad particularmente fuerte para predecir con más anticipación.

El modelo D, con el mejor R2 de 0.500, muestra su fortaleza en cuanto a la variabilidad de los datos que puede explicar. Además, presenta una precisión promedio excelente de 0.927 y es competitivo en todas las métricas de tiempo, destacando en los 5 minutos con una precisión de 0.933.

Para mas detalle se puede observar en el Capitulo Anexo la Tabla 7.6

Tras integrar los resultados de los conjuntos de validación y prueba con características InceptionV3, podemos concluir que los modelos C, Ñ y E muestran

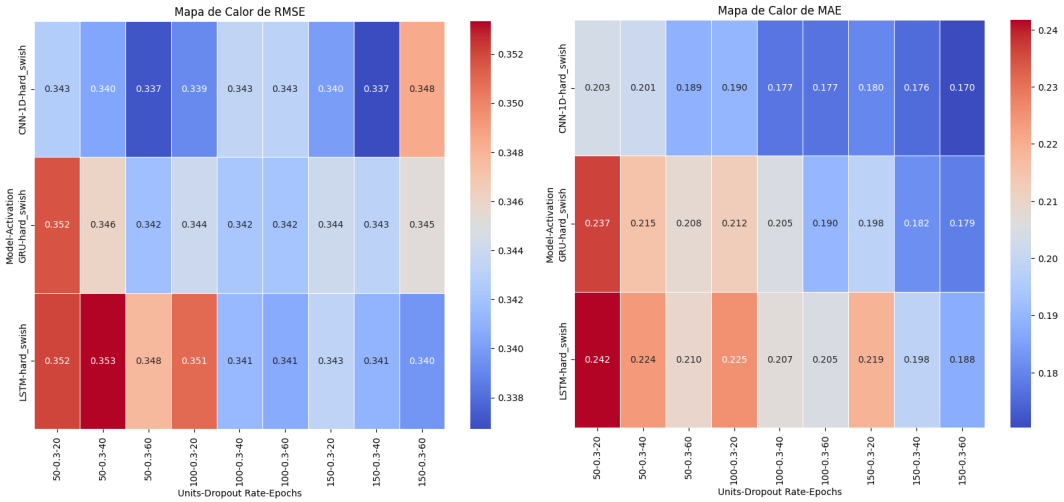

Figura 5.27: Métricas RMSE y MAE de RNNs en Validación con InceptionV3

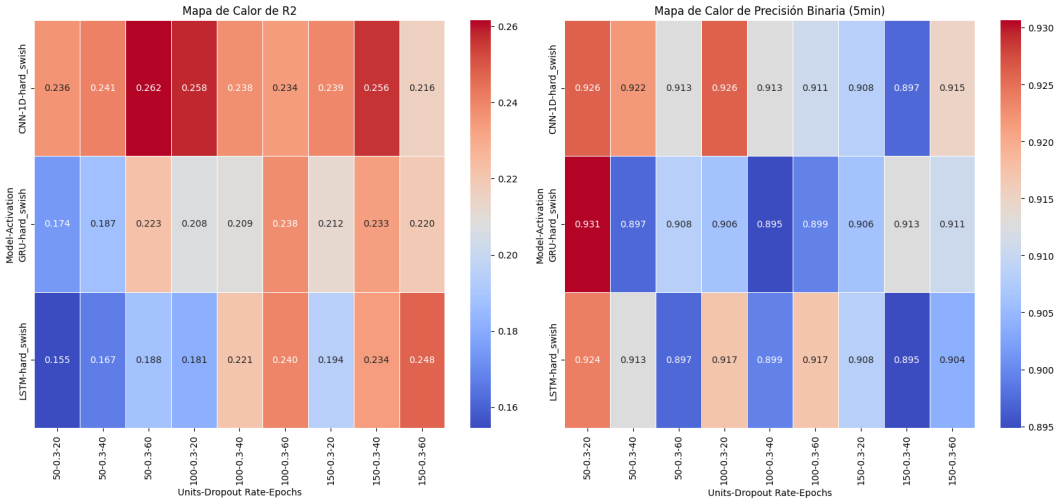

Figura 5.28: Métricas R2 y Precisión 5 min de RNNs en Validación con InceptionV3

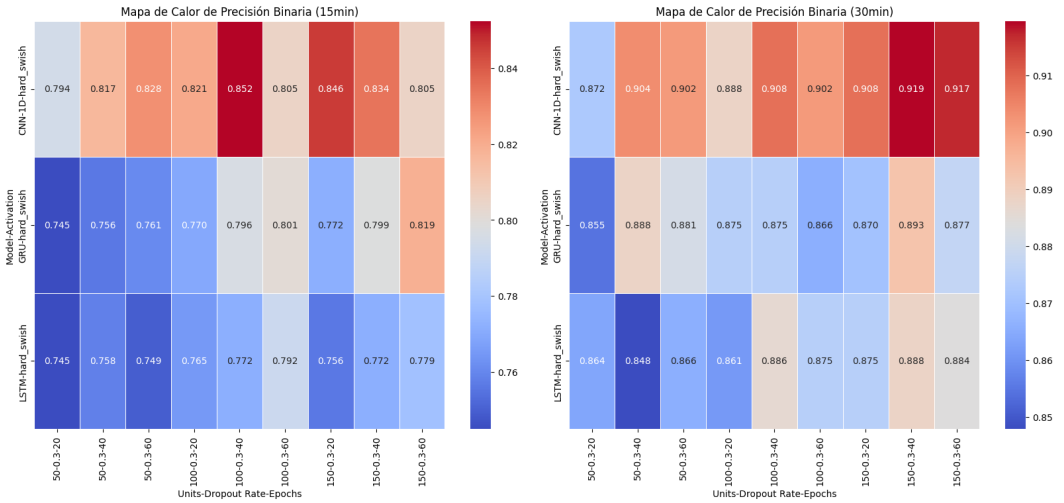

Figura 5.29: Métricas Precisión 15 min y Precisión 30 min de RNNs en Validación con InceptionV3

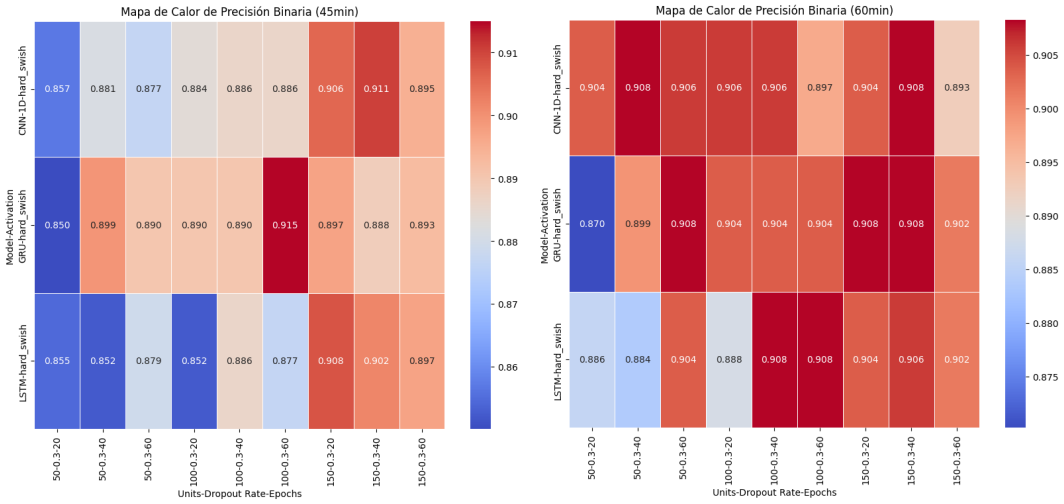

Figura 5.30: Métricas Precisión 45 min y Precisión 60 min de RNNs en Validación con InceptionV3

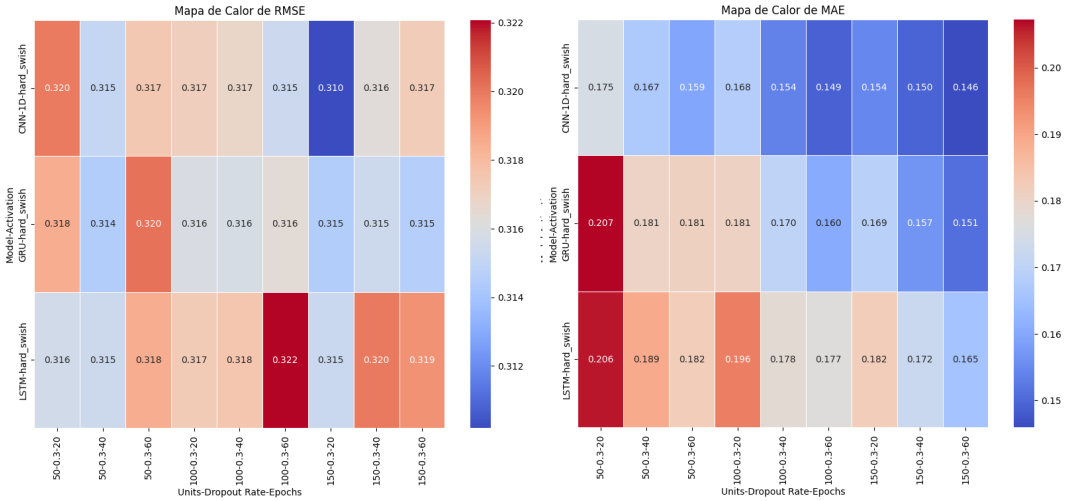

Figura 5.31: Métricas RMSE y MAE de RNNs en Prueba con InceptionV3

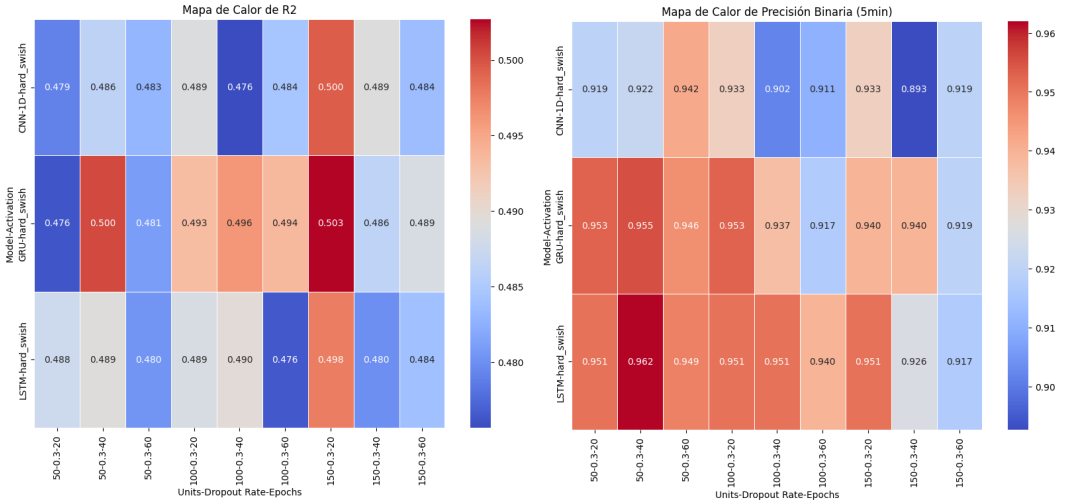

Figura 5.32: Métricas R2 y Precisión 5 min de RNNs en Prueba con InceptionV3

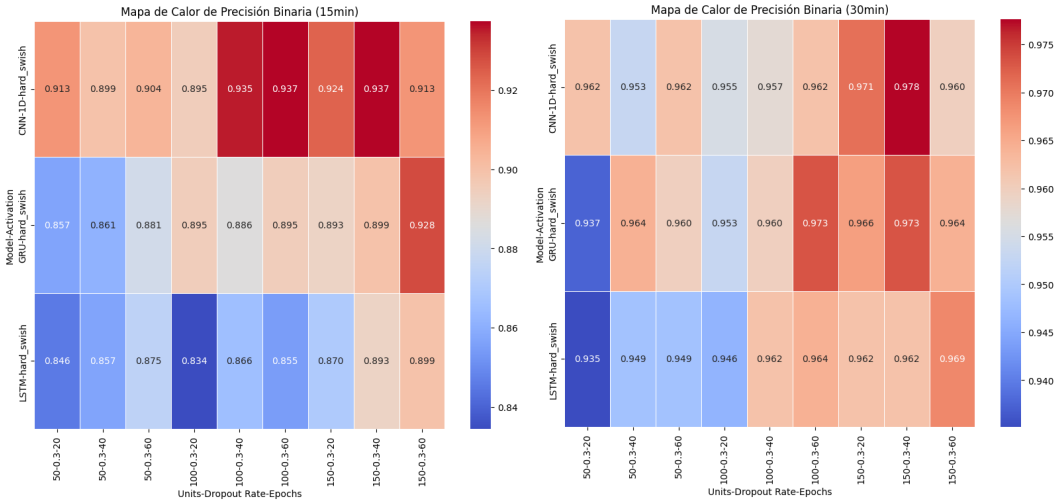

Figura 5.33: Métricas Precisión 15 min y Precisión 30 min de RNNs en Prueba con InceptionV3

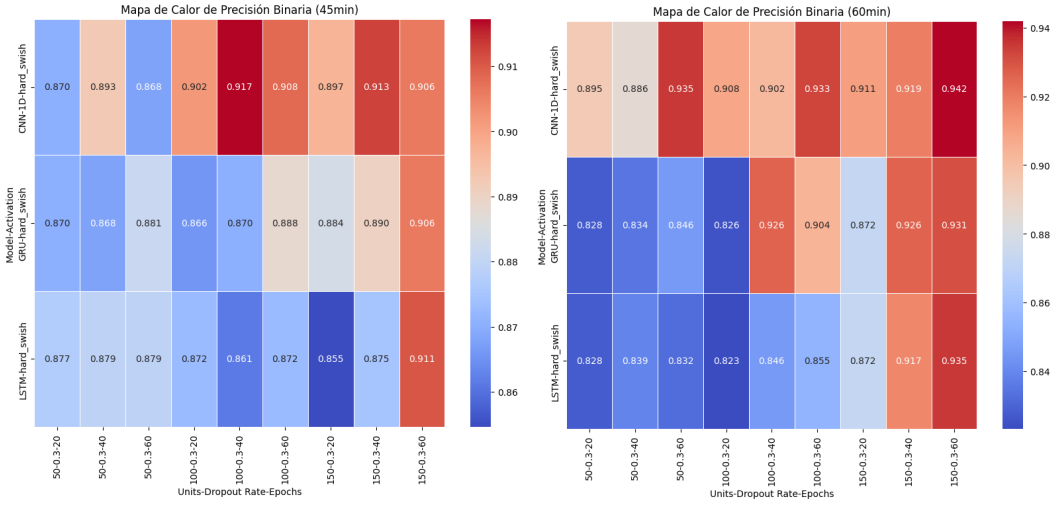

Figura 5.34: Métricas Precisión 45 min y Precisión 60 min de RNNs en Prueba con InceptionV3

un desempeño superior, con consistencia en sus métricas a través de los diferentes intervalos de tiempo y entre los conjuntos de datos. El modelo E lidera en validación con la mejor precisión promedio y métricas de error, mientras que los modelos C y Ñ sobresalen en la prueba con la precisión promedio más alta y excelentes resultados en intervalos específicos, lo que los hace candidatos ideales para aplicaciones prácticas como se observa en la Figura 5.35.

Al analizar los resultados de la validación con características MobileNetV2, observamos un rendimiento ligeramente más equilibrado en comparación con los modelos previos basados en InceptionV3.

Modelos LSTM, GRU y CNN-1D con caracteristicas MobileNetV2

El modelo K lidera el promedio de precisión con un valor destacado de 0.897, siendo también el más alto en los intervalos de 45 minutos con un 0.926. Esto sugiere que es particularmente eficaz para predecir con un rango medio de anticipación. Además, presenta un RMSE competitivo de 0.336 y un MAE de 0.200, aunque no son los valores más bajos de estas métricas en el conjunto de validación como se observa en la Figura 5.36 y 5.37.

En general, los modelos con características MobileNetV2 en Validación muestran una distribución de rendimiento más uniforme en todos los intervalos de tiempo como se observa en la Figura 5.39 y 5.38. Esto podría ser beneficioso para una integración futura donde se requiera una misma capacidad de predicción en distintos intervalos de tiempo.

Para mas detalle se puede observar en el Capitulo Anexo la Tabla 7.7

Los resultados del conjunto de prueba utilizando características de MobileNetV2 muestran que el modelo D se posiciona como el líder en general, obteniendo el menor RMSE con un valor de 0.299, el mejor R2 con un 0.545 y la precisión más alta en todos los intervalos de tiempo, excepto a los 60 minutos, donde su precisión es de 0.924. Este modelo demuestra no solo la capacidad de predecir con alta precisión a corto plazo, sino también de mantener esta precisión a largo plazo con un promedio impresionante de 0.942 como se observa en la Figura 5.40 y 5.41.

El modelo Ñ también tiene un rendimiento destacado, con el MAE más bajo con un valor de 0.146 y una precisión promedio muy cercana al líder, con 0.939, además de compartir la precisión más alta en los intervalos de 5 y 30 minutos con el modelo D, donde ambos modelos alcanzan una precisión de 0.971 y 0.982 respectivamente como se observa en la Figura 5.42 y 5.43.

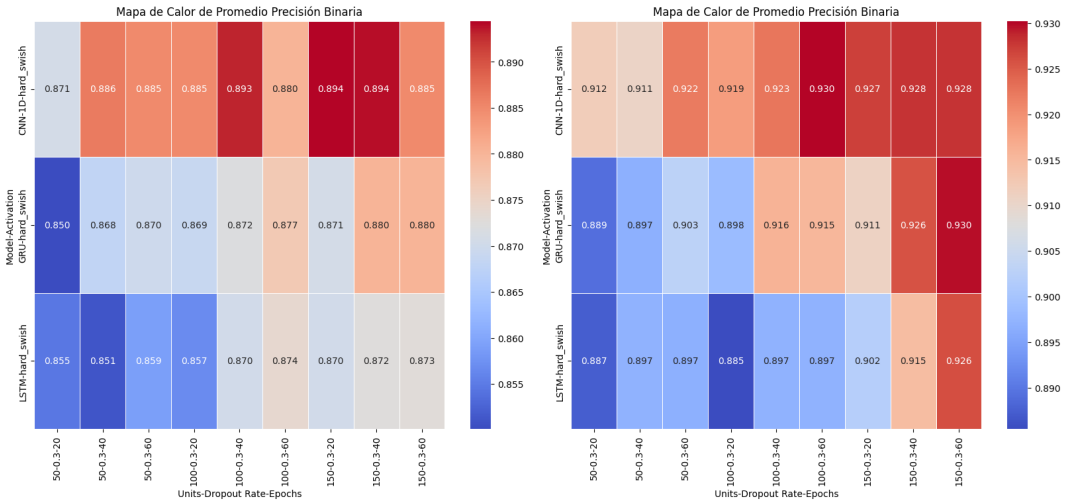

Figura 5.35: Métricas Precisión Promedio (Validación izquierda, Prueba Derecha) con InceptionV3

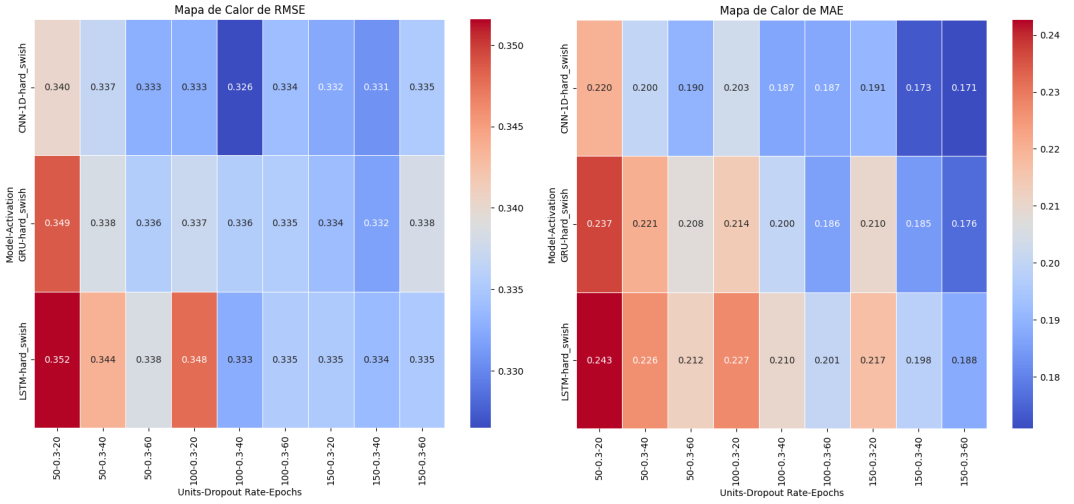

Figura 5.36: Métricas RMSE y MAE de RNNs en Validación con MobileNetV2

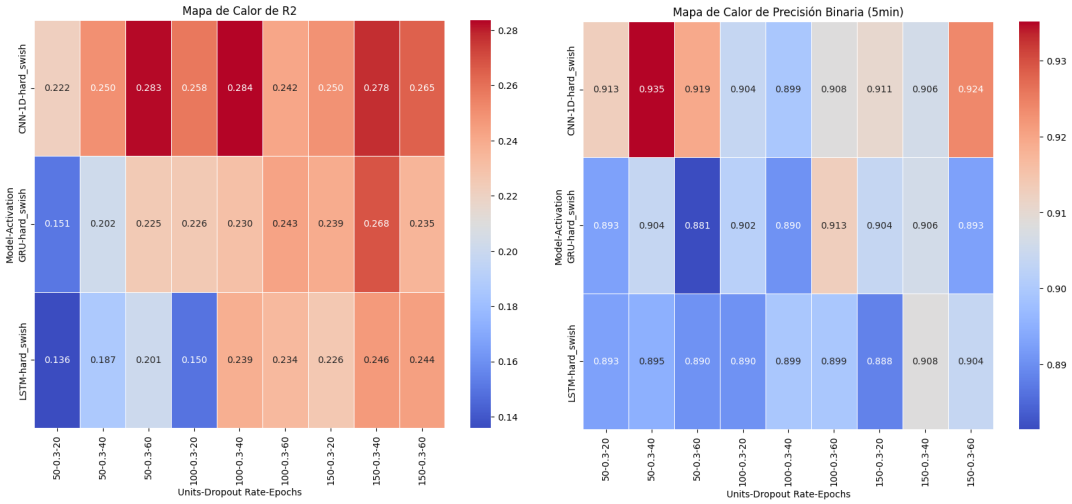

Figura 5.37: Métricas R2 y Precisión 5 min de RNNs en Validación con MobileNetV2

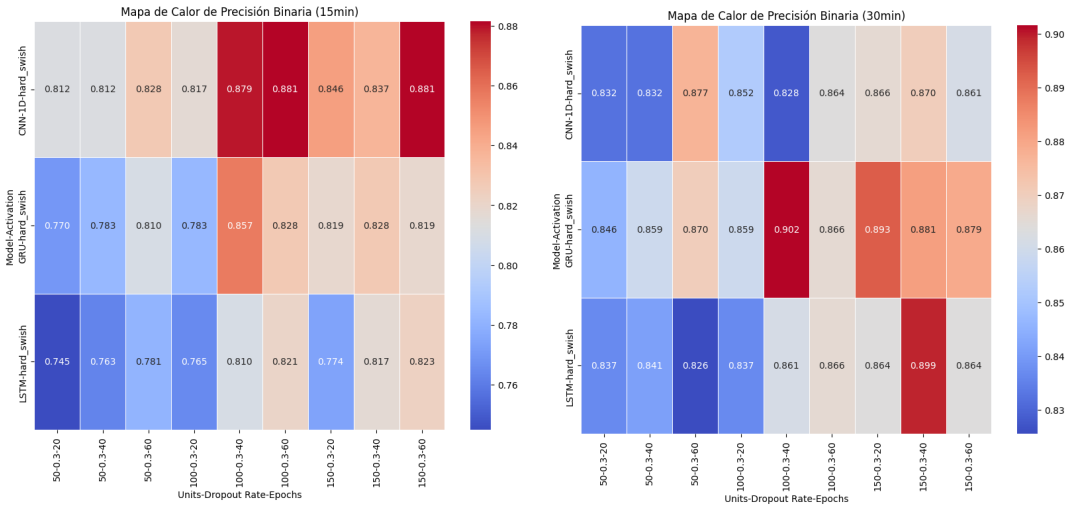

Figura 5.38: Métricas Precisión 15 min y Precisión 30 min de RNNs en Validación con MobileNetV2

El modelo C, aunque no el mejor en RMSE, destaca por su alto R2 de 0.531 y precisión consistente a través de todos los intervalos de tiempo, especialmente en los 15 minutos donde su precisión es de 0.915, y un valor sólido en la precisión promedio de 0.927 como se observa en la Figura 5.42.

Para mas detalle se puede observar en el Capitulo Anexo la Tabla 7.8

Integrando los resultados de validación y prueba con características MobileNetV2, destaca la consistencia en el rendimiento sobresaliente de los modelos D y C como se observa en la Figura 5.44. El modelo D no solo ostenta el menor RMSE de 0.299 y el más alto R2 de 0.545 en la prueba, sino también mantiene la precisión promedio más alta de 0.942, evidenciando su superioridad en la generalización de las predicciones a lo largo de todos los intervalos de tiempo. Por su parte, el modelo C se presenta como un competidor cercano, con un RMSE de 0.304 y un R2 de 0.531 en la prueba, complementado con una alta precisión, particularmente a los 15 minutos con un 0.915.

Estos modelos, con sus métricas destacadas, reafirman la utilidad de las características MobileNetV2 para proporcionar predicciones fiables y precisas, subrayando su idoneidad para implementaciones prácticas donde se requiere confiabilidad en los rangos de tiempo.

Modelos LSTM, GRU y CNN-1D con caracteristicas MobileNetV3 Small personalizado

Al evaluar los modelos con características de MobileNetV3 Small personalizado en el conjunto de validación, se observa que el rendimiento es moderado, con valores de R2 cercanos a cero o negativos, lo que indica una capacidad limitada de los modelos para capturar la varianza de los datos de destino.

El modelo I muestra un desempeño levemente mejor con el RMSE más bajo (0.385) y una precisión promedio de 0.868, destacando en la predicción a corto plazo (5 min) con una precisión de 0.884 como se observa en la Figura 5.45 y 5.46.

El modelo A tiene un RMSE de 0.374 y mantiene una precisión promedio de 0.864, sobresaliendo en el intervalo de tiempo de 45 minutos con una precisión de 0.902 como se observa en la Figura 5.48.

Finalmente, el modelo F, con un MAE de 0.241, sugiere una precisión razonable en las predicciones, y se destaca en el intervalo de 15 minutos con una precisión de 0.805 y una precisión promedio de 0.860 como se observa en la Figura 5.47.

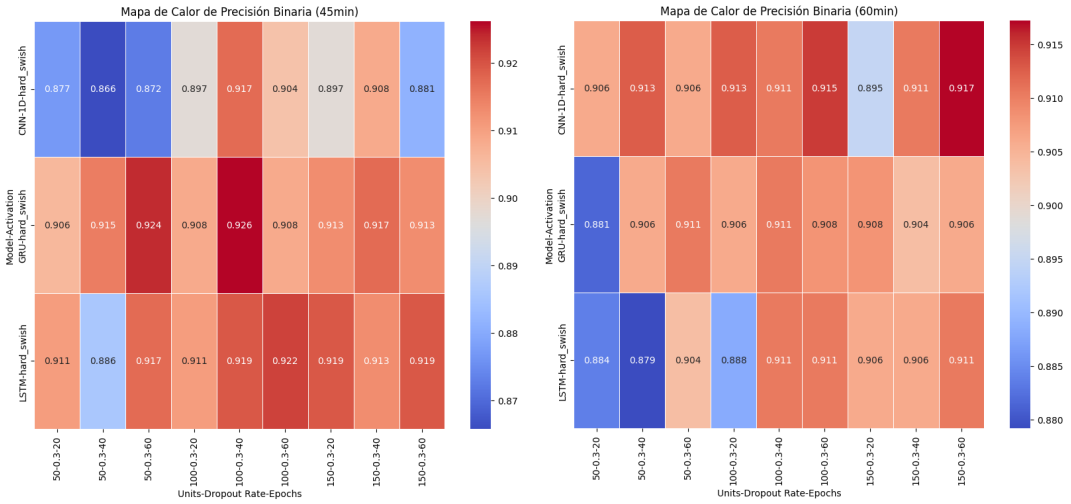

Figura 5.39: Métricas Precisión 45 min y Precisión 60 min de RNNs en Validación con MobileNetV2

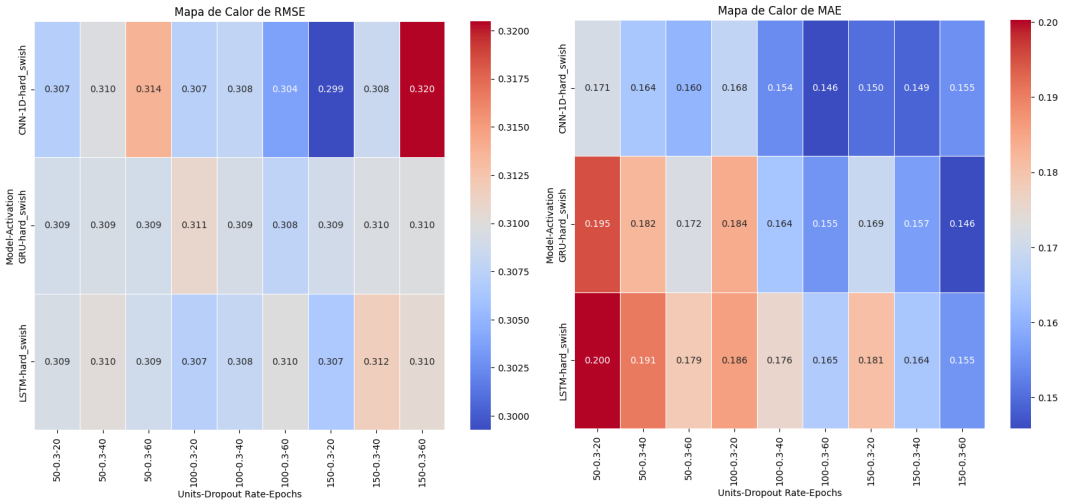

Figura 5.40: Métricas RMSE y MAE de RNNs en Prueba con MobileNetV2

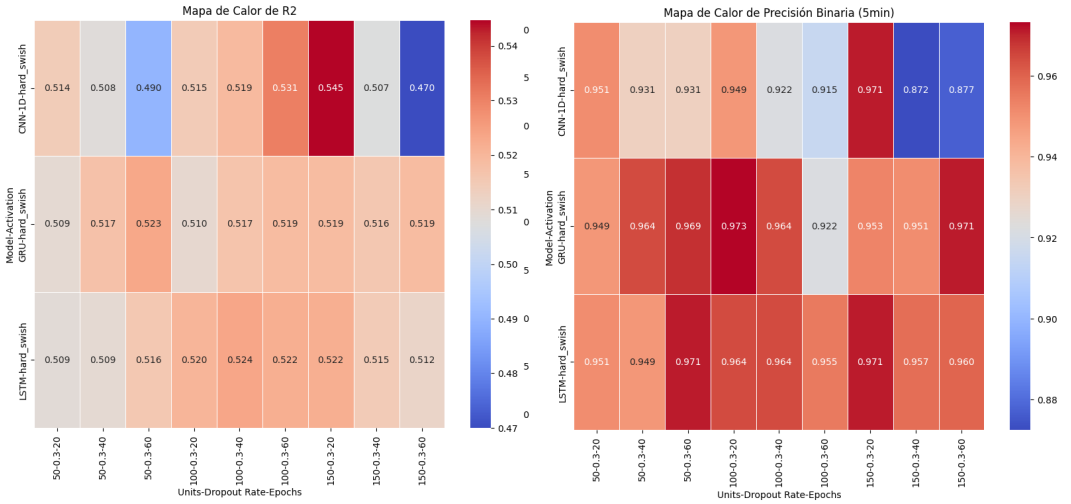

Figura 5.41: Métricas R2 y Precisión 5 min de RNNs en Prueba con MobileNetV2

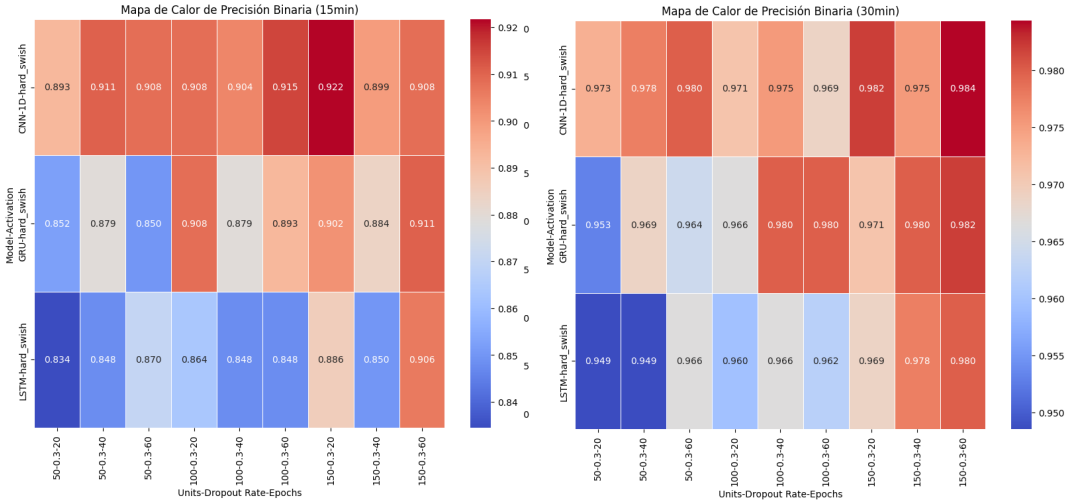

Figura 5.42: Métricas Precisión 15 min y Precisión 30 min de RNNs en Prueba con MobileNetV2

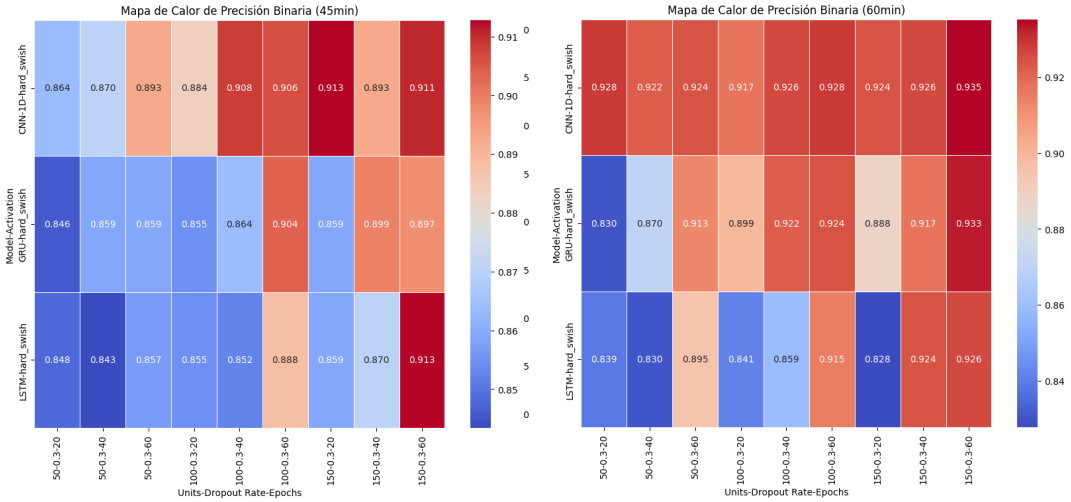

Figura 5.43: Métricas Precisión 45 min y Precisión 60 min de RNNs en Prueba con MobileNetV2

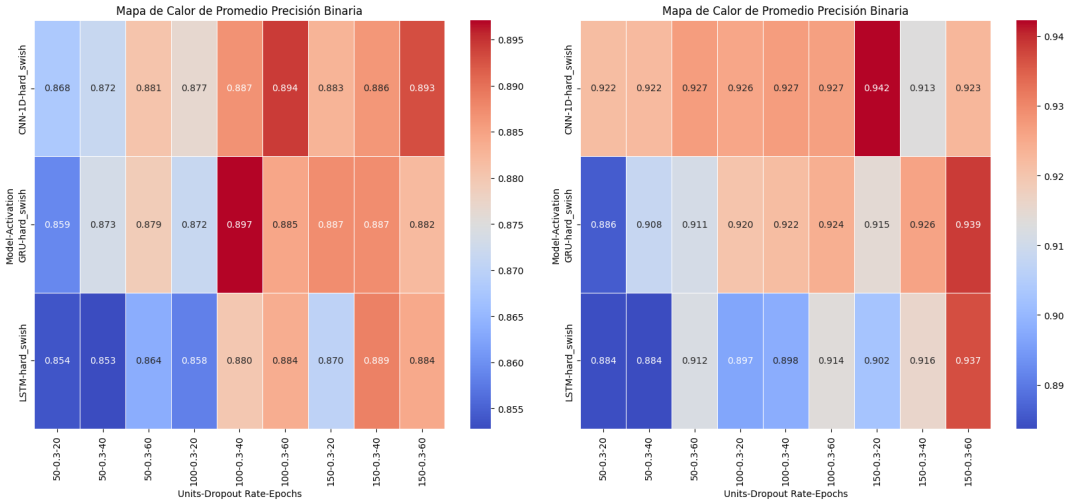

Figura 5.44: Métricas Precisión Promedio (Validación izquierda, Prueba Derecha) con MobileNetV2

Para mas detalle se puede observar en el Capitulo Anexo la Tabla 7.9

El modelo A presenta el mejor rendimiento promedio (0.865) y la mejor precisión a 5 y 15 minutos, con un RMSE de 0.354 y un R2 de 0.362 como se observa en la Figura 5.49 y 5.50.

El modelo E tiene un rendimiento similar con un RMSE de 0.352 y el R2 más alto (0.366), destacando especialmente a los 30 y 60 minutos como se observa en la Figura 5.51 y 5.52.

El modelo S tiene el RMSE más bajo (0.343) y el R2 más alto (0.392), lo que indica un buen ajuste general, aunque su desempeño en intervalos de tiempo específicos es superado por otros modelos.

Para mas detalle se puede observar en el Capitulo Anexo la Tabla 7.10

Al observar ambos conjuntos, el Modelo A se destaca consistentemente en los dos, lo que sugiere una buena generalización y robustez en su desempeño como se observa en la Figura 5.53 . El Modelo F también muestra un buen rendimiento en la validación, aunque no está entre los tres primeros en el conjunto de prueba.

Por otro lado, el Modelo E se destaca en el conjunto de prueba, especialmente en los intervalos de 30 y 60 minutos, lo que podría indicar una especialización en predicciones a medio plazo. Aunque no está entre los mejores en el conjunto de Validación, su buen rendimiento en prueba podría ser significativo dependiendo del uso de CNNs livianas en las cámaras lo que facilitaría la integración posterior.

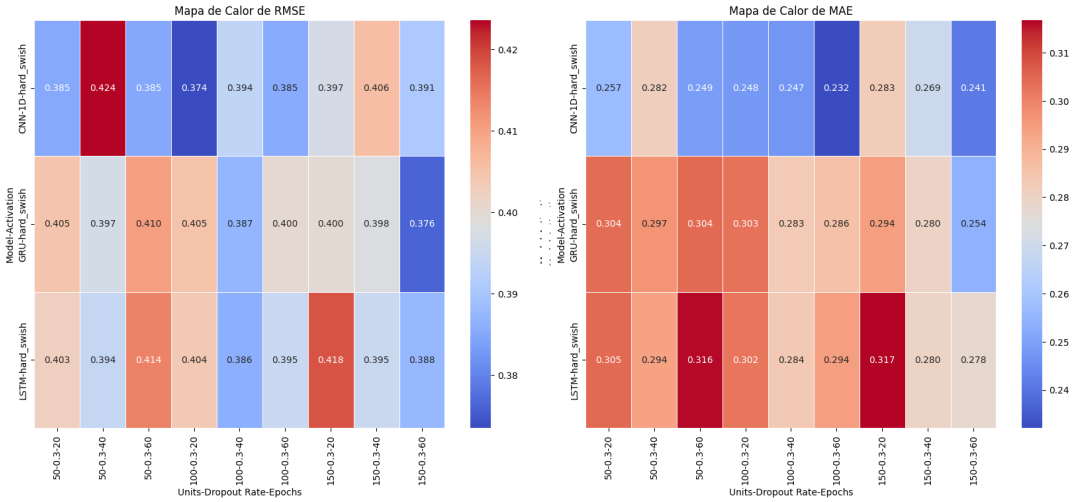

Figura 5.45: Métricas RMSE y MAE de RNNs en Validación con MobileNetV3 Small Personalizada

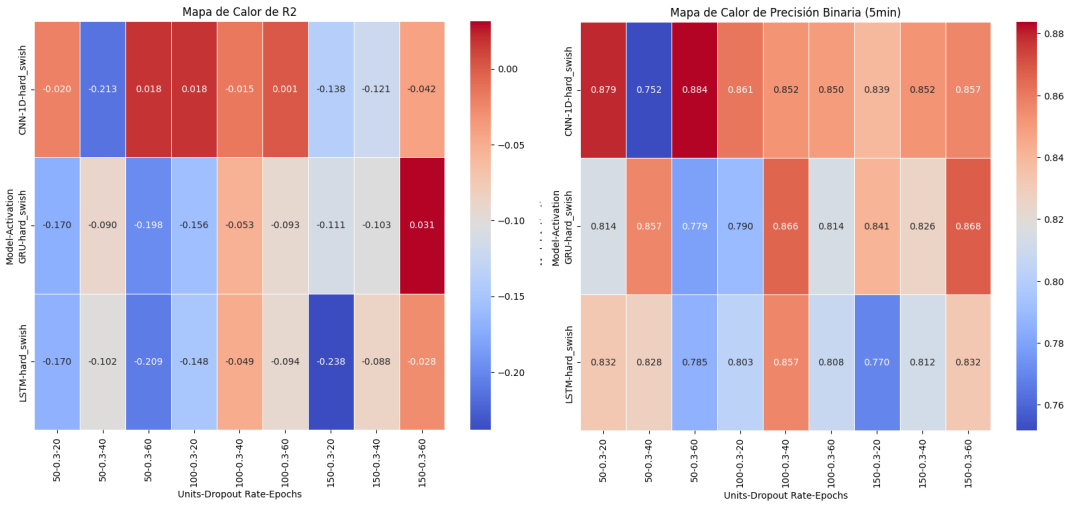

Figura 5.46: Métricas R2 y Precisión 5 min de RNNs en Validación con MobileNetV3 Small Personalizada

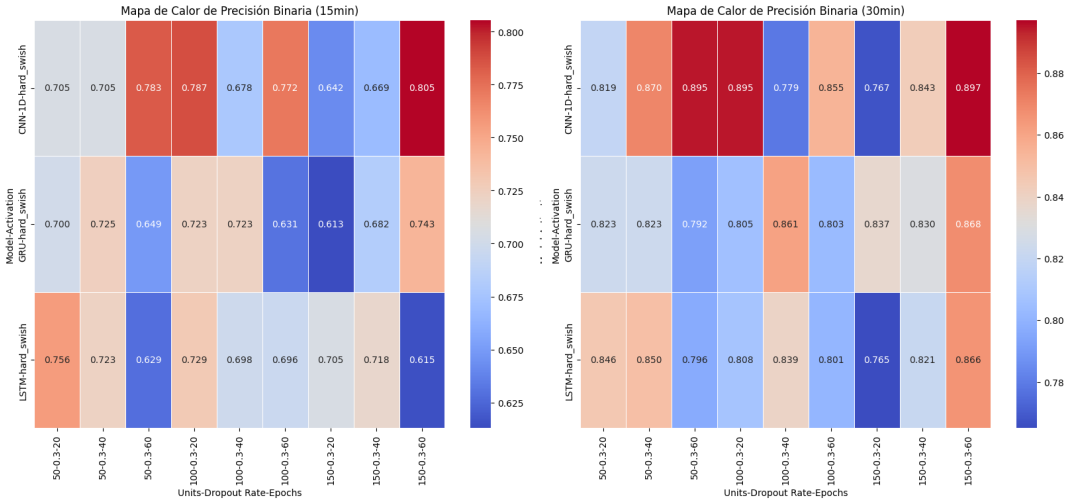

Figura 5.47: Métricas Precisión 15 min y Precisión 30 min de RNNs en Validación con MobileNetV3 Small Personalizada

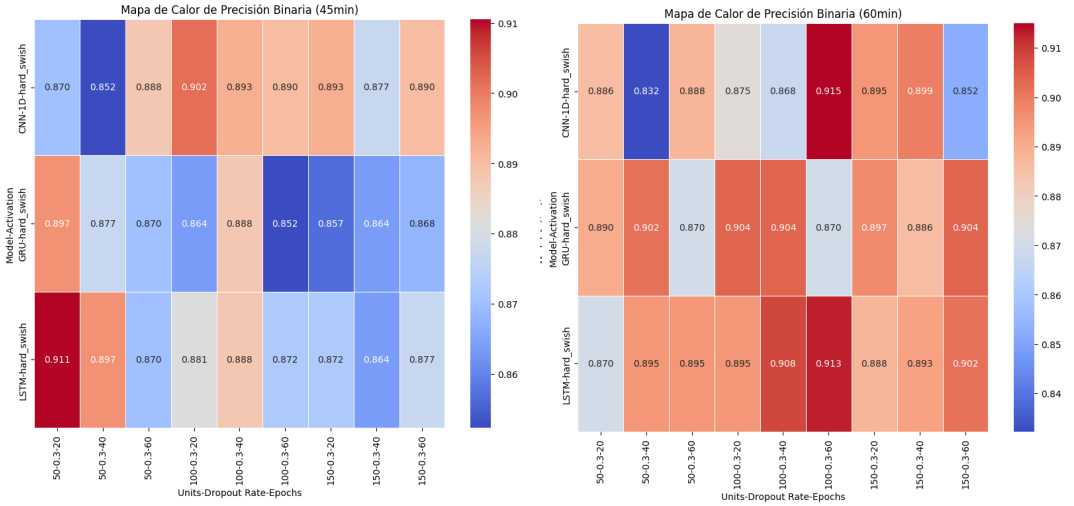

Figura 5.48: Métricas Precisión 45 min y Precisión 60 min de RNNs en Validación con MobileNetV3 Small Personalizada

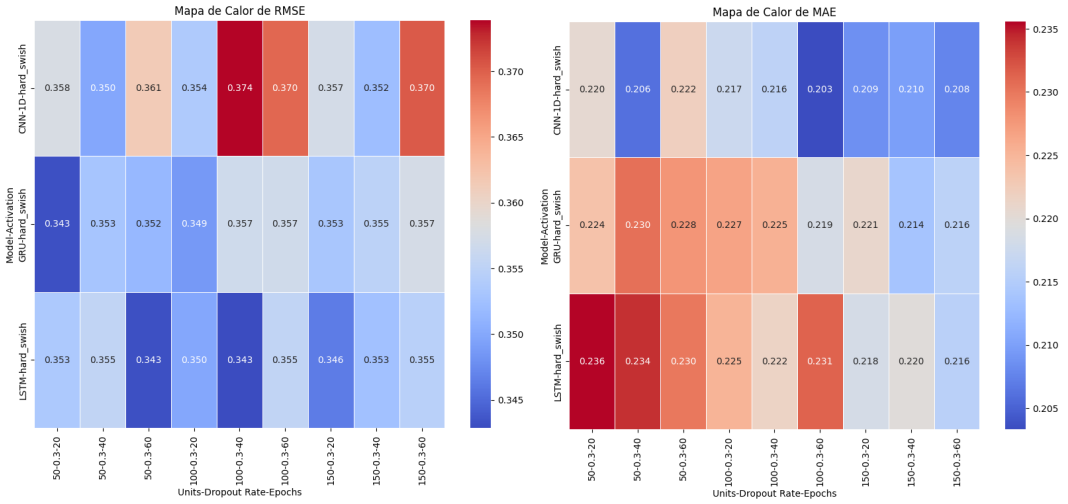

Figura 5.49: Métricas RMSE y MAE de RNNs en Prueba con MobileNetV3 Small Personalizada

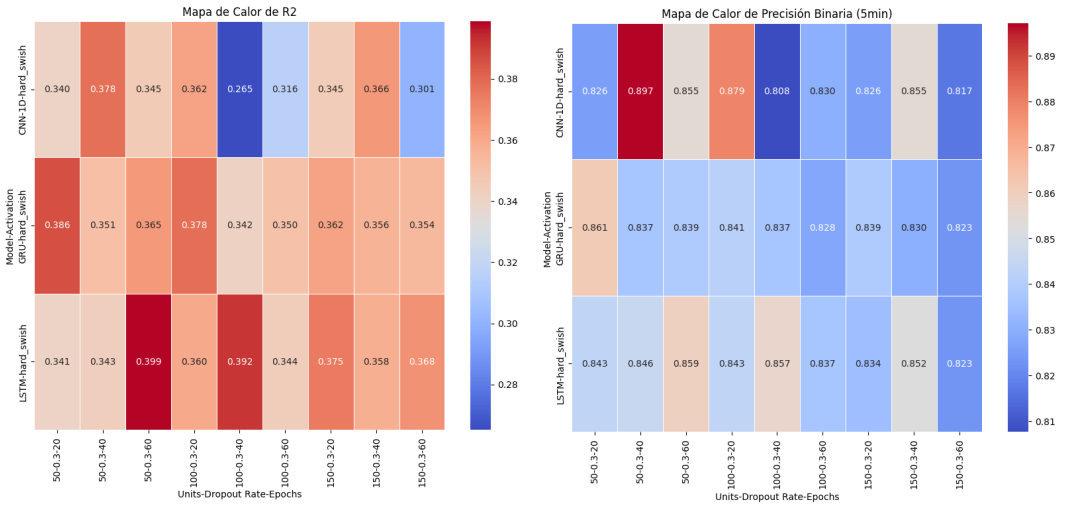

Figura 5.50: Métricas R2 y Precisión 5 min de RNNs en Prueba con MobileNetV2

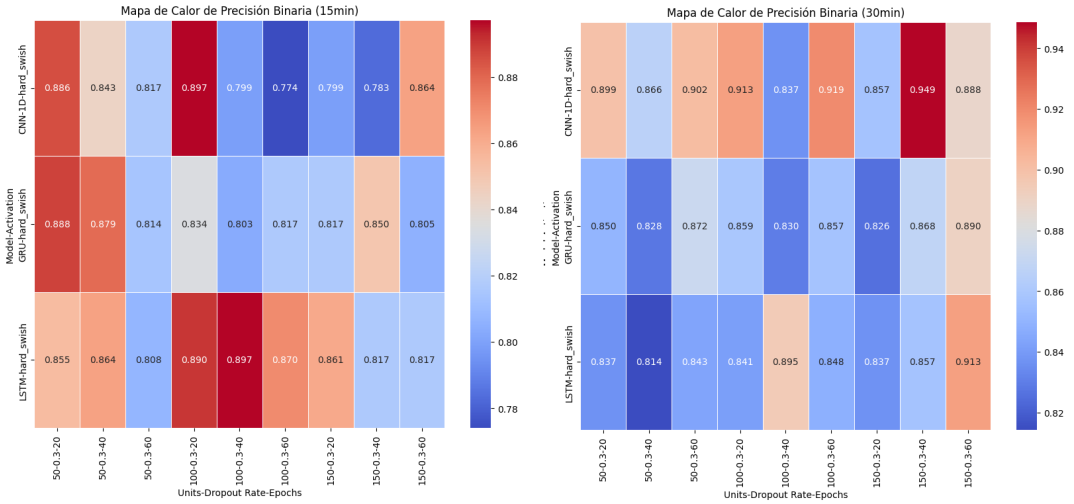

Figura 5.51: Métricas Precisión 15 min y Precisión 30 min de RNNs en Prueba con MobileNetV3 Small Personalizada

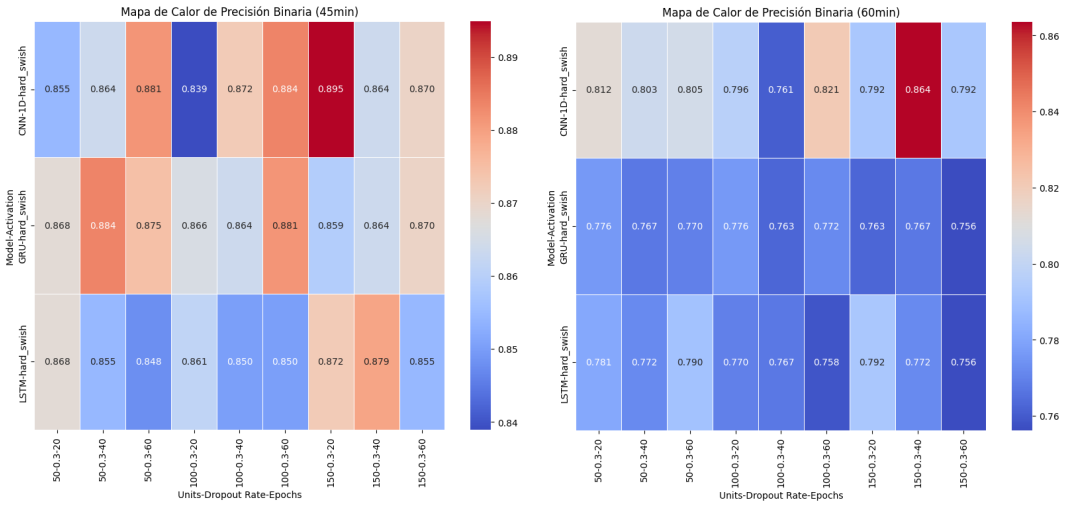

Figura 5.52: Métricas Precisión 45 min y Precisión 60 min de RNNs en Prueba con MobileNetV3 Small Personalizada

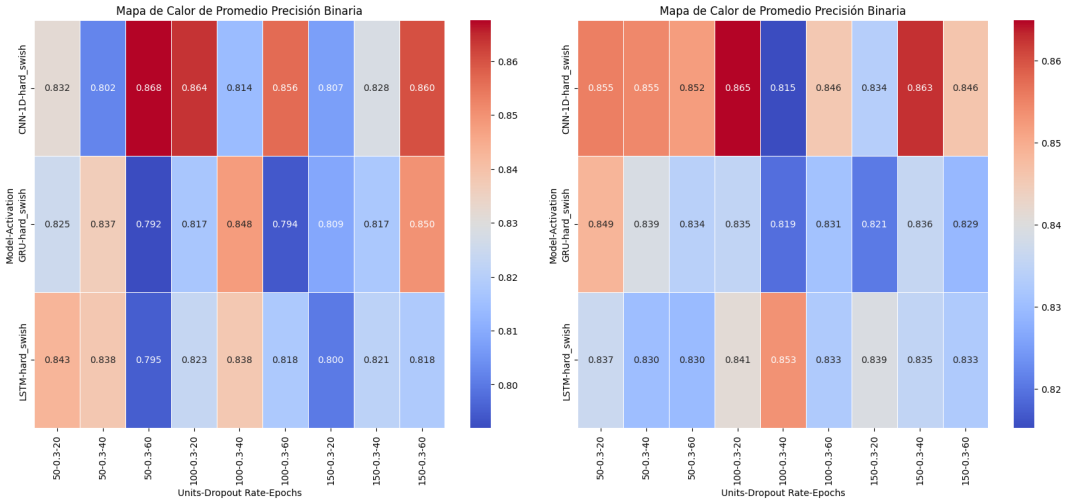

Figura 5.53: Métricas Precisión Promedio (Validación izquierda, Prueba Derecha) con MobileNetV3 Small Personalizada

Capítulo 6

Conclusiones y líneas de trabajo futuro

1. Introducción

Al abordar esta investigación, su núcleo ha sido la intersección de arquitecturas de redes neuronales convolucionales (CNN) y redes neuronales recurrentes (RNN) con el propósito de predecir la ocupación de espacios libres y ocupados de estacionamiento que al momento de su redacción no tiene un objeto comparable por su innovadora visión. Esta tarea, intrínsecamente desafiante por su naturaleza temporal y espacial, ha sido abordada con un enfoque experimental que atraviesa la selección de modelos CNN, la extracción de características temporales, y la síntesis con modelos RNN para la predicción final.

Durante la fase de desarrollo se enfrentaron distintos desafíos con los modelos CNN, sus parámetros de configuración, así como cuáles podrían tener mayor capacidad predictiva de espacios libres y ocupados, pero considerando la significancia y el impacto que podrían tener las características extraídas de las imágenes desde la perspectiva temporal en los modelos RNN que, en definitiva, serían quienes brindarían las predicciones finales.

Este trabajo se complementó luego con una exploración exhaustiva de modelos RNN que dio comienzo con la reducción de variabilidad, obteniendo a través de distintas configuraciones estabilidad hasta dominar el subajuste y sobreajuste que generaban las características en sí mismas.

2. Resumen de conclusiones

2.1. Sintonía entre mAlexNet y RNNs

mAlexNet ha demostrado ser una fuente robusta de características para los modelos RNN, y esto se evidencia en los datos. Por ejemplo, el modelo LSTM con 150 unidades y Dropout 0.3, una fusión de mAlexNet y LSTM, no solo obtiene una precisión promedio impresionante sino que también mantiene una desviación estándar relativamente baja en la precisión binaria a lo largo de múltiples horizontes temporales. Esto no solo subraya la capacidad de mAlexNet para capturar la esencia de las imágenes en el tiempo sino que también resalta la habilidad de LSTM para interpretar estas características con un sentido de memoria y continuidad.

Por otro lado, hay varias configuraciones de CNN-1D con características de mAlexNet que han demostrado una precisión destacada, como por ejemplo, la CNN-1D con 100 filtros, Dropout 0.3, usando Hard Sigmoid en 40 épocas con una precisión promedio de 0.921 agrupando las predicciones de validación y pruebas e incluso un 0.916 en precisión binaria en 5 minutos.

Si consideramos la estabilidad en la predicción, el modelo CNN-1D con 150 filtros, Dropout 0.3, Hard Sigmoid en 60 épocas tiene una capacidad de 0.903 promediando validación y pruebas con una desviación estándar de solo 0.020 respecto a las otras predicciones.

Se observa que los modelos con mAlexNet, a pesar de su mayor complejidad, tienden a combinarse bien con las tres variantes principales de RNN: LSTM, GRU y CNN-1D. Esto sugiere que mAlexNet es capaz de extraer características robustas que son bien capturadas por diferentes arquitecturas de RNN.

2.2. Alternativas Livianas

Aunque los modelos basados en mAlexNet han mostrado un rendimiento sobresaliente, la eficiencia es un factor crítico, existiendo otras alternativas para arquitecturas de sistemas futuros. En este contexto, LSTM combinado con MobileNetV2 emerge como una opción atractiva siendo el modelo LSTM con 150 unidades, Dropout 0.3 en 60 épocas muy efectivo con una capacidad promedio entre validación y pruebas de 0.932. Además, en términos de consistencia, este modelo ha mantenido un alto rendimiento con una desviación estándar de 0.0263, lo que indica que la variabilidad en las predicciones es controlada,

manteniendo la fiabilidad del modelo incluso en intervalos de predicción más largos como se puede observar en la Figura 6.1 y 6.2 que refleja en un instante especifico en azul tanto con datos de Validación como con datos de Prueba inferidos en verde el comportamiento de la alternativa liviana en rojo respecto a la información inferida con el margen de error asociado en amarillo de todo el estacionamiento. Esta precisión es fundamental para decisiones inmediatas y de largo plazo donde se puedan integrar actualizaciones periódicas de los datos y entrenamiento consecutivo de los modelos.

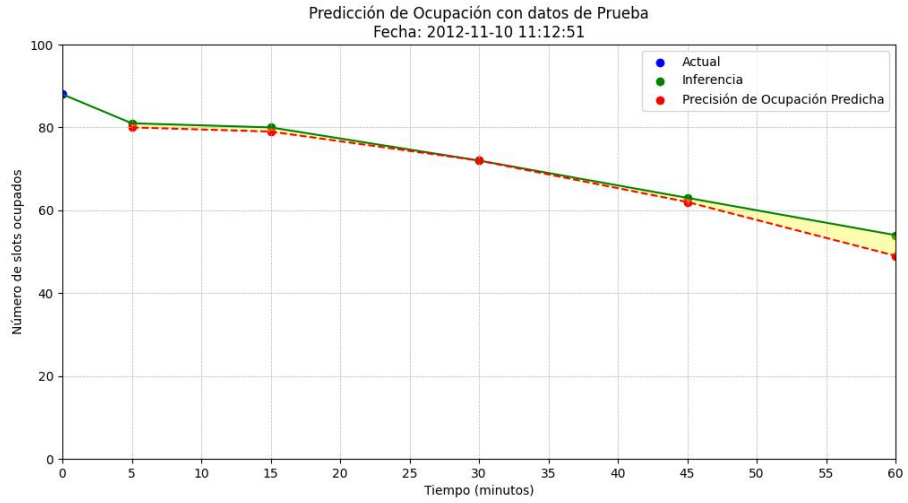

Figura 6.1: Predicción de Ocupación de espacios ocupados con datos de Prueba

2.3. Posibles Mejoras

La investigación realizada cubre algunos de los modelos propuestos y su interrelación, aunque no abarca la multitud de configuraciones posibles por la limitación temporal de su elaboración. Es de destacar que cada decisión en la etapa explorativa implicó una cantidad de pruebas importantes a medida que se desarrollaba la misma, siendo un factor clave la comprensión de las limitaciones de hardware disponible para el entrenamiento y valoración de caminos en la investigación.

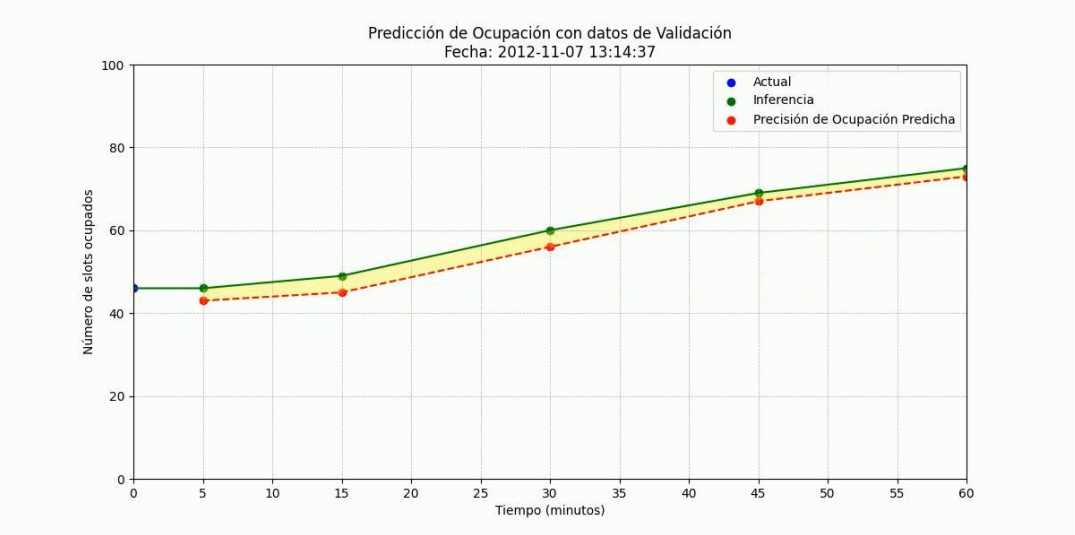

Figura 6.2: Predicción de Ocupación de espacios ocupados con datos de Validación

3. Trabajo Futuro

La identificación de los modelos óptimos de aprendizaje profundo, ajustados a las necesidades específicas de las soluciones de software en que se desplegarán, es solo un fragmento del amplio abanico que constituye este campo de estudio. Mirando hacia adelante, el trabajo futuro podría extenderse más allá de la búsqueda de eficiencia y optimización de los modelos descubiertos con la implementación de una arquitectura MLOps que permita el desarrollo de pruebas automáticas, versionado de modelos y estandarización del proceso científico [11].

También desde una visión practica se abre el espectro hacia la integración de estos modelos predictivos en aplicaciones reales, donde su funcionalidad pueda ser puesta a prueba en el contexto de ciudades inteligentes. Un objetivo clave sería el desarrollo de aplicaciones de geolocalización que no solo informen sobre la disponibilidad de espacios de estacionamiento sino que también se integren de manera fluida con los sistemas de gestión urbana, contribuyendo al flujo eficiente y sostenible del tráfico en entornos urbanos. Esto requerirá una colaboración interdisciplinaria que abarque desde la ingeniería de software hasta la planificación urbanística, con un énfasis en la usabilidad, accesibilidad y la experiencia del usuario.

4. Reflexiones Finales

Al reflexionar sobre el recorrido de esta investigación, se reconoce que el trayecto ha ido más allá de la construcción de modelos predictivos. Se ha entrelazado una narrativa en la que la adaptabilidad y la precisión se han convertido en protagonistas, con los datos contando una historia de avances y desafíos superados. El progreso alcanzado ilumina un pequeño paso hacia una solución paradigmática poniendo de manifiesto el papel fundamental que la tecnología podría desempeñar en la mejora de la vida cotidiana y el cuidado del medio ambiente.

La búsqueda de un equilibrio entre eficiencia y precisión plantea interrogantes cruciales para el futuro: ¿Cómo podemos maximizar el rendimiento de los modelos sin comprometer su exactitud? ¿Qué nuevas arquitecturas podrían emerger desde otras perspectivas como Transformers y revolucionar los paradigmas actuales? Estas son algunas de las preguntas que se quedan resonando al concluir este estudio, marcando el inicio de una exploración futura llena de oportunidades y retos aún por descubrir. Con cada pregunta que respondemos, emergen nuevas incógnitas donde reside el verdadero espíritu de la investigación científica.

Capítulo 7

Anexos

1. Tablas de evaluaciones de Modelos RNNs

Modelo	RMSE	MAE	R2	5 min	15 min	30 min	45 min	60 min	Promedio
B	0.350	0.212	0.190	0.879	**0.861**	**0.872**	**0.922**	**0.911**	**0.889**
C	0.353	0.201	0.181	0.919	**0.866**	0.828	0.904	**0.908**	**0.885**
E	0.345	**0.190**	0.220	0.899	**0.904**	0.810	0.906	0.895	**0.883**
V	**0.342**	0.209	**0.234**	0.913	0.808	**0.888**	0.908	0.899	**0.883**
H	0.352	0.227	0.183	**0.922**	0.830	0.848	0.902	**0.911**	0.882
Ñ	0.351	**0.187**	0.202	0.919	0.843	0.832	0.895	**0.908**	0.880
N	0.349	0.196	0.193	0.911	0.823	0.857	0.908	0.895	0.879
W	0.345	0.201	0.220	**0.928**	0.803	**0.870**	0.908	0.886	0.879
D	0.348	0.218	0.192	0.897	0.857	0.821	0.886	**0.908**	0.874
F	0.353	**0.183**	0.199	0.899	0.841	0.792	**0.928**	0.906	0.873
I	0.354	0.228	0.171	0.911	0.857	0.814	0.893	0.890	0.873
L	0.346	0.194	0.216	0.906	0.812	0.864	0.897	0.888	0.873
M	0.345	0.206	0.222	0.913	0.817	0.861	0.879	0.895	0.873
A	0.344	0.201	0.225	0.895	0.846	0.830	0.884	0.906	0.872
T	0.345	0.210	0.221	0.904	0.796	0.852	0.902	0.895	0.870
S	**0.343**	0.212	**0.229**	0.902	0.772	**0.870**	0.906	0.897	0.869
K	0.346	0.204	0.220	**0.922**	0.783	0.855	0.902	0.877	0.868
J	0.345	0.228	0.207	0.904	0.765	0.841	**0.919**	0.904	0.867
U	0.347	0.219	0.201	0.897	0.761	0.859	0.902	0.906	0.865
G	0.350	0.230	0.188	0.886	0.817	0.826	0.877	0.904	0.862
Q	**0.341**	0.218	**0.232**	0.902	0.765	0.834	0.904	0.899	0.861
Z	0.344	0.224	0.217	0.904	0.761	0.830	0.902	0.897	0.859
P	0.348	0.225	0.188	0.902	0.770	0.830	0.886	0.897	0.857
R	0.351	0.237	0.182	0.904	0.772	0.828	0.890	0.875	0.854
Y	0.351	0.236	0.185	0.893	0.761	0.839	0.893	0.870	0.851
X	0.354	0.243	0.156	0.881	0.745	0.852	0.897	0.877	0.851
O	0.356	0.245	0.155	0.915	0.749	0.801	0.875	0.884	0.845

Cuadro 7.1: Resultados de modelos LSTM, GRU y CNN-1D con características mLeNet en conjunto Validación

Modelo	RMSE	MAE	R2	5 min	15 min	30 min	45 min	60 min	Promedio
D	0.338	0.186	0.414	0.957	0.884	**0.973**	**0.917**	0.913	**0.929**
F	0.343	0.168	0.386	0.928	0.879	**0.969**	**0.913**	0.904	**0.919**
C	0.332	**0.166**	0.430	0.893	**0.893**	**0.971**	**0.915**	**0.926**	**0.919**
Ñ	0.328	**0.163**	0.443	0.931	0.881	0.962	0.911	0.908	**0.919**
N	0.323	0.167	0.467	**0.966**	0.832	0.964	**0.913**	**0.922**	**0.919**
B	0.330	**0.166**	0.436	0.955	**0.893**	0.940	0.904	0.899	0.918
M	0.323	0.179	0.458	0.960	**0.893**	0.951	0.859	**0.922**	0.917
W	0.327	0.174	0.452	0.944	0.859	0.966	0.904	0.899	0.915
I	0.327	0.173	0.458	0.933	0.855	0.964	0.906	0.911	0.914
L	0.324	0.168	0.460	0.960	0.848	0.957	0.872	**0.922**	0.912
E	0.336	0.172	0.410	0.890	**0.897**	0.955	0.911	0.904	0.911
H	0.335	0.185	0.418	0.949	0.877	0.955	0.857	0.919	0.911
V	0.327	0.184	0.447	**0.969**	0.879	0.953	0.870	0.875	0.909
K	0.322	0.177	0.471	**0.971**	0.855	0.946	0.870	0.895	0.907
G	0.327	0.194	0.461	0.955	0.868	0.928	0.872	0.899	0.905
A	0.333	0.182	0.423	0.899	0.881	0.931	0.899	0.915	0.905
T	0.323	0.183	0.469	0.955	0.843	0.962	0.861	0.875	0.899
P	**0.318**	0.189	**0.478**	0.955	0.859	0.946	0.859	0.848	0.894
S	0.323	0.193	0.469	0.955	0.826	0.955	0.859	0.864	0.892
J	**0.320**	0.191	**0.476**	0.964	0.846	0.937	0.864	0.839	0.890
Q	0.324	0.187	0.465	**0.966**	0.837	0.951	0.857	0.834	0.889
U	**0.320**	0.192	**0.474**	0.957	0.846	0.953	0.852	0.832	0.888
Z	0.324	0.198	0.456	0.957	0.823	0.946	0.866	0.837	0.886
Y	0.321	0.209	0.469	0.955	0.839	0.940	0.857	0.817	0.881
O	0.324	0.216	0.457	0.940	0.866	0.928	0.859	0.812	0.881
R	**0.319**	0.205	0.470	0.946	0.848	0.928	0.850	0.814	0.877
X	**0.320**	0.212	0.469	0.937	0.834	0.913	0.859	0.826	0.874

Cuadro 7.2: Resultados de modelos LSTM, GRU y CNN-1D con características mLeNet en conjunto Prueba

Modelo	RMSE	MAE	R2	5 min	15 min	30 min	45 min	60 min	Promedio
F	**0.321**	**0.158**	**0.311**	0.893	0.877	**0.919**	**0.928**	**0.911**	**0.906**
B	0.322	0.175	0.301	0.897	0.902	**0.922**	0.884	**0.913**	**0.903**
D	**0.321**	0.171	**0.319**	0.890	**0.917**	0.908	0.881	**0.913**	**0.902**
C	0.324	**0.166**	0.306	0.870	**0.942**	0.870	0.895	0.904	0.896
H	0.325	0.182	0.295	**0.899**	0.884	**0.915**	0.881	0.897	0.895
L	0.328	0.179	0.288	**0.899**	0.830	0.902	0.913	0.904	0.889
A	0.325	0.185	0.291	**0.906**	0.859	0.895	0.879	0.904	0.889
W	0.325	0.184	0.304	0.888	0.846	0.888	0.911	0.906	0.888
V	0.332	0.189	0.257	0.895	0.841	0.902	0.897	0.902	0.887
I	0.328	0.176	0.283	0.881	**0.908**	0.872	0.864	**0.911**	0.887
Ñ	0.329	0.171	0.285	0.884	0.870	0.857	0.919	0.904	0.887
E	**0.320**	**0.163**	**0.316**	0.884	0.902	0.832	0.908	0.899	0.885
N	0.327	0.177	0.295	0.877	0.861	0.864	0.919	0.904	0.885
T	0.332	0.198	0.267	0.884	0.826	0.872	0.924	0.908	0.883
K	0.329	0.194	0.281	0.895	0.826	0.877	0.911	0.904	0.882
M	0.329	0.197	0.281	0.879	0.834	0.897	0.897	0.899	0.881
J	0.337	0.209	0.228	0.881	0.787	0.881	**0.931**	**0.911**	0.878
U	0.338	0.217	0.226	0.893	0.787	0.859	**0.933**	0.899	0.874
G	0.340	0.193	0.251	0.895	0.826	0.884	0.850	0.908	0.872
S	0.336	0.205	0.235	0.886	0.781	0.861	0.926	0.908	0.872
O	0.345	0.234	0.183	0.893	0.783	0.877	0.904	0.879	0.867
Q	0.336	0.201	0.239	0.881	0.799	0.834	0.906	0.908	0.866
R	0.345	0.224	0.193	**0.904**	0.781	0.872	0.908	0.866	0.866
Z	0.337	0.215	0.217	0.881	0.785	0.850	0.919	0.895	0.866
Y	0.345	0.228	0.196	0.886	0.796	0.875	0.886	0.866	0.862
P	0.341	0.215	0.226	0.895	0.776	0.879	0.879	0.872	0.860
X	0.348	0.243	0.184	0.897	0.776	0.866	0.890	0.864	0.859

Cuadro 7.3: Resultados de modelos LSTM, GRU y CNN-1D con características mAlexNet en conjunto Validación

Modelo	RMSE	MAE	R2	5 min	15 min	30 min	45 min	60 min	Promedio
Ñ	0.308	0.150	0.513	0.964	0.940	0.982	**0.917**	**0.926**	**0.946**
B	**0.303**	0.148	**0.530**	0.935	**0.951**	0.973	**0.913**	0.919	**0.938**
W	0.310	0.163	0.511	0.969	0.926	0.971	0.902	**0.924**	**0.938**
I	0.310	0.152	0.509	0.962	0.942	0.980	0.879	0.919	0.936
N	0.306	0.155	0.514	0.917	**0.951**	0.980	0.897	**0.924**	0.934
F	0.311	**0.142**	0.516	0.915	**0.964**	**0.984**	0.875	**0.924**	0.932
L	0.307	0.153	0.522	**0.975**	0.908	0.969	0.879	**0.926**	0.932
D	0.307	0.156	0.515	0.897	**0.957**	**0.987**	**0.915**	0.902	0.932
C	0.309	**0.147**	0.504	0.902	0.949	**0.984**	0.902	0.913	0.930
A	**0.304**	0.157	0.516	0.949	**0.951**	0.971	0.870	0.908	0.930
E	**0.301**	**0.138**	**0.534**	0.915	0.944	0.969	0.899	0.915	0.928
H	0.306	0.155	0.513	0.949	0.899	0.982	0.881	0.913	0.925
M	0.309	0.171	0.511	0.971	0.915	0.966	0.875	0.895	0.924
K	0.306	0.162	**0.526**	**0.975**	0.895	0.962	0.861	**0.924**	0.923
V	0.306	0.159	0.519	0.964	0.906	0.960	0.875	0.913	0.923
G	0.311	0.162	0.498	0.955	0.895	0.957	0.857	0.913	0.915
Q	0.312	0.170	0.502	0.964	0.888	0.957	0.864	0.895	0.914
T	0.309	0.164	0.513	0.969	0.855	0.969	0.872	0.884	0.910
S	0.312	0.170	0.505	0.971	0.830	0.957	0.866	0.902	0.905
J	0.312	0.175	0.500	0.960	0.875	0.944	0.857	0.879	0.903
Z	0.311	0.177	0.509	0.964	0.830	0.964	0.855	0.879	0.898
O	0.310	0.194	0.504	**0.973**	0.837	0.955	0.866	0.852	0.897
U	0.315	0.182	0.494	0.969	0.837	0.951	0.864	0.864	0.897
Y	0.312	0.186	0.502	0.966	0.848	0.944	0.861	0.848	0.894
P	0.314	0.182	0.494	0.955	0.839	0.949	0.864	0.850	0.891
R	0.312	0.185	0.499	0.966	0.837	0.949	0.861	0.837	0.890
X	0.311	0.203	0.498	0.966	0.841	0.940	0.864	0.837	0.889

Cuadro 7.4: Resultados de modelos LSTM, GRU y CNN-1D con características mAlexNet en conjunto Prueba

Modelo	RMSE	MAE	R2	5 min	15 min	30 min	45 min	60 min	Promedio
E	**0.337**	**0.176**	**0.256**	0.897	**0.834**	**0.919**	**0.911**	**0.908**	**0.894**
D	0.340	0.180	0.239	0.908	**0.846**	**0.908**	0.906	0.904	**0.894**
B	0.343	**0.177**	0.238	0.913	**0.852**	**0.908**	0.886	0.906	**0.893**
H	0.340	0.201	0.241	0.922	0.817	0.904	0.881	**0.908**	0.886
F	0.348	**0.170**	0.216	0.915	0.805	**0.917**	0.895	0.893	0.885
A	**0.339**	0.190	**0.258**	**0.926**	0.821	0.888	0.884	0.906	0.885
I	**0.337**	0.189	**0.262**	0.913	0.828	0.902	0.877	0.906	0.885
C	0.343	**0.177**	0.234	0.911	0.805	0.902	0.886	0.897	0.880
Ñ	0.345	0.179	0.220	0.911	0.819	0.877	0.893	0.902	0.880
N	0.343	0.182	0.233	0.913	0.799	0.893	0.888	**0.908**	0.880
L	0.342	0.190	0.238	0.899	0.801	0.866	**0.915**	0.904	0.877
T	0.341	0.205	0.240	0.917	0.792	0.875	0.877	**0.908**	0.874
W	0.340	0.188	0.248	0.904	0.779	0.884	0.897	0.902	0.873
K	0.342	0.205	0.209	0.895	0.796	0.875	0.890	0.904	0.872
V	0.341	0.198	0.234	0.895	0.772	0.888	0.902	0.906	0.872
M	0.344	0.198	0.212	0.906	0.772	0.870	0.897	**0.908**	0.871
G	0.343	0.203	0.236	**0.926**	0.794	0.872	0.857	0.904	0.871
Q	0.342	0.208	0.223	0.908	0.761	0.881	0.890	**0.908**	0.870
U	0.343	0.219	0.194	0.908	0.756	0.875	**0.908**	0.904	0.870
S	0.341	0.207	0.221	0.899	0.772	0.886	0.886	**0.908**	0.870
J	0.344	0.212	0.208	0.906	0.770	0.875	0.890	0.904	0.869
P	0.346	0.215	0.187	0.897	0.756	0.888	0.899	0.899	0.868
Z	0.348	0.210	0.188	0.897	0.749	0.866	0.879	0.904	0.859
R	0.351	0.225	0.181	0.917	0.765	0.861	0.852	0.888	0.857
X	0.352	0.242	0.155	0.924	0.745	0.864	0.855	0.886	0.855
Y	0.353	0.224	0.167	0.913	0.758	0.848	0.852	0.884	0.851
O	0.352	0.237	0.174	**0.931**	0.745	0.855	0.850	0.870	0.850

Cuadro 7.5: Resultados de modelos LSTM, GRU y CNN-1D con características InceptionV3 en conjunto Validación

Modelo	RMSE	MAE	R2	5 min	15 min	30 min	45 min	60 min	Promedio
C	**0.315**	**0.149**	0.484	0.911	**0.937**	0.962	0.908	0.933	**0.930**
Ñ	**0.315**	0.151	0.489	0.919	0.928	0.964	0.906	0.931	**0.930**
F	0.317	**0.146**	0.484	0.919	0.913	0.960	0.906	**0.942**	**0.928**
E	0.316	**0.150**	0.489	0.893	**0.937**	**0.978**	**0.913**	0.919	**0.928**
D	**0.310**	0.154	**0.500**	0.933	0.924	0.971	0.897	0.911	0.927
N	**0.315**	0.157	0.486	0.940	0.899	**0.973**	0.890	0.926	0.926
W	0.319	0.165	0.484	0.917	0.899	0.969	**0.911**	**0.935**	0.926
B	0.317	0.154	0.476	0.902	**0.935**	0.957	**0.917**	0.902	0.923
I	0.317	0.159	0.483	0.942	0.904	0.962	0.868	**0.935**	0.922
A	0.317	0.168	0.489	0.933	0.895	0.955	0.902	0.908	0.919
K	0.316	0.170	0.496	0.937	0.886	0.960	0.870	0.926	0.916
L	0.316	0.160	0.494	0.917	0.895	**0.973**	0.888	0.904	0.915
V	0.320	0.172	0.480	0.926	0.893	0.962	0.875	0.917	0.915
G	0.320	0.175	0.479	0.919	0.913	0.962	0.870	0.895	0.912
M	**0.315**	0.169	**0.503**	0.940	0.893	0.966	0.884	0.872	0.911
H	**0.315**	0.167	0.486	0.922	0.899	0.953	0.893	0.886	0.911
Q	0.320	0.181	0.481	0.946	0.881	0.960	0.881	0.846	0.903
U	**0.315**	0.182	0.498	0.951	0.870	0.962	0.855	0.872	0.902
J	0.316	0.181	0.493	**0.953**	0.895	0.953	0.866	0.826	0.898
Y	**0.315**	0.189	0.489	**0.962**	0.857	0.949	0.879	0.839	0.897
P	**0.314**	0.181	**0.500**	**0.955**	0.861	0.964	0.868	0.834	0.897
T	0.322	0.177	0.476	0.940	0.855	0.964	0.872	0.855	0.897
S	0.318	0.178	0.490	0.951	0.866	0.962	0.861	0.846	0.897
Z	0.318	0.182	0.480	0.949	0.875	0.949	0.879	0.832	0.897
O	0.318	0.207	0.476	**0.953**	0.857	0.937	0.870	0.828	0.889
X	0.316	0.206	0.488	0.951	0.846	0.935	0.877	0.828	0.887
R	0.317	0.196	0.489	0.951	0.834	0.946	0.872	0.823	0.885

Cuadro 7.6: Resultados de modelos LSTM, GRU y CNN-1D con características InceptionV3 en conjunto Prueba

Modelo	RMSE	MAE	R2	5 min	15 min	30 min	45 min	60 min	Promedio
K	0.336	0.200	0.230	0.890	0.857	**0.902**	**0.926**	0.911	**0.897**
C	0.334	0.187	0.242	0.908	**0.881**	0.864	0.904	**0.915**	**0.894**
F	0.335	**0.171**	0.265	**0.924**	**0.881**	0.861	0.881	**0.917**	**0.893**
V	0.334	0.198	0.246	0.908	0.817	**0.899**	0.913	0.906	0.889
M	0.334	0.210	0.239	0.904	0.819	**0.893**	0.913	0.908	0.887
B	**0.326**	0.187	**0.284**	0.899	**0.879**	0.828	0.917	0.911	0.887
N	**0.332**	0.185	0.268	0.906	0.828	0.881	0.917	0.904	0.887
E	**0.331**	**0.173**	**0.278**	0.906	0.837	0.870	0.908	0.911	0.886
L	0.335	0.186	0.243	0.913	0.828	0.866	0.908	0.908	0.885
W	0.335	0.188	0.244	0.904	0.823	0.864	0.919	0.911	0.884
T	0.335	0.201	0.234	0.899	0.821	0.866	**0.922**	0.911	0.884
D	**0.332**	0.191	0.250	0.911	0.846	0.866	0.897	0.895	0.883
Ñ	0.338	**0.176**	0.235	0.893	0.819	0.879	0.913	0.906	0.882
I	0.333	0.190	**0.283**	**0.919**	0.828	0.877	0.872	0.906	0.881
S	0.333	0.210	0.239	0.899	0.810	0.861	0.919	0.911	0.880
Q	0.336	0.208	0.225	0.881	0.810	0.870	**0.924**	0.911	0.879
A	0.333	0.203	0.258	0.904	0.817	0.852	0.897	**0.913**	0.877
P	0.338	0.221	0.202	0.904	0.783	0.859	0.915	0.906	0.873
J	0.337	0.214	0.226	0.902	0.783	0.859	0.908	0.906	0.872
H	0.337	0.200	0.250	**0.935**	0.812	0.832	0.866	**0.913**	0.872
U	0.335	0.217	0.226	0.888	0.774	0.864	0.919	0.906	0.870
G	0.340	0.220	0.222	0.913	0.812	0.832	0.877	0.906	0.868
Z	0.338	0.212	0.201	0.890	0.781	0.826	0.917	0.904	0.864
O	0.349	0.237	0.151	0.893	0.770	0.846	0.906	0.881	0.859
R	0.348	0.227	0.150	0.890	0.765	0.837	0.911	0.888	0.858
X	0.352	0.243	0.136	0.893	0.745	0.837	0.911	0.884	0.854
Y	0.344	0.226	0.187	0.895	0.763	0.841	0.886	0.879	0.853

Cuadro 7.7: Resultados de modelos LSTM, GRU y CNN-1D con características MobileNetV2 en conjunto Validación

Modelo	RMSE	MAE	R2	5 min	15 min	30 min	45 min	60 min	Promedio
D	**0.299**	0.150	**0.545**	**0.971**	**0.922**	**0.982**	**0.913**	0.924	**0.942**
Ñ	0.310	**0.146**	0.519	**0.971**	**0.911**	**0.982**	0.897	**0.933**	**0.939**
W	0.310	0.155	0.512	0.960	0.906	0.980	**0.913**	0.926	**0.937**
C	**0.304**	**0.146**	**0.531**	0.915	**0.915**	0.969	0.906	**0.928**	0.927
B	0.308	0.154	0.519	0.922	0.904	0.975	0.908	0.926	0.927
I	0.314	0.160	0.490	0.931	0.908	0.980	0.893	0.924	0.927
A	**0.307**	0.168	0.515	0.949	0.908	0.971	0.884	0.917	0.926
N	0.310	0.157	0.516	0.951	0.884	0.980	0.899	0.917	0.926
L	0.308	0.155	0.519	0.922	0.893	0.980	0.904	0.924	0.924
F	0.320	0.155	0.470	0.877	0.908	**0.984**	**0.911**	**0.935**	0.923
H	0.310	0.164	0.508	0.931	**0.911**	0.978	0.870	0.922	0.922
G	**0.307**	0.171	0.514	0.951	0.893	0.973	0.864	**0.928**	0.922
K	0.309	0.164	0.517	0.964	0.879	0.980	0.864	0.922	0.922
J	0.311	0.184	0.510	**0.973**	0.908	0.966	0.855	0.899	0.920
V	0.312	0.164	0.515	0.957	0.850	0.978	0.870	0.924	0.916
M	0.309	0.169	0.519	0.953	0.902	0.971	0.859	0.888	0.915
T	0.310	0.165	0.522	0.955	0.848	0.962	0.888	0.915	0.914
E	0.308	**0.149**	0.507	0.872	0.899	0.975	0.893	0.926	0.913
Z	0.309	0.179	0.516	**0.971**	0.870	0.966	0.857	0.895	0.912
Q	0.309	0.172	0.523	0.969	0.850	0.964	0.859	0.913	0.911
P	0.309	0.182	0.517	0.964	0.879	0.969	0.859	0.870	0.908
U	**0.307**	0.181	0.522	**0.971**	0.886	0.969	0.859	0.828	0.902
S	0.308	0.176	**0.524**	0.964	0.848	0.966	0.852	0.859	0.898
R	**0.307**	0.186	0.520	0.964	0.864	0.960	0.855	0.841	0.897
O	0.309	0.195	0.509	0.949	0.852	0.953	0.846	0.830	0.886
Y	0.310	0.191	0.509	0.949	0.848	0.949	0.843	0.830	0.884
X	0.309	0.200	0.509	0.951	0.834	0.949	0.848	0.839	0.884

Cuadro 7.8: Resultados de modelos LSTM, GRU y CNN-1D con características MobileNetV2 en conjunto Prueba

Modelo	RMSE	MAE	R2	5 min	15 min	30 min	45 min	60 min	Promedio
I	**0.385**	0.249	**0.018**	**0.884**	**0.783**	**0.895**	0.888	0.888	**0.868**
A	**0.374**	0.248	**0.018**	0.861	**0.787**	**0.895**	**0.902**	0.875	**0.864**
F	0.391	**0.241**	-0.042	0.857	**0.805**	**0.897**	0.890	0.852	**0.860**
C	**0.385**	**0.232**	0.001	0.850	0.772	0.855	0.890	**0.915**	0.856
Ñ	**0.376**	0.254	**0.031**	**0.868**	0.743	0.868	0.868	0.904	0.850
K	0.387	0.283	-0.053	0.866	0.723	0.861	0.888	0.904	0.848
X	0.403	0.305	-0.170	0.832	0.756	0.846	**0.911**	0.870	0.843
Y	0.394	0.294	-0.102	0.828	0.723	0.850	**0.897**	0.895	0.838
S	0.386	0.284	-0.049	0.857	0.698	0.839	0.888	**0.908**	0.838
P	0.397	0.297	-0.090	0.857	0.725	0.823	0.877	0.902	0.837
G	**0.385**	0.257	-0.020	**0.879**	0.705	0.819	0.870	0.886	0.832
E	0.406	0.269	-0.121	0.852	0.669	0.843	0.877	0.899	0.828
O	0.405	0.304	-0.170	0.814	0.700	0.823	**0.897**	0.890	0.825
R	0.404	0.302	-0.148	0.803	0.729	0.808	0.881	0.895	0.823
V	0.395	0.280	-0.088	0.812	0.718	0.821	0.864	0.893	0.821
T	0.395	0.294	-0.094	0.808	0.696	0.801	0.872	**0.913**	0.818
W	0.388	0.278	-0.028	0.832	0.615	0.866	0.877	0.902	0.818
N	0.398	0.280	-0.103	0.826	0.682	0.830	0.864	0.886	0.817
J	0.405	0.303	-0.156	0.790	0.723	0.805	0.864	0.904	0.817
B	0.394	**0.247**	-0.015	0.852	0.678	0.779	0.893	0.868	0.814
M	0.400	0.294	-0.111	0.841	0.613	0.837	0.857	0.897	0.809
D	0.397	0.283	-0.138	0.839	0.642	0.767	0.893	0.895	0.807
H	0.424	0.282	-0.213	0.752	0.705	0.870	0.852	0.832	0.802
U	0.418	0.317	-0.238	0.770	0.705	0.765	0.872	0.888	0.800
Z	0.414	0.316	-0.209	0.785	0.629	0.796	0.870	0.895	0.795
L	0.400	0.286	-0.093	0.814	0.631	0.803	0.852	0.870	0.794
Q	0.410	0.304	-0.198	0.779	0.649	0.792	0.870	0.870	0.792

Cuadro 7.9: Resultados de modelos LSTM, GRU y CNN-1D con características MobileNetV3 Small personalizado en conjunto Validación

Modelo	RMSE	MAE	R2	5 min	15 min	30 min	45 min	60 min	Promedio
A	0.354	0.217	0.362	**0.879**	**0.897**	**0.913**	0.839	0.796	**0.865**
E	0.352	0.210	0.366	0.855	0.783	**0.949**	0.864	**0.864**	**0.863**
G	0.358	0.220	0.340	0.826	0.886	0.899	0.855	**0.812**	**0.855**
H	0.350	**0.206**	0.378	**0.897**	0.843	0.866	0.864	0.803	**0.855**
S	**0.343**	0.222	**0.392**	0.857	**0.897**	0.895	0.850	0.767	0.853
I	0.361	0.222	0.345	0.855	0.817	0.902	0.881	0.805	0.852
O	**0.343**	0.224	**0.386**	**0.861**	0.888	0.850	0.868	0.776	0.849
F	0.370	**0.208**	0.301	0.817	0.864	0.888	0.870	0.792	0.846
C	0.370	**0.203**	0.316	0.830	0.774	**0.919**	**0.884**	**0.821**	0.846
R	0.350	0.225	0.360	0.843	**0.890**	0.841	0.861	0.770	0.841
U	0.346	0.218	0.375	0.834	0.861	0.837	0.872	0.792	0.839
P	0.353	0.230	0.351	0.837	0.879	0.828	**0.884**	0.767	0.839
X	0.353	0.236	0.341	0.843	0.855	0.837	0.868	0.781	0.837
N	0.355	0.214	0.356	0.830	0.850	0.868	0.864	0.767	0.836
J	0.349	0.227	0.378	0.841	0.834	0.859	0.866	0.776	0.835
V	0.353	0.220	0.358	0.852	0.817	0.857	0.879	0.772	0.835
Q	0.352	0.228	0.365	0.839	0.814	0.872	0.875	0.770	0.834
D	0.357	0.209	0.345	0.826	0.799	0.857	**0.895**	0.792	0.834
W	0.355	0.216	0.368	0.823	0.817	**0.913**	0.855	0.756	0.833
T	0.355	0.231	0.344	0.837	0.870	0.848	0.850	0.758	0.833
L	0.357	0.219	0.350	0.828	0.817	0.857	0.881	0.772	0.831
Y	0.355	0.234	0.343	0.846	0.864	0.814	0.855	0.772	0.830
Z	**0.343**	0.230	**0.399**	0.859	0.808	0.843	0.848	0.790	0.830
Ñ	0.357	0.216	0.354	0.823	0.805	0.890	0.870	0.756	0.829
M	0.353	0.221	0.362	0.839	0.817	0.826	0.859	0.763	0.821
K	0.357	0.225	0.342	0.837	0.803	0.830	0.864	0.763	0.819
B	0.374	0.216	0.265	0.808	0.799	0.837	0.872	0.761	0.815

Cuadro 7.10: Resultados de modelos LSTM, GRU y CNN-1D con características MobileNetV3 Small personalizado en conjunto Prueba

Bibliografía

[1] Amato, G.; Carrara, F.; Falchi, F.; Gennaro, C.; Vairo, C. (2016). Car parking occupancy detection using smart camera networks and Deep Learning. In *2016 IEEE Symposium on Computers and Communication (ISCC)*, Messina, Italy, pp. 1212-1217. doi: 10.1109/ISCC.2016.7543901.

[2] Amato, G.; Carrara, F.; Falchi, F.; Gennaro, C.; Meghini, C.; Vairo, C. (2017). Deep learning for decentralized parking lot occupancy detection. *Expert Systems with Applications*, 72:327-334.

[3] Ayala, D.; Wolfson, O.; Xu, B.; DasGupta, B.; Lin, J. (2012). Pricing of parking for congestion reduction. In *Proceedings of the 20th International Conference on Advances in Geographic Information Systems*, pp. 43–51. doi: 10.1145/2424321.2424328.

[4] Chen, J.; Su, W.; Wang, Z. (2020). Crowd counting with crowd attention convolutional neural network. *Neurocomputing*, 382:210-220.

[5] Chen, L.-C.; Sheu, R.-K.; Peng, W.-Y.; Wu, J.-H.; Tseng, C.-H. (2020). Video-Based Parking Occupancy Detection for Smart Control System. *Applied Sciences*, 10:1079.

[6] Chin, V. (2018). Unlocking Cities. [Online]. Available: https://www.bcg.com/d/news/19april2018-traffic-jams-maybecausing-22-billion-loss-india-191853.

[7] Cho, K.; Van Merriënboer, B.; Bahdanau, D.; Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. arXiv preprint arXiv:1406.1078.

[8] Courbariaux, M.; Hubara, I.; Soudry, D.; El-Yaniv, R.; Bengio, Y. (2016). Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1. arXiv preprint arXiv:1602.02830.

[9] De Almeida, P. R. L.; Oliveira, L. S.; Britto, A. S.; Silva, E. J.; Koerich, A. L. (2015). PKLot – A robust dataset for parking lot classification. *Expert Systems with Applications*, 42(11): 4937-4949. doi: 10.1016/j.eswa.2015.02.009.

[10] Fan, J.; Hu, Q.; Tang, Z. (2018). Predicting vacant parking space availability: An SVR method with fruit fly optimisation. *IET Intelligent Transport Systems*, 12(10):1414–1420. doi: 10.1049/iet-its.2018.5031.

[11] García Velasco, A. (2023). MLOps: Administrando el Diseño y Ciclo de Vida de los Modelos de Machine Learning. [Online]. Available: https://oa.upm.es/74985/.

[12] Giuffrè, T.; Siniscalchi, S. M.; Tesoriere, G. (2012). A novel architecture of parking management for smart cities. *Procedia Social and Behavioral Sciences*, 53:16–28. doi: 10.1016/j.sbspro.2012.09.856.

[13] Goodfellow, I.; Bengio, Y.; Courville, A. (2016). *Deep Learning*. Massachusetts: MIT Press.

[14] He, K.; Zhang, X.; Ren, S.; Sun, J. (2016). Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[15] Heaton, J. (2018). Goodfellow, I.; Bengio, Y.; Courville, A.: Deep Learning. *Genetic Programming and Evolvable Machines*, 19(1): 305-307. doi: 10.1007/s10710-017-9314-z.

[16] Hochreiter, S.; Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8):1735-1780. DOI: 10.1162/neco.1997.9.8.1735.

[17] Howard, A.; Sandler, M.; Chu, G.; Chen, L.-C.; Chen, B.; Tan, M.; Wang, W.; Zhu, Y.; Pang, R.; Vasudevan, V.; Le, Q. V.; Adam, H. (2019). Searching for MobileNetV3. arXiv preprint arXiv:1905.02244.

[18] Huang, C. C.; Tai, Y. S.; Wang, S. J. (2013). Vacant parking space detection based on plane-based Bayesian hierarchical framework. *IEEE Transactions on Circuits and Systems for Video Technology*, 23(9):1598–1610. doi: 10.1109/TCSVT.2013.2254961.

[19] Krizhevsky, A.; Sutskever, I.; Hinton, G. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *Neural Information Processing Systems*, 25. doi: 10.1145/3065386.

[20] Klappenecker, A., Lee, H., Welch, J. L. (2010). Finding available parking spaces made easy. In *Proceedings of the 6th International Workshop on Foundations of Mobile Computing* (DIALM-POMC '10) (pp. 49–52). Association for Computing Machinery. doi: 10.1145/1860684.1860696.

[21] Lecun, Y.; Bottou, L.; Bengio, Y.; Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278-2324. doi: 10.1109/5.726791.

[22] Li, L. B.; Li, Y. (2021). Free prediction of parking demand for refined management. *Journal of Tongji University*, 49(9): 1301-1306.

[23] Mehta, P.; Bukov, M.; Wang, C.-H.; Day, A. G. R.; Richardson, C.; Fisher, C. K.; Schwab, D. J. (2019). A high-bias, low-variance introduction to Machine Learning for physicists. *Physics Reports*, 810, 1–124. doi: 10.1016/j.physrep.2019.03.001.

[24] Minsky, M. L.; Papert, S. A. (1969). *Perceptrons: An Introduction to Computational Geometry*. The MIT Press.

[25] Ng, A. Y. (2004). Feature selection, L1 vs. L2 regularization, and rotational invariance. In *Proceedings of the Twenty-First International Conference on Machine learning (ICML '04)*. ACM, New York, NY, USA, 78. doi: 10.1145/1015330.1015435

[26] Ng, C. K.; Cheong, S. N.; Foo, Y. L. (2020). Low latency deep learning based parking occupancy detection by exploiting structural similarity. In *Computational Science and Technology*, pp. 247-256. Springer, Singapore.

[27] Nurullayev, S.; Lee, S.-W. (2019). Generalized Parking Occupancy Analysis Based on Dilated Convolutional Neural Network. *Sensors*, 19(2):277.

[28] Rianto, D.; Erwin, I. M.; Prakasa, E.; Herlan, H. (2019). Parking Slot Identification using Local Binary Pattern and Support Vector Machine. In *2018 International Conference on Computer, Control, Informatics and its Applications*, pp. 129–133. doi: 10.1109/IC3INA.2018.8629530.

[29] Rosenblatt, F. (1957). The Perceptron - A Perceiving and Recognizing Automaton. *Cornell Aeronautical Laboratory*.

[30] Sandler, M.; Howard, A.; Zhu, M.; Zhmoginov, A.; Chen, L.-C. (2018). Inverted Residuals and Linear Bottlenecks: Mobile Networks for Classification, Detection, and Segmentation. arXiv preprint arXiv:1801.04381.

[31] Shakhnarovich, G. (2011). Notes on derivation of bias-variance decomposition in linear regression. *University of Chicago*.

[32] Shao, W.; Zhang, Y.; Guo, B.; Qin, K.; Chan, J.; Salim, F. D. (2018). Parking availability prediction with long short term memory model. In *Proc. ICGPC*. Cham, Switzerland: Springer, pp. 124-137.

[33] Sharma, S.; Sen, S. (2020). One-dimensional convolutional neural network-based damage detection in structural joints. *Journal of Civil Structural Health Monitoring*, 10: 1057–1072. doi: 10.1007/s13349-020-00434-z.

[34] Shoup, D. C. (2006). Cruising for parking. *Transport Policy*, 13(6):479–486. doi: 10.1016/j.tranpol.2006.05.005.

[35] Simonyan, K.; Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv preprint arXiv:1409.1556.

[36] Song, H.; Liang, H.; Li, H.; Dai, Z.; Yun, X. (2019). Vision-based vehicle detection and counting system using deep learning in highway scenes. *European Transport Research Review*, 11(1):51.

[37] Srivastava, N.; Hinton, G.; Krizhevsky, A.; Sutskever, I.; Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15: 1929–1958.

[38] Sukarno, I.; Matsumoto, H.; Susanti, L. (2016). Transportation energy consumption and emissions-a view from city of Indonesia. *Future Cities and Environment*, 2:6.

[39] Szegedy, C.; Vanhoucke, V.; Ioffe, S.; Shlens, J.; Wojna, Z. (2015). Rethinking the Inception Architecture for Computer Vision. arXiv preprint arXiv:1512.00567.

[40] Tu, L.; Ma, Z.; Huang, B. (2019). Analysis and Prediction of Differential Parking Behaviors. In *2019 International Conference on Cyber Science and Technology Congress*, pp. 44-49. doi: 10.1109/DASC/PiCom/CBDCom/CyberSciTech.2019.00023.

[41] Uttara, S.; Bhuvandas, N.; Aggarwal, V. (2012). Impacts of urbanization on environment. *International Journal of Research in Engineering and Applied Sciences*, 2:1637–1645.

[42] Wang, Z.; Xue, W.; Hei, X. (2019). Research on parking space prediction based on long short-term memory. In *Proc. ICICAS*, Dec. 2019, pp. 392-396.

[43] Xiao, J.; Lou, Y.; Frisby, J. (2018). How likely am I to find parking?—A practical model-based framework for predicting parking availability. *Transportation Research Part B: Methodological*, 112:19–39. doi: 10.1016/j.trb.2018.04.001.

[44] Yang, L. L. (2017). Properative analysis and residual vehicle position prediction study on parking data. M.S. thesis, Wuhan Univ., Wuhan, China.

[45] Yang, S.; Ma, W.; Pi, X.; Qian, S. (2019). A deep learning approach to real-time parking occupancy prediction in transportation networks incorporating multiple spatio-temporal data sources. *Transportation Research Part C: Emerging Technologies*, 107:248–265. doi: 10.1016/j.trc.2019.08.010.

[46] Yuldashev, Y.; Mukhiddinov, M.; Abdusalomov, A. B.; Nasimov, R.; Cho, J. (2023). Parking Lot Occupancy Detection with Improved MobileNetV3. *Sensors*, 23: 7642. doi: 10.3390/s23177642.

[47] Zheng, Y.; Rajasegarar, S.; Leckie, C. (2015). Parking availability prediction for sensor-enabled car parks in smart cities. In *Proceedings of the 2015 IEEE 10th International Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP)*, pp. 1–6. doi: 10.1109/ISSNIP.2015.7106902.